

АК Экзамен

Экзаменационные вопросы по курсу "Архитектура компьютера"

Вопросы сформулированы таким образом, чтобы ответ по существу можно было дать за пол минуты -- минуту.

Версия документа: 1.7 (234/234 вопросов)

!Примечание! некоторые вопросы в последний момент были разбиты/объединены, поэтому нумерация вопросов здесь более не совпадает с нумерацией в репозитории (было бы очень долго вручную перенумеровывать все вопросы и ссылки на них)

Ответы на основе курса лекций подготовил: Савелий Тарасов
tg:@TarasovSavely

!! Далее по тексту, текст расположенный в блоке, подобном этому (между двумя горизонтальными линиями), означает информацию, прямо не относящуюся ни к какому из вопросов, но важную для понимания контекста дисциплины

Формат проведения экзамена:

1. Преподаватель формирует билеты для учащихся.
2. Студенты сдают экзамен по очереди. Для каждого студента:
 - озвучивается вопрос, учащийся отвечает на него без подготовки в общем виде; Ответ в общем виде подразумевает краткое изложение "сути" вопроса. Детали и примеры могут быть уточнены в рамках уточняющих вопросов.
 - преподаватель может прервать учащегося, задать уточняющие вопросы или попросить сфокусироваться на одном из его аспектов;
3. Результаты экзамена публикуются в конце экзаменационного дня.

p.s. принимаю донаты на шоколадки и реабилитацию после кучи часов вдумчивого слушания лекций с написанием много тысяч букв 🤔

<1 лекция>

1. Что такое "вычислительная платформа" с точки зрения пользователя? Каков её состав?

Вычислительная платформа - такой набор инструментов, которые позволяют организовать какие-то вычисления, вычислительный процесс.

Классические примеры - C, Python и так далее. Еще более традиционный инструментарий - система команд процессора.

На каждой вычислительной платформе предоставляем пользователю инструментарий чтобы он мог описать вычислительную модель и потом ее запустить.

4 компонента вычислительной платформы:

- Язык
В рамках которого мы формулируем вычислительный процесс (синтаксис Си)
- Наборы типов
Данные, которыми мы оперируем в рамках языка (машинные слова, int, char, pointer)
- Инстансы
Набор примитивных кусочков из которых составляется вычислительный процесс (операция "+", вызов функции и так далее)
- Паттерны
Элемент, объединяющий предыдущие три. Практика, определяющая то, как мы используем вычислительную платформу (недостаточно отвертки, надо еще понимать, какой стороной ее к винту прикладывать). Элемент, выводящий инструмент с уровня технических ресурсов на уровень человеческой деятельности.

2. Какие уровни абстракций относятся к дисциплине "Архитектура компьютера"? Почему границы определены именно так?

В целом вот иерархия уровней платформ/абстракций:

Application Software	Программы
Операционные системы	Драйверы устройств
Архитектура	Регистры инструкций
Микро - архитектура	контроллеры каналы передачи данных
Логика	Сумматоры Память
Цифровые схемы	AND вентили NOT вентили
Аналоговые схемы	Усилители фильтры
Устройства	Транзисторы Диоды
Физика	Электроны

К нашей дисциплине относятся уровни примерно с середины операционных систем до цифровых схем. Все что ниже нас не очень интересует потому что мы не собираемся тут проектировать аналоговые схемы и не очень углубляемся в физику. А операционные системы целиком нам не очень интересны, потому что это уже достаточно высокий уровень и изучать его надо отдельно (и изучать мы его будем позже)

3. В чём заключается отличие Platform-Based Design от проектирования для конкретной платформы?

Традиционно как мы пишем софт? Приходим, берем ЯП, берем либы и фреймворк, переиспользуем этот код и реализовываем свой софт. Но при разработке низкоуровневых систем, с высокими требованиями, ограничением по батарее, сложными условиями эксплуатации мы так сделать не можем - потому что каждый раз, когда мы фиксируем универсальную "вычислительную платформу" у этой универсальности есть большая цена - потому что появляется куча уровней абстракции, внутренних процессов, которые мы не можем контролировать. Platform-Based Design - реализует другой подход, когда разработка системы идет двунаправленная: берем какую-то низкоуровневую вычислительную платформу, на базе нее строим тот самый вычислитель, ту самую платформу которая

будет закрывать наше пространство задач, что называется системной платформой (по сути также вычислительная платформа, выращенная с какой-то базы), это одна сторона. С другой стороны используем эту платформу и ее механизмы под наши задачи.

Если мы говорим о разработке встроенных систем, то такая концепция получается by design (с одной стороны разработать железу, с другой софт который будет этим рулить)

<2 лекция>

Компьютерные системы - любые системы, оснащенные внутренними алгоритмами, зафиксированной логикой работы.

4. Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей? Приведите примеры.

Если есть компьютерная система и в процессе ее разработки огромное влияние оказывает разработка программной составляющей то компьютерная система - программная (с преобладающей программной составляющей)

Причина в том, что тотально снижаются затраты на "железную" составляющую. За счет ее переиспользования, очень редко она разрабатывается под систему, за счет модернизации производства и за счет соответственно роста затрат на программную составляющую.

За редчайшими исключениями любая компьютерная система на самом деле программная система.

Пример: даже когда мы говорим про производство и покупку процессоров, там от 50 до 75% это уже затраты на ПО. Или ноутбуки Apple, железная составляющая занимает слишком сильно меньшую составляющую чем операционная система, которая стоит над этим железом и позволяет так классно им пользоваться.

5. Что такое информационная и управляющая система? Каковы их отличия (функции, ПО, аппаратура)?

Информационная система - те системы, задача которых:

получать данные, преобразовывать/накапливать и выдавать в измененном/обработанном виде (бд в простейшем случае, гугл).

Batch computing / distributed computing /

Важный акцент на скорости - в общем случае чем быстрее - тем лучше.

Управляющая система - те системы, которые направлены и сконцентрированы на взаимодействие с реальным физическим миром с целью контроля или управления посредством *получения данных, преобразования/накопления, и выдачи в измененном/обработанном виде*

Для нее характерны:

1. встроенное исполнение
 - интеграция в реальный мир
 - ограниченные ресурсы (энергия, тепло)
 - специализация функции, специализация платформы, аппаратуры
2. автономная эксплуатация
3. ограниченные вычислительные ресурсы
4. работа в режиме реального времени

6. Что такое "требование реального времени"? Примеры систем. Что противоречит РВ в современных компьютерах?

Если система работает в режиме реального времени, это не значит, что она работает быстро, не значит, что работает абсолютно точно, не значит, что работает без отказов.

Это значит, что система работает **предсказуемо и в срок**

Пример: система управления ГЭС, а именно системы водосброса. Есть системы, которые должны обеспечить открытие водосброса в случае, если внутренние бассейны критически переполняются. Если датчик регистрирует превышение допустимого уровня воды, то в течение 7 минут мы следим за значением, и если оно стабильно высокое и стабильно превышает, только только после этого мы считаем, что вода действительно переполняет и в течение 7 минут мы поэтапно открываем водосброс.

Есть продукты, которые могут выступать как вычислительная платформа и как управляющая система: *ffmpeg*, которая может как

конвертировать видео из одного формата в другой, так и читать файл с диска, выводя на экран в реальном времени.

В современных компьютерах реальному времени противоречит очень многое, потому что все оптимизации, улучшения и так далее, которые в том числе рассматриваются далее по курсу прежде всего направлены на увеличение производительности в среднем, но чаще всего могут вносить неопределенные задержки в реакции системы, система теряет предсказуемость времени выполнения

7. Каковы этапы эволюции информационных систем: от пакетной обработки до облачных систем?

Любой информационной системе свойственно, что чем быстрее - тем лучше. И любое решение, которое может в среднем улучшить время ответа - это хорошо.

Этапы эволюции:

- Пакетная обработка: задачи формируются одна за одной, передаются в систему и обрабатываются также одна за одной
- Распределенное вычисление: мы уходим от единого сервера, в сторону нескольких небольших узлов, которые могут обрабатывать задачи и быть использованы независимо
- Пресональные компьютеры и компьютерные сети - здесь появляется классическая "рабочая станция"
- Появление интернета
- Мобильные сети
- IoT, облачная инфраструктура, вычислительные мощности более не ценность сама по себе, теперь это сервис, услуга, как станок в лизинг

8. Каковы этапы эволюции управляющих систем: от ИУС до КФС?

Этапы эволюции:

ИУС - информационно управляющие системы.

Объект управления и управляющая система непосредственно разделены, и могут быть представлены как два прямо отдельных объекта

ВсС - встроенные системы

Когда компьютеры стали достаточно маленькие их стало возможно конструктивно объединить с объектом управления. В целом все осталось также, просто физически одно располагается внутри другого, конструктивное отличие.

РВсС - распределенные встроенные системы

Пример: наушники с подавлением шума. Мы в принципе не можем себе позволить разделить объект управления с управляющей системой (будут адские задержки) и нам придется поставить контроллер в наушник, который будет заниматься шумоподавлением, максимально близко к управляемому объекту. Теперь мы разрабатываем систему не как монолит, а как распределенную систему, это фундаментальное усложнение.

ВК КФС - киберфизические системы (система не может существовать без управления. Пример: гироскутер, вращающиеся двери)

Во всех прошлых у нас есть объект управления и мы им как-то управляем, а здесь в целом сама система не может существовать без системы управления.

9. Каковы задачи и предмет дисциплины "Системная инженерия"? Какова роль системных инженеров?

Системная инженерия - пытается ответить на вопрос а как делать успешные системы?

Системная инженерия занимается разработкой больших систем. Большая система - это когда мы без генерального инженера не можем собрать систему у кого-то в голове целиком.

Предмет системной инженерии - Audi, Boeing 787, нефтедобывающая платформа. Словом - предмет системной инженерии это большие системы

Роль системного инженера - человек который согласует составные части больших систем (но не как менеджер), отвечает за целостность системы. Согласовывает разработчиков разного профиля, согласовывает процессы, команды разработки.

10. Что такое успешная система? Какие точки зрения (заинтересованные стороны) необходимы для построения успешной системы?

В ЭТОМ ВОПРОСЕ ПОНЯТИЕ "ТОЧКИ ЗРЕНИЯ" НЕ ИСПОЛЬЗУЕТСЯ, вторая часть вопроса подразумевает раскрытие понятия "заинтересованная сторона" и описание, какие заинтересованные стороны стоит учитывать

Успешная система - это та, при реализации которой удовлетворены все значимые заинтересованные стороны (стейкхолдеры).

Под сторонами понимаются все люди, которые имеют власть над нашим проектом, в широком смысле от разработчиков до государства и сертифицирующих органов. Те, кто может повлиять на наш проект.

Как пример - активисты, которые просто против всего, тоже будут являться заинтересованной стороной и иногда бывает полезно учитывать в некотором объеме

Точки зрения на понятие системы:

- система как совокупность частей, которые при этом (!) дают какое-то новое качество - новые свойства, которые позволяют получить что-то новое, чего раньше не было
- система как функциональное место - где мы можем приткнуться эту систему, и получить какую-то пользу
- система как жизненный цикл - откуда система появляется и что нам делать потом

Разработка успешной системы требует:

1. Рассмотрения её структуры
2. Рассмотрения её функционального места
3. Рассмотрения её операционного окружения
4. Рассмотрения её жизненного цикла
5. Рассмотрения её обеспечивающих систем

11. Как можно рассматривать систему с точки зрения структуры? Каковы причины множественности структуры?

Система как совокупность частей (с точки зрения структуры) - мы всегда можем построить "диаграмму деления" - декомпозировать систему на кусочки, на этажи, выстраивая иерархию и дерево этих подсистем и из составных частей.

В реальной жизни систему можно рассматривать как несколько подсистем, которые с разных точек зрения принимают различные структуры и взаимодействия.

И для каждого заинтересованного лица система может принимать свою структуру, это и есть причина ее множественности. Пример, дом как система может быть рассмотрен с точки зрения установки несущих конструкций, с точки зрения электрики, с точки зрения вентиляции, подводки воды, с точки зрения расстановки мебели и техники. Это все одна и та же система, но на каждом слое ее составные части взаимодействуют по разному и для электрика не подойдет структура, рассказанная для строителя. Можно сделать "гипер структуру" которая будет содержать все, о чем было упомянуто и сразу, но это не будет иметь смысла ввиду сложности такой структуры.

12. Как можно рассматривать систему с точки зрения функционального места? Что такое операционное окружение?

Идентификация системы, определение системы идет не от внутренней конструкции, а от того функционального места, для которого она сделана, куда она будет размещена/поставлена. Т.е. рассматриваем как мы ее применяем, с точки зрения интерфейсов и не важно как она устроена внутри.

Пример с насосной станцией: у нас есть станция, у нее два насоса. Один из них ломается. Мы говорим "поменяй насос 2", а не "поменять составляющую системы с серийным номером 123". Хотя именно серийный номер идентифицирует сломанную часть системы. Более того, мы можем поменять насос на немного другой, оставив внешний интерфейс таким же. Или вообще поменять тип насоса. Тогда, рассматривая систему с точки зрения структуры это новая система, но с точки зрения функционального места это та же самая система.

Функциональное место определяется:

- Стейкхолдерами (те самые заинтересованные стороны)
 - Точками зрения - спецификация соглашений, правил построения и использования с целью решения проблем заинтересованных сторон. Во многом определяется заинтересованной стороной.
 - Представлением с заданной точки зрения
-

Операционное окружение - то окружение, в рамках которого она функционирует. Для процессора, например, это система питания и вся периферия, с которой он вынужден работать.

12. Кто такие заинтересованные стороны (Stakeholders) системы и какова их роль в разработке?

Стейкхолдеры (stakeholders) или заинтересованные стороны это физические лица, группы или организации, которые влияют на деятельность компании/проекта или испытывают влияние от ее результатов.

В процессе разработки разные стейкхолдеры могут являться источниками требований, принимать решения, давать обратную связь и осуществлять приемку.

13. Как можно рассматривать систему с точки зрения жизненного цикла? Что такое обеспечивающая система?

Разрабатывая какую-то систему мы должны понимать, как мы проведем ее от концепции до вывода из эксплуатации.

Типовые этапы жизненного цикла:

1. Концептуальный этап (зарождение идеи)
2. Этап разработки
3. Этап производства
4. Этап использования (utilization)
5. Этап поддержки
6. Этап вывода из эксплуатации

Важно выделять (в т.ч. специфичные) для каждой системы этапы жизненного цикла, продумывать как они могут быть реализованы. Для каждого этапа притягивается обеспечивающая система - система, которая дополняет данную на этапе жизненного цикла, но может не вносить непосредственный вклад в ее функционирование.

Примеры обеспечивающих систем: команды разработчиков, промышленные производства и т.д.

Причем у каждой из обеспечивающих систем есть свой жизненный цикл и его неплохо бы продумывать

14. Каковы цели архитектурного проектирования компьютерных систем?

(на лекции не упомянуто, ответ по записи прошлого года)

Главная цель архитектурного проектирования системы в первую очередь **снижение рисков**, что включает в себя обнаружение ошибок на ранних этапах, обеспечение целостности системы. Структуризация и поддержка процессов системы на всех ее жизненных циклах.

Также архитектурный подход обеспечивает предсказуемость сроков и бюджета, и по сути все что мы получаем на практике - повышение вероятности того, что эти предсказания будут верны.

В цели **не входит** "сделать дешевле", "сделать быстрее"

15. Что такое архитектура по Гради Бучу? Что представляют собой логическая и физическая структура?

Архитектура - логическая и физическая структура компонентов системы и их взаимосвязи, сформированные всеми стратегическими (это важно в долгосрок) и тактическими (сейчас так удобно) проектными решениями, применяемыми во время разработки.

Логическая структура - устанавливает существование и роль ключевых абстракций и механизмов, которые будут определять архитектуру и общий дизайн системы

Физическая структура - описывает конкретный программный и аппаратный состав реализации системы. Зависит от конкретной технологии

16. Что такое архитектура согласно ISO 42010? Что такое архитектурное описание?

Архитектура - фундаментальные концепции и свойства системы, воплощенные в ее окружении и элементах, отношениях между элементами системы, принципах дизайна и развития.

Архитектурное описание - документ, в котором описана архитектура для какого-то стейкхолдера.

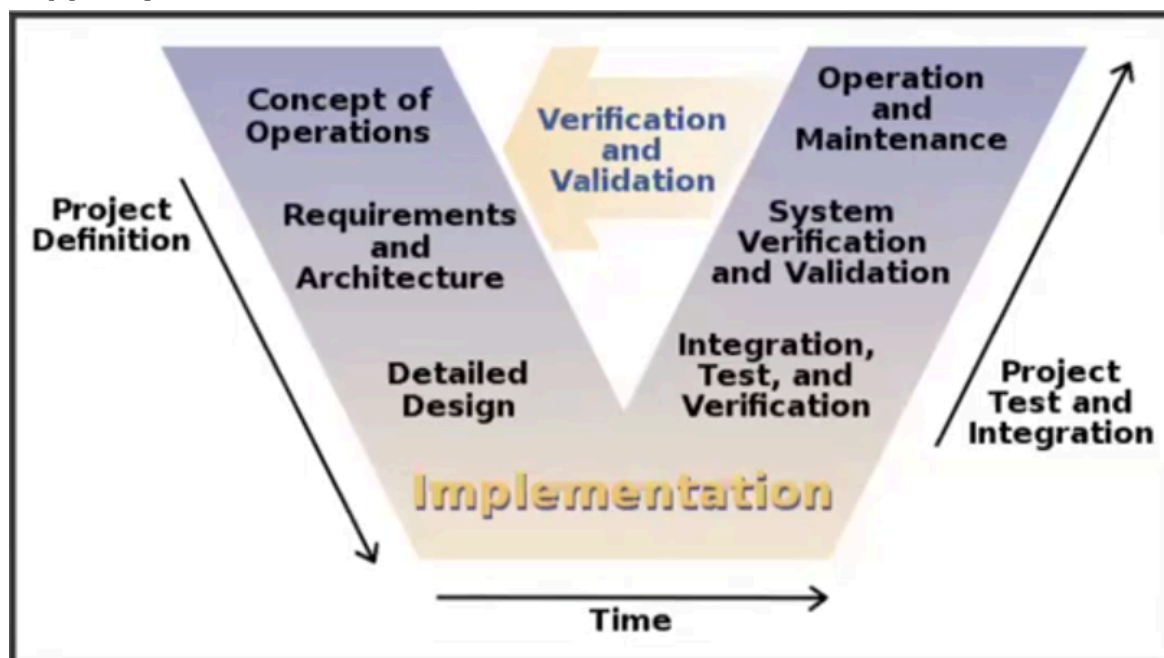
Немного шутильных, но сущностных определений:

Архитектура - все важное

Архитектура - это такие проектные решения, ошибка в которых скорее всего приведет к тому, что проект будет закрыт, отменен или провален

17. Как архитектурные решения влияют на проектные метрики?
Что такое V-диаграмма и какую особенность разработки она демонстрирует?

V-диаграмма:



По вертикали - уровень детализации (чем ниже тем более конкретно, чем выше тем детализация на более высоком уровне), по горизонтали - время

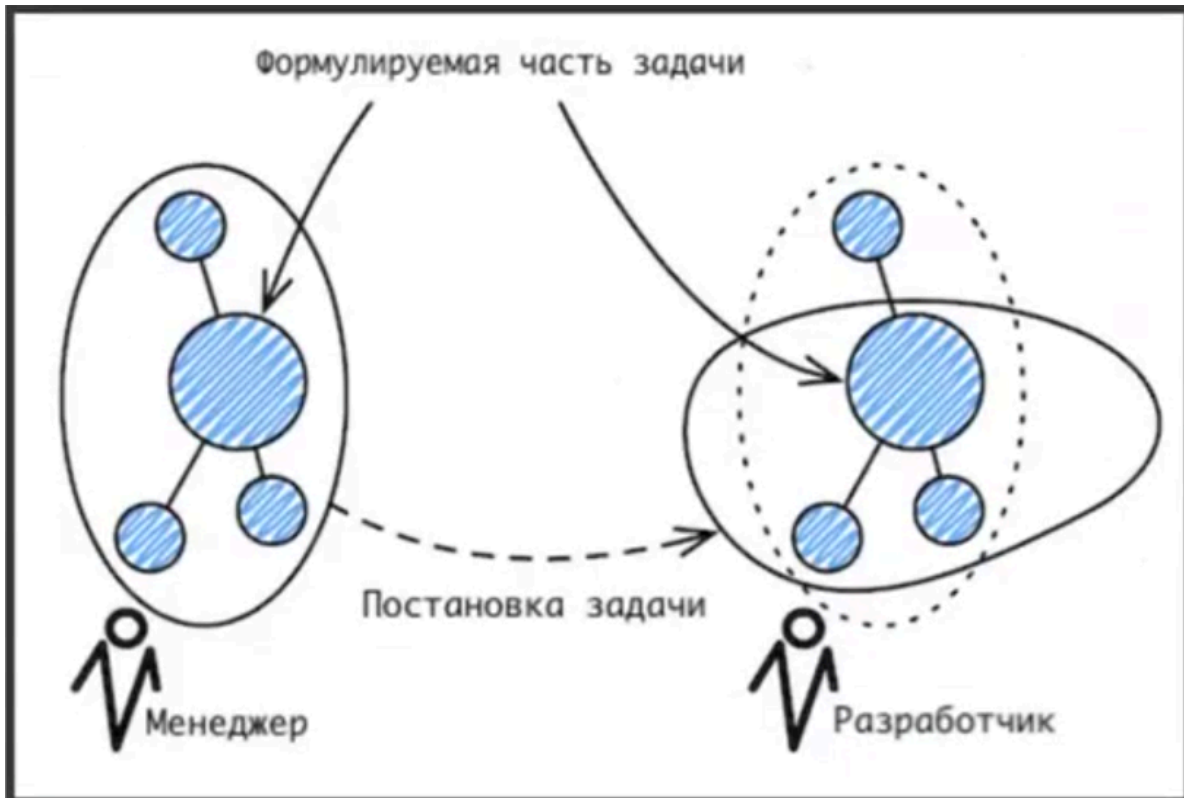
Как все проходит: Концептуальный дизайн - требования и архитектура - детальный дизайн - реализация - интеграция, тестирование и верификация - системная верификация и валидация - эксплуатация и обслуживание.

Особенность - чем выше уровень, на котором мы находим ошибку, тем дороже нам влетит ее исправление. Также для левой части, чем выше уровень, на котором мы совершим ошибку, тем дороже будет ее исправление.

18. Почему плохой менеджмент может увеличить бюджет проекта быстрее, чем другие факторы? Проблема передачи информации.

Менеджмент - коммуникация между разными элементами систем проектов.

Ключевая проблема - проблема передачи информации.



Менеджер (нетехнический специалист) формулирует задачу разработчику, у них сильно отличается круг важных вопросов, часть задачи в целом непонимаема разработчиком.

Плохая коммуникация между обеспечивающими системами могут привести к большим проблемам, неэффективным решениям, дорогим ошибкам и так далее.

(Пример с металлическими уголками, где думали что производство будет делать лазером и сделали информационные отверстия в уголках, а оказалось что на производстве делают это вручную, что оказалось гораздо сложнее из-за этих отверстий. Срок выполнения увеличился за просто так. Пример с контроллером освещения, где из-за разности расположения платы на столе разработчика и при реальной эксплуатации расположение фаз получилось в обратном порядке. Придется потом переделывать.)

<3 лекция>

19. Сравните подходы к реализации вычислений в арифмометре и на логарифмической линейке. Аналогии с современными компьютерами.

Вычисления на логарифмической линейке осуществляются путем заранее, по определенной схеме сделанных насечек, которые ввиду математических свойств их размещения позволяют получить значение какой-либо функции. В компьютерной технике подобный подход применяется в формате кэширования значения функции, получения ее значения по заранее вычисленным таблицам. Позволяет сильно упростить и зачастую ускорить алгоритм, зачастую в ущерб занимаемой памяти

В случае арифмометра мы не пытаемся представить всевозможные решения в виде данных. Мы анализируем вычислительный процесс, бьем его на некоторые повторяемые кусочки, воспроизводимые шаги чтобы получить результат для входных данных из области определения.

А еще арифмометр - цифровое устройство, а линейка - аналоговое.

Ключевое отличие арифмометра и логарифмической линейки

1. Рост области определения функций:

- для арифмометра: медленный рост сложности устройства, средний рост длительности расчёта (зависит от функции);
- для линейки/таблицы: быстрый рост размера линейки, минимальный рост длительности расчёта.

2. Рост области значений функции:

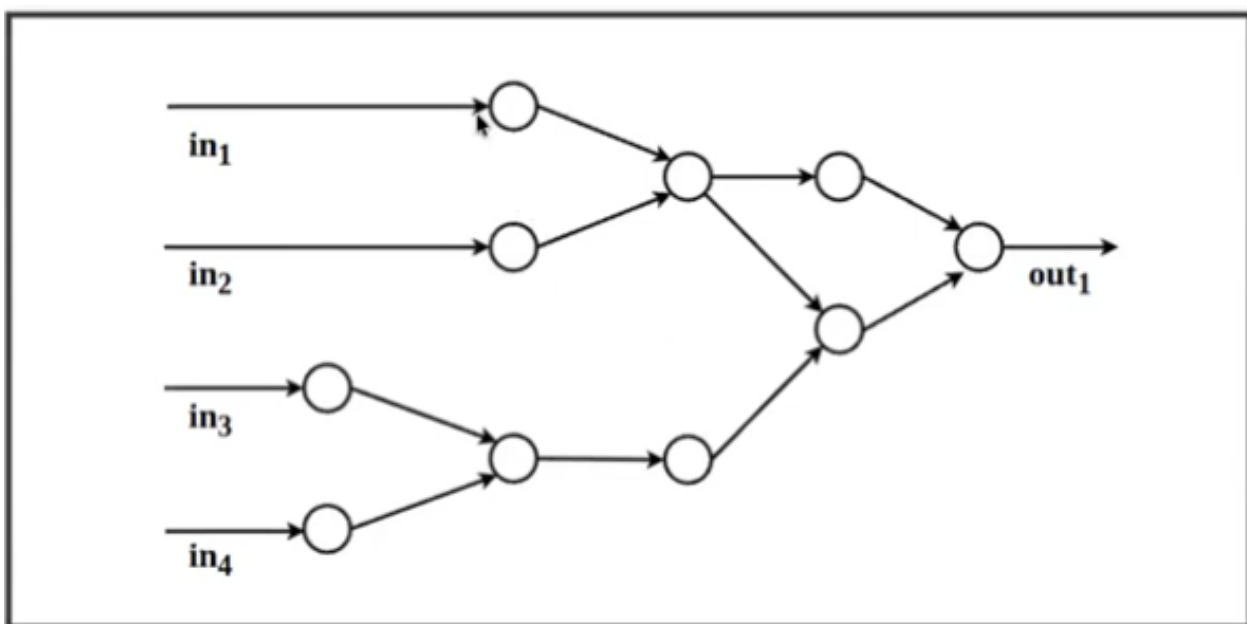
- для арифмометра: медленный рост сложности устройства, средний рост длительности расчёта (зависит от функции);
- для таблицы: средний рост размера таблицы, минимальный рост длительности расчёта.
- для линейки: быстрый рост размера линейки или погрешности, минимальный рост длительности расчёта.

3. Линейка — аналоговое устройство.

4. Арифмометр/Таблица — цифровое устройство.



20. Каков был подход к расчёту артиллерийских таблиц группой людей? Аналогии с современными компьютерами (параллелизм уровня инструкций, надёжность)

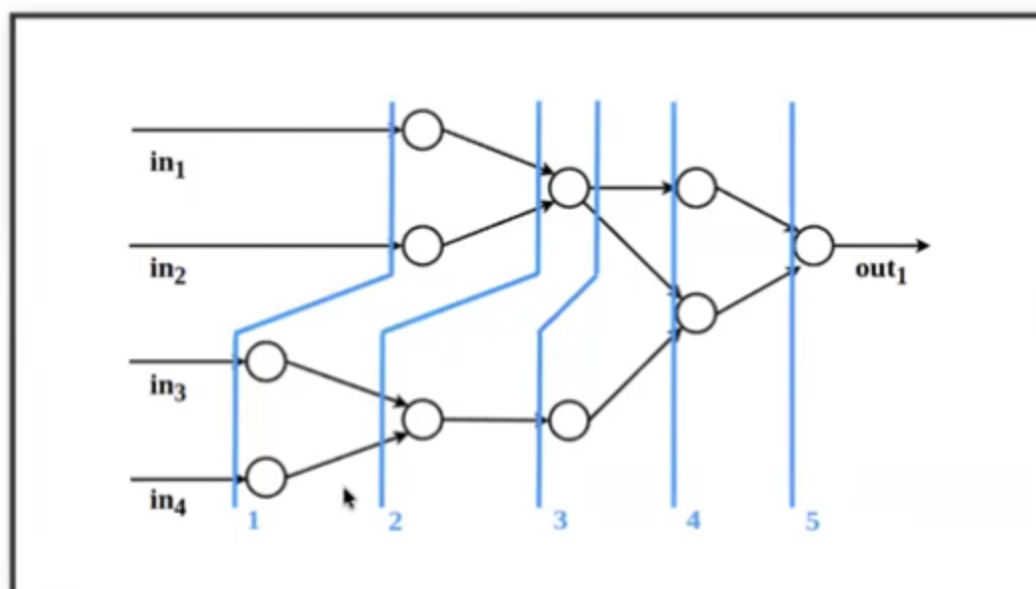


Был применен подход **реконфигурируемого потокового вычислителя**.

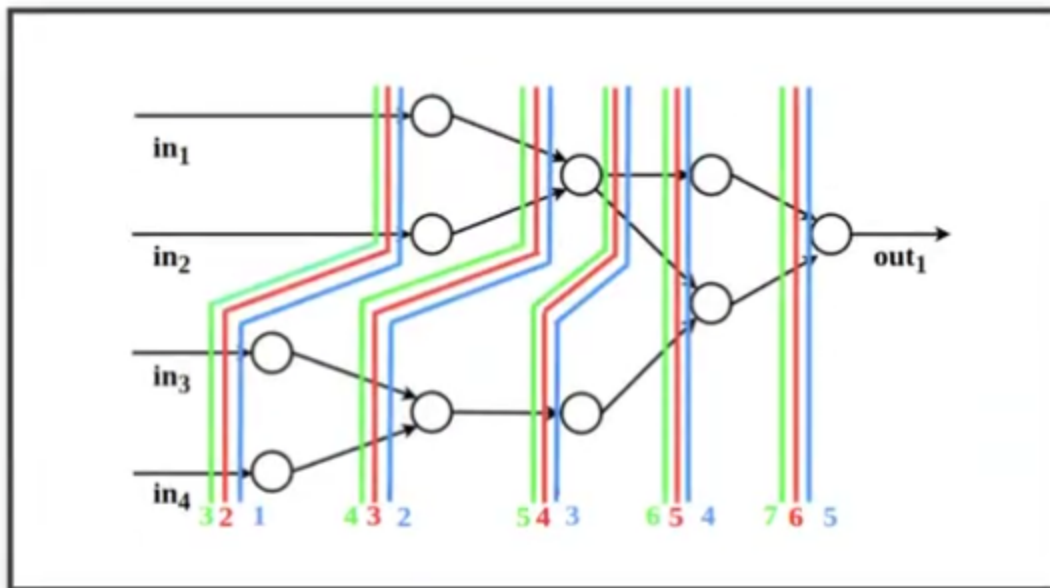
Для вычисления таблиц мы берем математика, который формирует граф вычислений. В каждую вершину графа ставим человека, который совершает только одну операцию, передает результат дальше.

Можно вычислять дважды, собрав два графа, дать им одни и те же значения, на выходе сравнить и если что пересчитать.

Плюсы в параллелизме и конвейнеризации



Конвейеризации расчётов



Получается 10 операций, но каждый такт будет выходить готовое значение, если мы смоги загрузить такую систему.

На практике это встречается в современных компьютерных системах:

- Процессорах общего назначения (RISC)
- В специализированных процессорах (CGRA процессора)
- В распределенных системах

21. Каково устройство электрического реле? Какие виды реле существуют и каковы области их применения?

Электрическое реле - автоматизированная кнопка, может быть нормально разомкнутое, нормально замкнутое. Если подан ток - катушка создает магнитное поле и одни контакты замыкаются, другие размыкаются.

Области применения электрического реле:

- когда-то для компьютеров
- там, где транзисторы не выдерживают, силовые шкафы управления, неблагоприятная среда
- в ПЛК

Виды:

- Механические (концевой выключатель в лифте)
- Тепловые (в чайнике, биметаллическая пластину изгибается и отключает нагрев)
- Оптические (реагируют на освещение)
- Реле времени (отключают по прошествии времени)

22. Как реализуется булев базис на электрических реле?

Надо реализовать И, ИЛИ, НЕ.

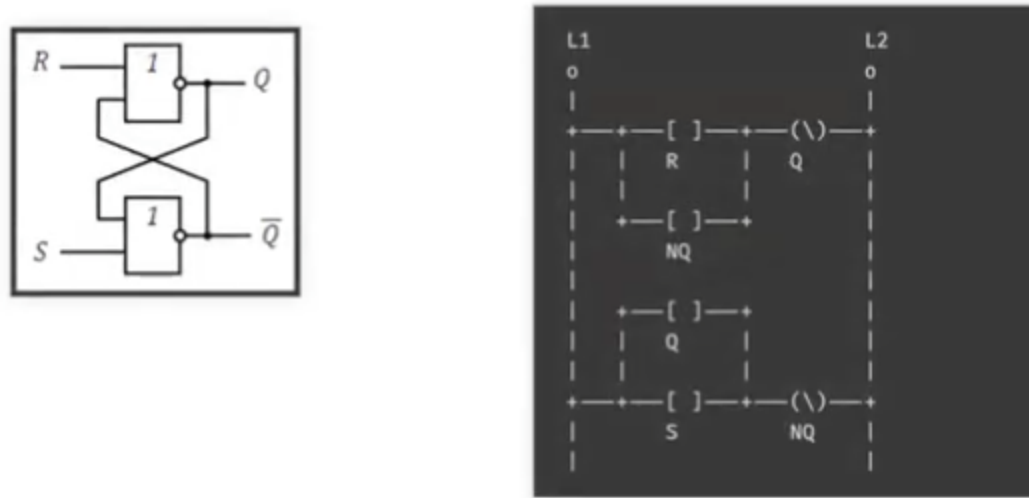
И - просто последовательно два реле (нормально разомкнутых)

ИЛИ - параллельно два реле (нормально разомкнутых)

НЕ - одно нормально замкнутое реле

RS-триггер - простейшая ячейка памяти

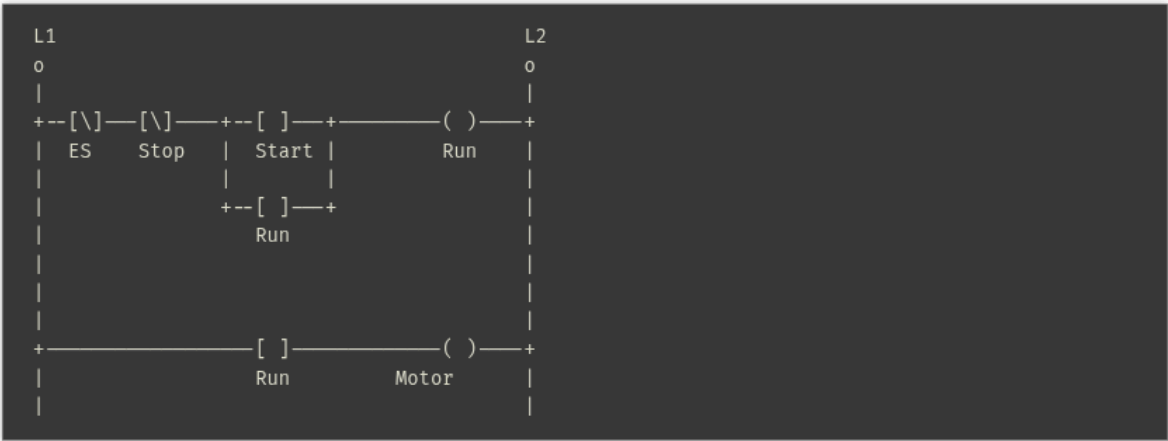
RS-триггер на реле



Теперь у нас есть все, чтобы построить компьютер.

23. Опишите классическую схему включения электродвигателя с аварийной остановкой. Объясните, как работает обратная связь.

Классическая схема включения электродвигателя с аварийной остановкой.



Причем на схеме, судя по всему круглые скобки означают прием сигнала, его сток. А квадратные источник сигнала, его исток.

Обратная связь - реле run начинает самоподдерживать свое состояние

<4 лекция old>

24. В чём заключается принцип развития иерархических систем Седова? Иллюстрируйте применительно к вычислительным платформам.

Принцип развития иерархических систем Седова - для того, чтобы у нас появилось многообразие на следующем уровне организации системы, нам необходимо ограничить его на нижележащих уровнях. Пример: чем больше операционных систем у нас будет, тем меньше разнообразия хорошего софта мы получим; чем больше языков программирования, тем меньше у нас будет библиотек для них; так и в биологических системах - на уровне клеток, на уровне белков есть строгое ограничение типов белков, типов клеток и на основе этого мы получаем невероятное разнообразие организмов

Другая концепция, предложенная Турчиным Валентином - **метасистемный переход** - говорит о том, что если мы находим новый базис, какой-то новый набор "кирпичиков" и наращиваем их количество, в какой-то момент, используя этот базис можем строить принципиально новые вещи, находить новые способы организации вышестоящих уровней.

25. Что такое программируемые логические контроллеры (ПЛК) и какова область их применения?

ПЛК - (упрощенно) специальная разновидность ЭВМ, в котором как правило есть много клеммников и разъемов. Задача (и один из вариантов использования) в том, что на компьютере есть возможность программировать данные контроллеры как схемы реле или функциональных блоков, без изучения кода и программирования.

Масштабирование сложности, нет износа, соответствие "документации" реальности (ибо то, что в контроллер зашито)

Применяются в системах управления в промышленности, в системах с традиционно высокими требованиями к надежности. Можно применять для модернизации областей, где применялись релейные шкафы (огромные релейные схемы, вот их можно заменить куда меньшей по габаритам схемой. Кроме того эта схема будет надежнее за счет отсутствия механических частей)

26. Какова структура программируемых логических контроллеров (ПЛК)? Причины отделения инструментальной составляющей?

Устроены ПЛК примерно так:

входные модули ввода -> само ПЛК (представляет собой такой микро-компьютер) -> модули вывода

Каждая стрелка снабжена оптической изоляцией. Позволяет отделить контроллер от объектов, с которыми он интегрирован (если что-то пойдет не так на входе или выходе, это не выведет из строя сам контроллер). Вся система имеет источник питания.

Инструментальная составляющая (среда разработки и прочее) отделены от исполняющей системы ввиду тотальной ограниченности ресурсов самого контроллера и по сути в объединении не было бы смысла.

27. Каковы особенности аппаратного обеспечения программируемых логических контроллеров (ПЛК)? Объясните причины таких особенностей.

Предназначены для работы в агрессивных средах, защищенные корпуса. Имеют гальваническую развязку модулей ввода и гальваническую развязку модулей вывода. Нет механических элементов. Часто устойчивы к электромагнитным помехам, механическим повреждениям, имеют защищенные выводы, внутренние системы, перезагружающие схему, если она (вдруг) зависнет

Причины очень просты - они разрабатывались под конкретный контекст их применения. А применять их планировалось (так и получилось в целом) как раз в агрессивных средах, в системах, где очень важна надежность, где есть требование к устойчивости к электромагнитным помехам, где схема должна отрабатывать с минимальными рисками и максимально предсказуемо.

**28. Каковы особенности программного обеспечения программируемых логических контроллеров (ПЛК)?
Объясните причины таких особенностей.**

ПЛК и возможность их программирования с помощью ladder diagram (релейно-контактные схемы) и это очень хороший пример, как вычислительные платформы и уровни формируются не исходя из того что у нас есть, а исходя из того, что нужно пользователю.

Также ПЛК можно программировать с помощью:

- Function Block Diagram (функциональных блоковых схем)
нечто среднее между логической схемой с простыми условиями, соединениями
- Sequential Function Chart (последовательные функциональные диаграммы)
Что-то типа конечных автоматов, позволяет описать систему с переключающимися состояниями
- Structured text (структурированный текст)
почти программирование

Это позволяет использовать ПЛК не только программистам, визуально. Тут кстати появляется требование реального времени.

29. Что такое булев базис и какова его роль в вычислительной технике? Приведите примеры.

Основой современной вычислительной техники являются следующие принципы, построенные на булевой алгебре:

- двоичная логика - все можно представить в виде некоторого количества единиц и нулей
- полный набор булевых функций, комбинируя которые мы можем реализовать любое преобразование из формата А в формат Б
 - И, ИЛИ, НЕ
 - штрих Шеффера
 - стрелка Пирса
- комбинационные схемы - из булевых функций собираем преобразователи, которые работают за один прогон
- триггер - хранение состояния

Из перечисленных выше пунктов (имея их) мы можем построить

все современную вычислительную технику (кроме квантовых компьютеров и аналоговых вычислителей)

30. Что такое двоичное кодирование сигналов? Каковы его достоинства и недостатки? Что такое запретная зона?

У нас есть электрический сигнал - мы говорим о том, что некоторый диапазон значений это единица, другой интервал за логический ноль. Далее этот сигнал передается из источника по проводнику в приемник, где мы декодируем его по другим, немного расширенным диапазонам. Те значения, которые не входят ни в один из диапазонов считаются запретной зоной - те значения, которые, как мы считаем, не должны быть в нашей системе в норме.

И туууут мы по сути совершаем метасистемный переход, где грубо ограничиваем весь аналоговый мир цифрой, булевыми значениями.

Достоинства - надежно и помехоустойчиво, простая арифметика, точность и диапазоны наращиваются разрядностью, погрешности можно рассчитать изначально в процессе проектирования

Недостатки - нечитаемое представление, например в запретной зоне мы не можем однозначно понять как оно будет интерпретировано приемником сигнала; простые десятичные дроби представляются как бесконечные двоичные; дискретное кодирование (точность)

31. Какова роль машинного слова в устройстве процессора? Что такое Big- и Little-endian?

Один бит это не интересно и не прикольно. Мы хотим представлять числа покрупнее и поинтереснее, поэтому очень быстро приходим к тому, что составляем биты в некоторые конструкции.

машинное слово - некоторая совокупность бит, с которым компьютер может "спокойно работать", условно обработать их совокупность одновременно (напр. сложить числа)

Не всегда машинного слова бывает достаточно для наших вычислений. Тогда, например, большое число может быть представлено как совокупность машинных слов и возникает вопрос как из отдельных слов составить полное число:

big-endian - запись слов "большим концом", т.е. сначала идут старшие машинные слова (или байты), а затем младшие:

0xABCDEF -> 0xAB 0xCD 0xEF

little-endian - запись слов "малым концом", т.е. сначала идут младшие машинные слова (или байты), а затем старшие
0xABCDEF -> 0xEF 0xCD 0xAB

32. Какие существуют способы кодирования целочисленных данных? Что такое позиционное кодирование, дополнительный код, код Грея, BCD?

Позиционная система счисления - в зависимости от расположения цифры зависит ее общий вклад в число, первая с конца это 1, затем 2, затем 4, затем 8 и так далее.

Дополнительный код - некоторое отображение позиционной системы счисления, для работы (причем удобной работы) с отрицательными числами

Символьное хранение - храним число как строку. Например,
String str = "42"

Код Грея - такой способ двоичного кодирования, что два соседних значения (например, 123 и 124) закодированные в этом коде будут отличаться только на один разряд

BCD - записываем десятичное число по разрядам, каждый разряд десятичного числа - 4 бита

Дробные числа - с фиксированной точкой, с плавающей точкой.

Big & little endian - когда число не помещается в машинное слово, его разбивают

- big endian - записывают с большего конца, например год 20-26
- little endian - записывают с маленького конца, например год 26-20

таргетированные данные - когда рядом с данными мы храним что это за данные такие

little-endian, например, имеет такую особенность, что можно идти по памяти и печатать куда угодно, что обеспечит нормальную человекочитабильную распечатку

33. Какие существуют способы кодирования бинарных данных?

Что такое Base64, Base58?

Исторически так сложилось, что многие протоколы заточены на передачу текстовой информации, а текст не очень предназначен, чтобы передавать бинарные данные. Поэтому придумали, как можно бинарные данные представлять в виде текста

Base64 - каждые три байта преобразуются в 4 печатных символа латинского алфавита, цифр и знаков + /. Просто берем данные по 6 бит, сопоставляем с алфавитом. Все. Если надо дополнить есть символ "=" которыми можно дополнить до кратности длины до 4 символов.

Base58 - по сути Base64, из которого убрали похожие символы, вроде O и 0; l, I и 1 потому что формат ориентирован на ручной ввод. Плюс к этому в конце добавляется хэшсумма, чтобы случайная ошибка ввода могла быть обнаружена

34. Что такое комбинационные схемы? Что такое переходный процесс и с чем связаны задержки? Как комбинационные схемы связаны с параллелизмом уровня бит?

Комбинационная схема - схема, составленная из набора элементов, в совокупности реализующая заданную таблицу истинности. Простейшие элементы:

AND OR NOT BUF (буфер просто повторяет вход, надо чтобы усиливать какой-то сигнал и повторять его дальше)

Каждая функция - это некоторое отображение одного множества в другое. Любое значение любой сложности фиксированного диапазона фиксированной точности может быть представлено в виде совокупности бит. Следовательно любую функцию можно реализовать как отображение одного бинарного значения в другое. Значит можно построить комбинационную схему, которая это реализовывает.

- Можно построить через таблицу истинности (черный ящик)
Там просто строим таблицу истинности, записываем это как совокупность булевых функций (в канонической форме ДНФ, КНФ) и дальше там можно минимизировать, оптимизировать

- Через алгоритмизацию (белый ящик, структурность/этапность расчета)

Может быть проще и компактнее, но это творческий процесс, нет четкого алгоритма

Переходный процесс - время, которое нужно схеме, чтобы выход "устаканился", перешел в стабильное состояние, соответствующее входу.

Параллелизм уровня бит - узлы с параллельным включением работают параллельно. Применимо буквально везде в цифровой схемотехнике и комбинационных схемах.

Может происходить накопление ошибки в физическом процессе, может приводить к ошибке интерпретации логического уровня. Проблему решают буферы/повторители.

35. Какие состояния существуют в комбинационных схемах (0, 1, x, z) и что они означают?

Из очевидного в комбинационных схемах может быть состояние 1 и 0.

x - неизвестно, значение может быть произвольным don't care.

Например когда входное значение не входит в область определения функции

z - отключено, когда провод висит в воздухе, состояние высокого сопротивления

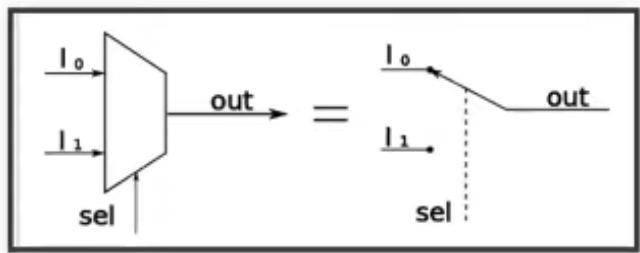
<5 лекция>

36. Каковы особенности реализации "условного оператора" в комбинационных схемах?

Глобально есть два сценария, как мы можем получать условный оператор. Эти два сценария покрывают практически все наши потребности, это:

- Есть два входа и мы хотим выбрать, из какого входа получить данные

Решение - мультиплексор, у него просто три входных переменных, одна выходная. Один из входов выбирает, какой из других двух входов будет транслироваться на выход



- Есть один вход и мы хотим выбрать на какой выход передать данные. На какой из двух приемников отправить сигнал
Тут у нас проблема - мы не можем остановить распространение сигнала и "не передать" что-то куда-то. Появляется D-триггер, который может "защелкнуть" значение по клоку. И если нам надо выбрать, в какой триггер принять данные, то мы втыкаем "И" которое пропускает клок только если это то устройство, которое должно принять этот сигнал

37. Что такое триггеры в цифровых схемах? Каковы варианты их использования?

Триггер - класс электронных устройств, обладающих способностью длительно находиться в одном из двух устойчивых состояний и уметь менять их под действием внешних факторов

Позволяют зафиксировать состояние линии на длительное время (всевозможные ячейки памяти).

Сократить задержку в комбинационной схеме, что позволит одной части комбинационной схемы работать уже над другими входными данными, пока остальная схема доделывает прошлые.

Позволяют создавать многотактовые схемы

38. Что такое D-триггер и RS-триггер? Какие существуют варианты условия изменения состояния?

Условие изменения состояния триггера:

- По фронту (по *изменению* входного клона, с 0 на 1 например)
- По уровню (фиксируем при определенном уровне)

Есть D-триггер - запоминает состояние входа и дает его на выход

RS - есть два входа один устанавливает выходное значение другой сбрасывает

39. Что такое пространственные и временные вычисления? Как они могут быть использованы для оптимизации процессоров?

Суть в том, что мы например можем сделать один большой сумматор, на 16 бит, который сразу их сложит, но займет больше места. Либо переиспользовать то, что у нас есть и то, что занимает меньше места (сумматор на 8 бит), тогда будет вычисляться больше времени.

Возможность перехода от временных вычислений к пространственным позволит организовать параллелизм уровня задач, инструкций, бит. И он же позволит сделать конвейерную организацию работ за счет многотактовости

40. Что называют синхронной схемотехникой? Каковы её достоинства и недостатки?

Синхронной схематехникой называют такую схематехнику, в которой изменение элементов памяти происходит одновременно по блоку, т.е. возможно существование многотактовых схем

Лучше тем, что проще синхронизация, легко рассчитать время прохождения сигнала. Нет гонок, переходные процессы сведены к минимуму.

Недостатки:

- Тактовая частота схемы определяется самым медленным блоком
- Дискретизация входных данных по времени (например АЦП, который получает на вход сигнал не дискретизированный, но получать и обрабатывать он его будет как дискретный)

41. Почему цифровая схемотехника оперирует уровнями, а не сигналами? Какие возможности это открывает и какие проблемы создаёт?

Здесь вопрос про то, что цифровая схемотехника существует в поле времени, а не дискретных сигналов, которые пересылаются между составными частями системы.

Почему уровни а не сигналы ("сигнал" здесь используется в значении пакета данных) здесь речь о том, что уровень, например, нельзя не послать в отличие от сигнала.

Возможности:

- в случае с цифровой схемотехникой, нет последовательности отправки значений между разными шинами, все отправляются одновременно
- Проблемы:
- Больше кол-во логики, которая нужна чтобы правильно выбирать сигналы

Ключевые отличия схемотехники от программирования

1. Все процессы между регистрами всегда происходят параллельно. Не читайте код как алгоритм, рисуйте схему.
2. Передача сигнала — физический аналоговый процесс. Есть питание и контакт — есть передача. Сигнал не может не идти.
3. Нет понятия "система остановилась". Она всегда работает, если есть питание.
4. Таблица истинности неполна — результат будет случайным, но будет (и возможно воспроизводимым).

42. Каковы ключевые тенденции в производстве радиоэлектронной аппаратуры и связанные с этим проблемы?

Монтаж радиоэлектронной аппаратуры - процесс сборки отдельных частей (реле, транзисторы) в одно целое как нам надо

Ключевые тенденции:

- Рост уровня интеграции (в меньший объем пространства поместить всего побольше. Хороший пример - наушники с шумодавом, ведь это сам по себе очень технологичный аппарат)
- Снижение затрат на производство изделия (при массовом производстве)
- Усложнение производственной цепочки (из-за п.1 необходимо подтягивать технологичность производства, в частности его автоматизацию, что приводит к усложнению производственной цепочки)

43. Какие виды монтажа электроники существуют? Как они соотносятся по гибкости и сложности производства?

Навесной монтаж - берем компоненты, условно на картонке как-то закрепляем, проводками спаиваем. Простота производства, простота подготовки производства, гибко (если не залить компаундом)

Недостатки:

- дорого в серии
- сложность автоматизации
- плотность ограничена человеческими возможностями
- надежность (особенно без компаунда)

Монтаж на печатную плату - у каждого элемента есть свое место, он монтируется на пластину диэлектрика, где есть дорожки, которые соединяют все как надо.

Есть однослойные, двуслойные, многослойные платы. Все это достаточно сложный производственный процесс с нанесением пасты, расположением элементов, прогревом, очисткой, проверкой готовых плат и так далее.

Можно очень сильно повысить уровень интеграции за счет автоматизации производства. Есть BGA-шки. Можно даже внутри плат делать встроенные компоненты (между слоями)

Недостатки:

- удлинение производственной цепочки
- удорожание для единичных экземпляров (но для серийного - очень классно)
- меньше гибкости, если не предусмотрели, но в целом можно исправлять

Монтаж на печатную плату включает в себя два подхода:

- **Штырьевой монтаж**, когда элементы физически вставляются в отверстия и припаиваются
- **Поверхностный монтаж**, условно когда отверстий в плате нету, элементы очень маленькие и паяются сразу на поверхность платы к контактными площадкам

44. Какова производственная цепочка поверхностного монтажа? Каковы её этапы?

(есть поверхностный smd=Surface Mount Device, есть выводной, когда штырьками втыкаем все это дело в плату)

Монтаж на печатную плату - у каждого элемента есть свое место, он монтируется на пластину диэлектрика, где есть дорожки, которые соединяют все как надо.

Есть однослойные, двуслойные, многослойные платы. Все это достаточно сложный производственный процесс с нанесением пасты, расположением элементов, прогревом, очисткой, проверкой готовых плат и так далее. (это и есть этапы производства поверхностного монтажа)

45. Какова производственная цепочка кремниевого производства? Каковы особенности формирования цены изделия?

Очень похож на производственный процесс печатных плат, но в сильно уменьшенном размере, очень плотно это все.

Этапы:

- Кремниевый песок -> выращиваем монокристалл кремния, высочайшей степени очистки (буля), потому что на мельчайших размерах (4нм) даже небольшое искривление может привести к большим проблемам
- Пилим булю очень точной плитой на пластины ваферы (wafer)
- Далее на кремниевую подложку (вафер) наносят материал для рисунка
 - нанести фоторезист
 - экспонировать через фотошаблон
 - удаляем то, что не закреплено
- Послойно делаем проводящие, непроводящие, полупроводниковые слои (выстреливая ионами в кремний) и так далее
- Дальше вафер режем на чипы
- Корпусируем (выводим микро проводки на более толстые, погружаем в корпус)

Надежность кремниевого производства достаточно низкая, хоть это и суууупер высокотехнологичное производство. Поэтому иногда дефектные части процессоров отключаются программно или на этапе производства и продаются как отдельный sku (отдельный

продукт). Многие процессоры в целом отменяются на этапе тестирования из-за дефектов

<6 лекция>

46. Какие трудности связаны с производством аппаратного обеспечения (подготовка производства, элементная база)?

Собственно трудности, по сравнению с производством программного продукта (или веб-сервиса), будут возникать на каждом этапе:

- Логистика
- Склады
- Специалисты
- Производственная цепочка (**github actions**)
- **Тестирование**
- Упаковка
- Дистрибуция (**docker hub**)
- Гарантийный ремонт (**support**)
(**жирным** выделено то, что есть и в производстве программного продукта)

47. Какие трудности связаны с эксплуатацией аппаратного обеспечения (амортизация, физический доступ, среда эксплуатации)?

Собственно трудности, по сравнению с эксплуатацией программного продукта (или веб-сервиса), будут возникать на каждом этапе:

- Амортизация. Выход оборудования из строя (программы сами по себе не деградируют, не выходят из строя)
- Необходимость физического доступа
- Особенности среды эксплуатации
- Долгосрочные эффекты (ну типа вот этих усов, которые олово само из себя выкристаллизует)
- Поддержка, отладка и восстановление
- Обновление: сложность изменений (например, обновить аппаратное обеспечение на вояджере, который уже за пределами солнечной системы. Или на маяке посреди океана)

48. Какие подходы к решению проблемы "устаревающей аппаратуры" существуют с точки зрения аппаратуры и ПО?

Проблема устаревающей аппаратуры - ситуация, когда важная система еще работает, но запчасти и в целом подобные системы больше никто не производит

Подходы:

- Перепроектирование на новой элементной базе
- Модульная организация: через стандартные интерфейсы, типа PCI, взяли и заменили, имитация старых систем (например имитация перфокарт или магнитных лент, хотя на самом деле это современный ssd)
- Виртуализация: симуляторы, виртуальные машины

49. Что такое концепция 2-этапного производства? Какие подходы к "конфигурированию" продукции существуют?

2-этапное производство - идея производства в два этапа.

Изготавливаем одинаковые железки в первом этапе, а во втором конфигурируем их под задачи пользователей, получая абсолютно разные продукты с разными характеристиками.

Производство "универсальной" компьютерной системы, т.е. не специально под задачу, а пытаемся сделать так, чтобы ее можно было конфигурировать под решение задачи.

А сверху над этим строим прикладное ПО, которое заставляет железо работать над нашей задачей, уже конкретной.

Количество промежуточных уровней оно произвольно в целом.

Есть подходы к обеспечению этой концепции:

Закладываем при проектировании, например, чтобы можно было вставить платы расширения, можно было конфигурировать там и все такое. Подходы:

- На уровне сборки и комплектации:
 - Джемперы
 - Платы расширения
 - Макетные платы
- Реконфигурация:

- Например, выстраивание людей для расчета таблиц.
Перекоммутация вычислительных блоков
- Программирование
 - Кусок инструкции, который определяет, как должен работать вычислитель прямо здесь и сейчас

50. Каково определение программной системы согласно OMG Essence? Каковы её части?

Программная система - система, собранная из программного обеспечения, аппаратного обеспечения и данных, которые являются частью этой системы (дают ей первичную ценность), но не программное обеспечение

Пример данных - шрифты

51. Что такое программное и аппаратное обеспечение? Что означают понятия Hardware и Software? Сопоставьте их.

Аппаратное обеспечение - электронные и механические части вычислительного устройства, входящие в состав системы или сети, исключая программное обеспечение и данные. Другими словами:

Аппаратное обеспечение - это железка из каких-то *обычно* радиоэлектронных элементов, которые взаимодействуя между собой позволяют реализовать какую-то прикладную задачу.

Программное обеспечение - совокупность программ, системы обработки информации и программных документов, необходимых для эксплуатации. Позволяет аппаратному обеспечению вычислительно системы выполнять вычисления или функции управления

Software - что-то гибкое, изменяемое, мягкое. Легко/быстро/дешево поменять

Hardware - долгие этапы, что-то что мы не можем менять, твердое

Мы не ставим соответствия ПО и software. Пример, Minix - популярное ОС, которая стоит внутри процессоров, это ПО, но это hardware.

52. Какие возможности открывает программное обеспечение (цикл разработки, гибкость, контроль, и т.д.)?

Свойства программного обеспечения:

- Быстрый цикл разработки

- Легко заменяется прямо у пользователя
- Возможно удаленное обновление, в том числе без информированного согласия
- Процесс создания и внедрения ПО патоматизируется
- Высокая сложность программ

53. Чем отличается типовое проектирование Hardware/Software от совместного (CoDesign) проектирования Hardware/Software? Каковы достоинства и недостатки?

Типовое HW/SW проектирование:

Проектируем аппаратуру:

- система команд, интерфейсы, схемотехника и система команд

Пишем программу (используя инструментарий аппаратуры):

- прикладная задача
- язык и библиотеки

Проблемы:

- Шаблонное проектирование (разделение аппаратчиков и программистов приводит к шаблонным решениям, потому что одни не знают что делать, а другие не знают, что им надо, типа "что дали, то и хорошо")
- Высокие интеграционные риски (может получиться, что HW+SW вместе просто не заработают, и такое часто бывает)
- Последовательная разработка (растягивает сроки)

Совместное (CoDesign) проектирование Hardware/Software:

Пробуем работать без жесткого разделения на HW SW и это разделение оставить на поздние этапы проектирования. Как можно дольше оставемся на уровне моделей, где можно двигать границу Hw/Sw. На этом выехал Node.js на котором можно написать и фронт и бэк, что стерло между ними границу и позволило удобнее работать

54. Что такое "Модель вычислений"? В чём назначение моделей вычислений? Приведите примеры.

Модель вычислений - средство формального описания вычислительного процесса. Используется для формальных

верификаций, теоретического доказательства алгоритмов, в дизайне языков программирования, модель-ориентированная инженерия

55. Что такое последовательные модели вычислений?

Приведите примеры. Как в них представляется вычислительный процесс?

Последовательные модели вычислений - такие модели вычислений, в которых инструкции, события, действия идут строго последовательно, одна за одной.

Очень яркий пример - конечные автоматы. Задаем состояния, и условия перехода состояний между ними

56. Что такое машина Тьюринга и почему она важна для теории вычислений?

Машина Тьюринга - бесконечная лента, на которой есть машина. Все, что может делать машина - это ездить по ленте вперед/назад, читать и записывать данные.

-
- не может быть реализована на практике из-за того, что бесконечных лент не найти :(
 - обладает полнотой по Тьюрингу (можно реализовать любой **известный** алгоритм)
 - проблема остановки (нет возможности понять для любого алгоритма, остановится ли он когда-нибудь)
 - Данные и управление отделены
 - Ориентировка на реализацию
-

57. Что такое Random Access Machine? Каково её устройство и место сегодня? Какова её связь с машиной Тьюринга?

Random Access Machine - машина, у которой есть бесконечное множество регистров из которых можно читать и писать. В остальном это эквивалент машины Тьюринга. Применяется где-то в компиляторах llvm

58. Что такое функциональные модели вычислений?

Приведите примеры. Как в них представляется

вычислительный процесс?

Функциональные модели вычислений - представление вычислительного процесса как совокупности символов и правил их преобразований. Не задают жестко последовательность вычислений

Примеры - арифметика, лямбда исчисление, Рефал

Лямбда исчисление:

4 вида выражений:

- переменные
- константы
- комбинация
- абстракция

3 вида преобразований:

- переименование переменных
- переименование функции
- сокращение аргумента

Вычислительный процесс задается декларативно по сути

59. Что такое параллельные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

Параллельные модели вычислений - системы, включающие несколько одновременных взаимодействующих процессов

Примеры:

- Actor model - отправка сообщений из разных потоков в "почтовые ящики" и кто-то их будет разгребать
- Discrete Event - смотрим что куда защелкивается по блоку, можем описать систему как последовательность состояний в разные моменты времени
- Multi-Thread Model - несколько потоков с общей памятью, это то, что у нас сейчас во многих ЯПах есть

<7 лекция>

60. Что такое Model-Driven Engineering (MDE)? Чем цепочка трансформации в MDE отличается от языков высокого уровня? Приведите примеры.

Model-Driven Engineering - когда мы реализуем не на мощном языке программирования со сложным вычислительным процессом, а строим некоторую модель, которая уже затем транслируется в нужное нам представление. В этом случае в центр процесса разработки закладываются модели.

Пример xtUML (Executable UML) чтобы описать поведение вычислительной системы не в терминах обычных программ, а в терминах конечных автоматов и систем, которые взаимодействуют между собой пересылкой сообщений. Можно скомпилировать/преобразовать в целевой язык программирования.

Другой пример - предметно-ориентированные языки (Domain Specific Language). Это подход, позволяющий писать на языке, разработанном специально для нашей предметной области. Например, все современные ЯПы, это DSL специально для алгоритмов. SQL - язык для доступа к реляционным БД.

Другие примеры MDE

1. **Lingua Franca** — язык координации для распределённых параллельных приложений реального времени.
2. **CASE-технология** — набор инструментов и методов программной инженерии для проектирования ПО с целью обеспечения качества, надёжности и поддерживаемости продуктов
3. **JetBrains MPS** — платформа для создания предметно-ориентированных языков с проекционным редактированием
4. **LabVIEW** — графическая среда разработки, ускоряющая создание тестовых систем через интуитивное программирование и поддержку любого измерительного оборудования
5. **IEC 61131** — международный стандарт для языков программирования промышленных логических контроллеров (ПЛК)

Отвечая на вопрос о цепочке трансформации: Использование моделей подразумевает отвязку от вычислительной платформы. Есть ведь императивное (когда мы говорим **как** делать), а есть декларативное (когда мы говорим **что** в итоге нужно сделать) программирование. И короче цепочка трансформации состоит в том, что в модели мы разрабатываем безотносительно того где и как это будет выполняться, появляется этот доп слой, который уже

трансформируется в что-то, что будет выполняться на конкретной вычислительной платформе. Это чисто декларативный подход

61. Что такое универсальный процессор? Каковы его особенности и свойства?

Двухэтапное производство - когда мы разрабатываем некоторый универсальный вычислитель, а потом программируем его. Разделяем физическое производство и наделение его функциональностью.

Информационный процессор - абстракция, которая позволяет обозначить устройство, которое может преобразовывать информацию. Включает в себя:

1. Ввод
 2. Процессор
 3. Хранилище
 4. Вывод
-

Универсальный процессор - информационный процессор, позволяет решать широкий круг задач, настройка которых производится после производства. (как раз подходит для двухэтапного производства)

Особенности:

- Универсальность это не всегда полнота по Тьюрингу
Многие языки даже не полны по Тьюрингу
- То что универсально в теории не всегда универсально в практике
- Универсальность противоречит эффективности (чем универсальнее платформа, тем менее она эффективна. Реализация "под задачу" всегда будет эффективнее универсального решения)
- Современный процессор - совокупность software и hardware

Свойства:

- можно использовать в 2-этапном производстве
- Полнота по Тьюрингу (почти всегда для процессора)

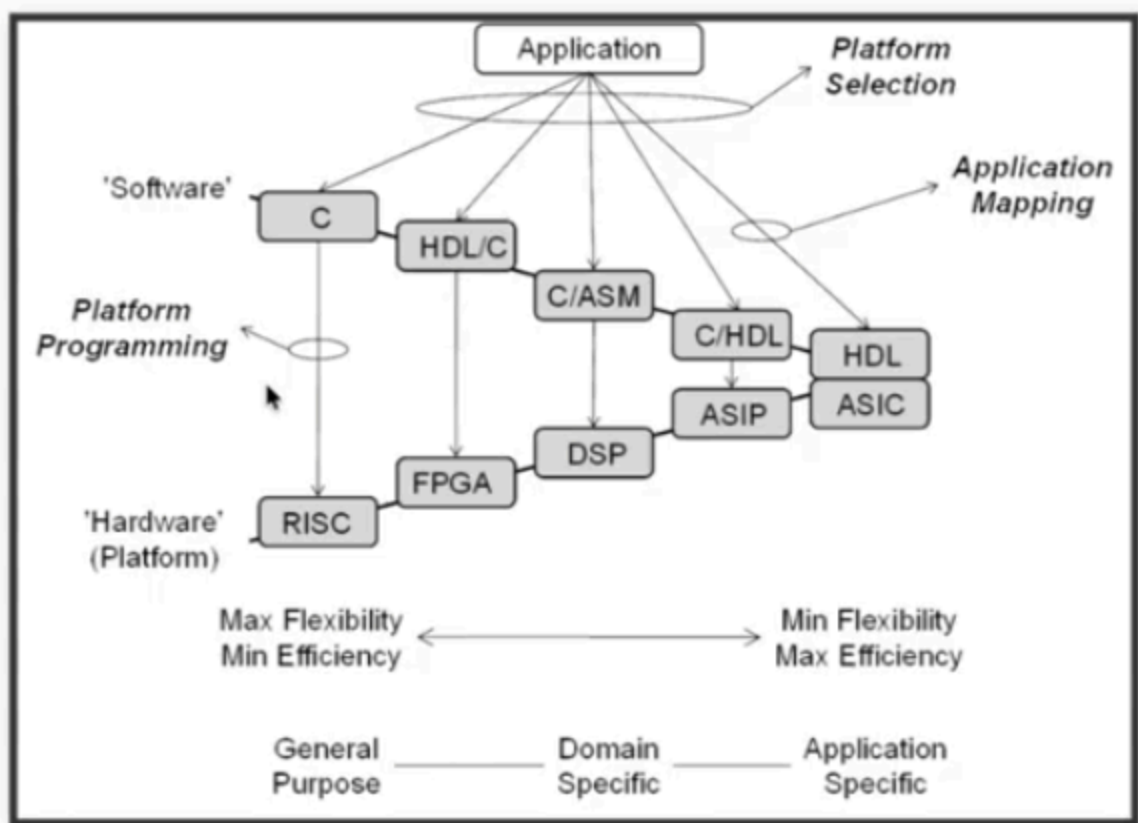
- Отсутствие "серьезных" ограничений на "объем" программы в условном внешнем storage
- Изменяемость ПО - это необходимое условие для двухэтапного производства

62. В чём заключается противоречие между универсальностью и эффективностью в разных видах процессоров (СБИС, FPGA, CGRA, GPU, DSP, CPU)?

Тут кратко должна быть суть в том, что когда мы делаем универсально - мы жертвуем эффективностью и наоборот - эффективное решение зачастую очень неуниверсально и пишется буквально по месту, максимально "одноразово" и под задачу, но получается просто сверхэффективно

- **СБИС** — *Сверхбольшая интегральная схема.*
(В контексте данного вопроса под СБИС понимается **ASIC** — *Application-Specific Integrated Circuit* / Интегральная схема специального назначения).
Конфликт: Абсолютная аппаратная эффективность, но нулевая универсальность (алгоритм «выжжен» в кремнии, чип нельзя перепрограммировать).
- **ПЛИС** — *Программируемая логическая интегральная схема.*
(Зарубежный аналог: **FPGA** — *Field-Programmable Gate Array* / Программируемая пользователем вентильная матрица).
Конфликт: Позволяет аппаратно перестроить схему под задачу, но «софтовая» гибкость дается ценой сложнейшего и долгого процесса синтеза логики.
- **CGRA** — *Coarse-Grained Reconfigurable Architecture* (Крупноблочная переконфигурируемая архитектура).
Конфликт: Проще в настройке, чем ПЛИС, но эффективна только для регулярных математических потоков; сложные ветвления программ на ней работают неэффективно.
- **GPU** — *Graphics Processing Unit* (Графический процессор).
Конфликт: Максимальная скорость на массивно-параллельных задачах (миллионы однотипных операций), но падение эффективности на хаотичном коде с обилием условий «если/то».
- **DSP** — *Digital Signal Processor* (Цифровой сигнальный процессор / ЦСП).
Конфликт: Аппаратно ускоряет потоковую математику обработки сигналов (звук, радио), но архитектурно непригоден для запуска обычного софта или операционных систем.
- **CPU** — *Central Processing Unit* (Центральный процессор / ЦП).
Конфликт: Абсолютная универсальность (выполнит любой алгоритм), но худшая эффективность — тратит огромную часть транзисторов и энергии на декодирование инструкций, а не на сами вычисления.

63. Как соотносятся вычислительные платформы и языки описания вычислительного процесса? Как это связано с эффективностью и областью применения?



Вопрос заточен под эту картинку. Сверху перечислены языки описания вычислительного процесса (по сути описания алгоритмов?), каждому из которых соотносится вычислительная платформа.

Вторая часть вопроса про конфликт гибкости (=то, насколько *широкая* область применения у инструмента) и эффективности. Потому что слева у нас максимально широкая область применения - ну си он просто везде используется, можно компилировать запускать где угодно и на очень многих вычислительных платформах, однако мы теряем в эффективности (энергоэффективности, скорости и так далее) за счет того, что наше решение отдаляется от вычислительной платформы (например, может не учитывать приколы конкретного процессора, особенности работы с ним). Справа же рассматриваются вещи, которые очень эффективны, но заточены под конкретную прикладную задачу с очень узкой областью применения.

- **RISC** (*Reduced Instruction Set Computer*) — **Компьютер с сокращённым набором команд**. Это универсальные процессоры (как в iPhone или современных ноутбуках), которые умеют выполнять любые задачи за счёт простых и быстрых инструкций.
- **FPGA** (*Field-Programmable Gate Array*) — **Программируемая логическая интегральная схема (ПЛИС)**. «Чип-конструктор»,

внутреннюю структуру которого можно перенастроить программно уже после его изготовления.

- **DSP** (*Digital Signal Processor*) — **Цифровой сигнальный процессор**. Специализированный чип, «заточенный» под быструю математическую обработку потоков данных (звук, видео, радиосигналы).
 - **ASIP** (*Application-Specific Instruction set Processor*) — **Процессор с системой команд под конкретное приложение**. Это нечто среднее: база от обычного процессора, но с добавлением уникальных команд для ускорения специфических задач (например, шифрования).
 - **ASIC** (*Application-Specific Integrated Circuit*) — **Интегральная схема специального назначения**. Чип, созданный на заводе под одну-единственную задачу (например, чип в банковской карте или контроллер SSD). Его нельзя перепрограммировать, но он работает быстрее всех.
-

64. Что такое архитектура и микроархитектура процессора? В чём различие между ними?

Архитектура процессора - то, как видит компьютер программист. Практически всегда это система команд. В целом включает в себя еще и регистры и память (как место нахождения операндов) и вычислительные механизмы (типа кешей, прерывания)

Альтернатива и по сути антонимичное понятие:

Микроархитектура процессора - соединение простейших цифровых элементов в логические блоки, предназначенные для выполнения команд определенной архитектуры.

По сути для заданной архитектуры есть практически бесконечное количество возможных микроархитектур

65. Что такое система команд и какова её роль в архитектуре процессоров? Что определяет система команд?

Система команд по сути "словарный запас" программиста.

Формальное определение:

Система команд - абстрактная модель процессора, формирующая интерфейс взаимодействия между программным обеспечением и

процессором. Формальный язык, описывающий правила вычислений и то, как они должны выполняться.

Включает в себя:

- типы данных и значений, которые она способна обработать
- набор инструкций
- механизмы обработки прерываний и исключений
- методы ввода и вывода (как часть *информационного процессора*)

При этом часто не включает в себя энергопотребление, производительность и задержки (для современного процессора практически невозможно понять, какое время будет выполняться та или иная команда)

66. Что такое машина фон Неймана? Какова её связь с машиной Тьюринга? Каковы её ключевые принципы?

- Развивает машину Тьюринга
 - Только заменили ленту на Random-access Memory
 - Инструкции и данные объединены (в машине Тьюринга алгоритм хранится "как-то" внутри машины)
- Максимальная простота в реализации и производстве
- ООП реализует похожий принцип - у нас есть состояние, к которому прикручен фиксированный набор инструкций который меняет это состояние

Машина фон Неймана. Особенности

1. Использование двоичного кодирования.
 - Встречается троичное и двоично-десятичное кодирование.
2. Программное управление. Команды выполняются последовательно.
 - Последовательность и порядок сегодня условны.
3. Память компьютера однородно хранит данные и программы.
 - Однородность спорна.
4. Ячейки памяти компьютера имеют адреса. Random-Access Memory.
 - Адрес не сводится к целому числу сегодня.
 - Память не является пассивным элементом компьютера.
5. Возможность условного перехода.
 - И перехода по иным причинам.

Принцип, на котором стоит эта машина - хранит инструкции в памяти, и есть дисциплина исполнения: читаем инструкцию, делаем

ее, переходим к следующей. Короче линейность выполнения (с поправкой на переходы).

Также есть машинное слово - то, что можем адресовать в памяти, то что единица пересылки информации из памяти, то, что хорошо ложится в регистры и так далее.

67. Что такое микропроцессор и микроконтроллер? В чём их отличия?

[Микро]процессор - мы фокусируемся на самом процессе вычислений, обработке информации и ее хранении

[Микро]контроллер - это расширение процессора периферией - таймеры, память, порты ввода вывода, счетчики

68. Что такое Control Unit и Data Path? Каковы их назначение и принципы взаимодействия?

По сути процессор можно разделить на две части: Control Unit и Data Path. Похожи, но их противопоставление имеет практический и логический смысл.

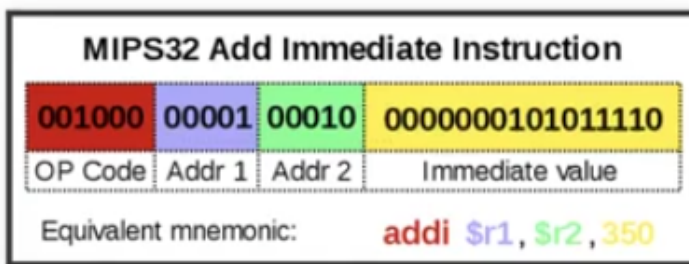
Control Unit - синхронизирует работу всех узлов, по сути заставляет инструменты Data Path работать во времени и последовательности выполнения так, как ожидается. Инициатор всех действий

Data Path - тут физически находятся и обрабатываются данные. Включает в себя: регистры, АЛУ, шины. Выполняет арифметику и пересылает байты с места на место

69. Какие виды инструкций существуют в машине фон Неймана? Приведите примеры. Каковы принципы кодирования?

- Работа с памятью
- Арифметические, логические битовые
- Управляющие операции (переходы всякие, вызов и возврат из подпрограмм)
- Инструкции для сопроцессоров

Можно кодировать, например, так:



- OP Code — код операции.
- Addr 1/2 — номера регистров общего назначения.
- Immediate value — непосредственное значение.
- $\$r1 \leftarrow \$r2 + 350$
- При переполнении:
 - $\$r1$ — без изменений,
 - установить флаг overflow.

<8 лекция>

-- 70 и 71 вопросы теперь объединены в один **"Сравните принстонскую и гарвардскую архитектуры: особенности, достоинства, недостатки и области применения."** --

70. Каковы особенности принстонской архитектуры? Каковы её достоинства и недостатки? Какова область применения?

Пристонская архитектура = фон-неймановская архитектура

Главный недостаток - один канал памяти, узкое горлышко.

Машина фон Неймана. Особенности

1. Использование двоичного кодирования.
 - Встречается троичное и двоично-десятичное кодирование.
2. Программное управление. Команды выполняются последовательно.
 - Последовательность и порядок сегодня условны.
3. Память компьютера однородно хранит данные и программы.
 - Однородность спорна.
4. Ячейки памяти компьютера имеют адреса. Random-Access Memory.
 - Адрес не сводится к целому числу сегодня.
 - Память не является пассивным элементом компьютера.
5. Возможность условного перехода.
 - И перехода по иным причинам.

Принцип, на котором стоит эта машина - хранит инструкции в памяти, и есть дисциплина исполнения: читаем инструкцию, делаем ее, переходим к следующей. Короче линейность выполнения (с поправкой на переходы).

Также есть машинное слово - то, что можем адресовать в памяти, то что единица пересылки информации из памяти, то, что хорошо ложится в регистры и так далее.

71. Каковы особенности гарвардской архитектуры? Каковы её достоинства и недостатки? Какова область применения?

Гарвардская архитектура. Особенности Достоинства

1. Два физических канала между процессором и памятью.
2. Одновременный доступ к памяти.
3. Разная ширина машинного слова и адреса для данных и программ.
 - Оптимизация под решаемую задачу.
4. Изоляция памяти инструкций.

Недостатки

1. Сложность и стоимость реализации.
2. Изоляция инструкций и данных:
 - Запуск результата компиляции.
 - Указатели на функции.

4 пункт достоинств это про безопасность, что программа сама себя не поменяет например.

2 пункт недостатков это про то, что компилятор - тоже программа, но как ей тогда запускать другую программу на этом же процессоре, если она не имеет доступа к памяти инструкций? Грустно

(8 лекция 11:12 рассказал хорошо и коротко)

И нельзя сказать, что два канала к памяти = два банка памяти, потому что это разные понятия. В случае гарвардской архитектуры мы лишь говорим о том, что у нас есть два канала доступа к памяти, которая, чаще всего, конечно, реализована как два отдельных устройства памяти, но их нельзя назвать банками памяти. Почему? Потому что банки памяти это внутренние составные части устройства памяти (условно микросхемы памяти, повешенные на

одну шину). А здесь мы говорим не про устройство памяти, а про архитектуру процессора

72. Как организовано адресное пространство в гарвардской и принстонской архитектурах?

В гарвардской у нас два канала доступа к памяти. Ответить что дает это разделение (это преимущества гарвардской архитектуры по сути). Еще у нас может быть разная разрядность памяти команд и данных.

А в принстонской - один контроллер памяти, один канал доступа, инструкции и данные хранятся в месте.

73. Какие существуют варианты обхода ограничений гарвардской архитектуры, включая "Модифицированную гарвардскую архитектуру"?

Вариации Гарвардской архитектуры	
Архитектура "Память инструкций как данные" (Instruction-memory-as-data)	реализуется возможность читать и писать данные в память программ. Позволяет генерировать и запускать машинный код.
Архитектура "Память данных как инструкции" (Data-memory-as-instruction)	реализует возможность запуска инструкций из памяти данных. Позволяет генерировать и запускать машинный код, но параллельный доступ ограничен.
Модифицированная Гарвардская архитектура (main stream)	Доступ к памяти реализуется через независимые кеша для данных и программ, за счет чего, с точки зрения внутренней организации процессора, доступ реализован независимо, при этом канал между процессором и памятью один.

По сути первые два пункта убивают безопасность, и отчасти убивают преимущество в производительности, ведь используя один тип памяти сразу вместо двух порождает проблему одного канала.

Последний пункт самый часто используемый сейчас и в целом

74. Что такое "аккумуляторная система команд"? Каковы основные элементы? Приведите примеры. Каковы достоинства и недостатки?

Это по сути система команд процессора, у которого есть некоторое накопленное состояние, которое можно читать и записывать в память и делать с ним нужные операции.

С точки зрения системы команд получается так, что почти все инструкции унарные с неявным вторым аргументом - аккумулятором. Простая и компактная. Но достаточно неудобна как раз из-за ее простоты - постоянно надо доставать что-то из памяти в аккумулятор, что-то с ним делать и класть обратно, в общем много нудных повторяющихся операций.

75. Что такое Register-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

Пример это мотаролла 68000, у нас называется m68k.

По сути у нас есть тот же аккумулятор, но их несколько, типа там 8, 16, 32 и так далее. И над ними можем выполнять операции (соответственно с этими регистрами). Здесь у нас больше свободы, упрощается код, меньше обращений к памяти ну и так далее

76. Что такое Register-to-Register системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

Классический RISC вариант (?). Короче отличие от прошлой в том, что там аргументом операции могло служить что-то из памяти. Но как только мы говорим, что здесь можно подгрузить что-то из памяти, сразу возникает куча способов как это сделать, начиная от прямой и косвенной адресации заканчивая косвенными и всякими такими ужасами. И простая арифметическая инструкция превращается в монстра-универсала и это все очень сильно усложняет.

Здесь же мы можем или: запросить что-то из памяти или сделать что-то с регистрами. Это упрощает конвейнеризацию (и суперскалярные дела, и в целом разделение структуризирует то, что мы делаем), позволяет настроить общий "ритм" процессора (инструкции с регистрами будут занимать одно конкретно время,

инструкции с памятью другое конкретное время. А не просто "какое-то в зависимости от опреанда" как в прошлой системе команд)

Но минус в том, что больше инструкций надо выполнить, чтобы сделать то же самое по сравнеиню с прошлой системок команд

В современном варианте можно сказать что этот вариант вытесняет другие.

77. Что такое Memory-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

Можно сказать что самая развитая архитектура/система команд с точки зрения удобства программирования. Мы опускаем эти приколы под названием регистры и непосредственно оперируем операциями над памятью.

Это круто тем, что ассемблер непосредственно делаем то, что мы хотим без костылей с регистрами. Простой, компактный читаемый код.

Недостаток непосредственно в том, что для каждой инструкции аж три цикла обращения к памяти, которая обычно медленная, время доступа которой может отличаться на порядки в зависимости от например кэширования.

78. Что такое CISC процессоры? Каковы их ключевые особенности, достоинства и недостатки?

Complex Instruction Stet Computer - архитектура процессора, в которой одной инструкцией можно описать довольно сложное действие.

Причины появления:

- Низкоуровневые языки.
- Разнообразие архитектур.
- Неразвитость компиляторов.
- Удобство программирования.
- Высокая производительность.
- Минимизация объёма программ.
- Минимизация накладных расходов.

Проблемы:

- Сложная система команд (использование, анализ).
- Сложное устройство процессора и Control Unit.
- Сложно генерировать эффективный машинный код.

Последний пункт проблем это про то, что когда у нас появляется разнообразие архитектур (а в CISC оно появляется очень быстро),

то писать компиляторы становится сложно - нам надо учитывать все особенности / возможности к оптимизации и паттерны каждой процессорной архитектуры, чтобы в том числе сохранять эффективность.

79. Какие виды адресации аргументов инструкции в CISC процессорах?

Их может быть вообще великое множество чисто из философии CISC-like процессоров. Например, из того, что реализовано во wrench (архитектура m68k):

- Прямая регистровая - значение из регистра
- Непосредственная (Immediate) - значение напрямую
- Косвенная - берем значение по адресу, который задан значением в регистре
 - Косвенная с пост-инкрементом/пре-декрементом - когда после инструкции адрес будет смещен
- Косвенная со смещением - как косвенная, но с добавлением некоторой константы
- Косвенная с индексом и смещением - как прошлая, но еще плюс добавить значение некоторого регистра

80. В каких случаях CISC инструкции эффективны? Приведите примеры.

CISC инструкции эффективны за счет того, что они достаточно узкоспециализированы (тот самый конфликт гибкости и эффективности), каждая из них делает очень конкретную задачу, но с очень узкой областью применения.

Чтобы это нивелировать их очень много разных, чтобы покрыть все сценарии использования.

Пример:

```
move.l 4(A0, D1), D0
```

в m68k за счет применения CISC одной инструкцией делает:

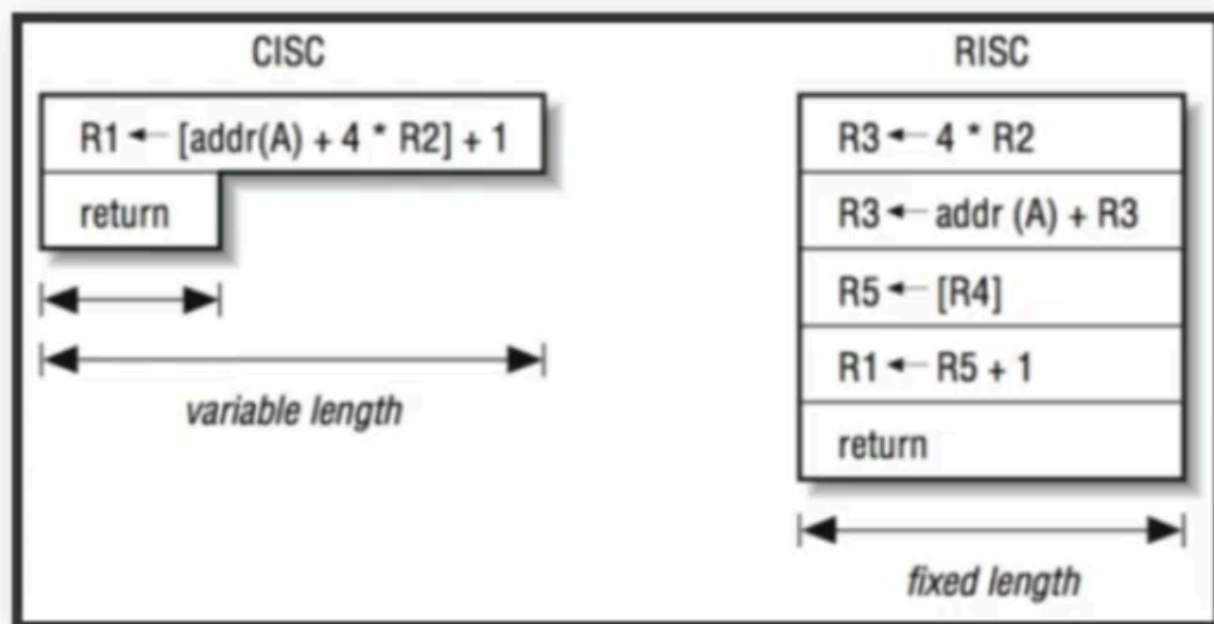
- загрузить значение из адреса (адрес + 4 + D1) в регистр D0

В RISC это были бы отдельные операции:

- Загрузить значение A0 куда-то

- Прибавить ему 4, сохранить
- Прибавить ему D1, сохранить
- Обратиться к памяти и загрузить значение по полученному адресу
- Сохранить это значение в D0 (примерно)

Хорошая картинка на эту тему:



81. В чём выражается комплексность систем команд CISC?

Комплексность (она же сложность) инструкций заключается в том, что они выполняют много простых операций оборачивая их в одну атомарную. Как бы сложность переносится из нашего кода в Control Unit и в целом в железо процессора. И за счет этого можно повысить эффективность (меньшее количество тактов для такого же алгоритма по сравнению с другими архитектурами)

82. Какие существуют подходы к реализации Control Unit?

Достоинства и недостатки Hardwired и микропрограммного управления?

Два базовых подхода к созданию Control Unit:

Hardwired:

Мы просто создаем большую комбинационную схему, которая работает от нескольких регистров и тактов собственно. На основе этого CU генерирует необходимые сигналы, управляя Data Path

Microcoded

Суть в том, что внутри CU у нас будет еще один "процессор", выполняющий простейшие микрокоманды, у него будет свой микрокомандный counter. И свое хранилище микрокоманд,

распознающее наши инструкции и реализовывающее все, что нам надо.

83. **Что такое микропрограммное управление? Что такое микроинструкция? В чём отличия инструкций и микроинструкций?**

Микропрограммное управление - один из способов реализации Control Unit в процессоре, когда каждая инструкция разбивается на микроинструкции, генерирующие необходимую последовательность управляющих сигналов на Data Path.

Инструкции - кирпичики, выполняя прикладные задачи.
Микроинструкции - обеспечивают выполнение инструкций.

84. **Как можно оптимизировать инструкции при помощи микропрограммирования? Приведите пример.**

Можно в теории исправлять ошибки в реализации, закрывать уязвимости уже после производства. Можно создавать очень сложные инструкции даже на простой архитектуре, которые комбинационной схемой в лоб было бы куда сложнее реализовать. Привносит понятности в Control Unit.

Микропрограммное управление	
Достоинства:	Недостатки:
1. Простота реализации сложных ISA (CISC).	1. Хранение микрокода в процессоре.
2. "Программирование" системы команд.	2. CISC долго учить.
3. Доступ к микрокоду для программиста.	3. Разнообразие архитектур → проблемы инструментария.
4. Генерация ISA под задачу (сократить объём, повысить эффективность), см. «Самсон».	4. Разнообразие команд (форматы, размеры, длительности, доступ). Усложняет: • оптимизацию процессора; • инструментарий.
	5. Микрокод привносит все проблемы программирования (сложность, отладка, методы).

85. **Какие существуют подходы к поиску первой микроинструкции для заданной инструкции?**

Поиск адреса микрокода для инструкции

1. **Алгоритмически:** сопоставление инструкции с условием + условный переход в рамках микропрограммы.
2. **Отображение кода операции на адрес микроинструкции**
 - **Прямое отображение** — opcode выступает в роли адреса. К примеру: opcode 0101 — 0×05 адрес микроинструкций.
 - **Непрямое отображение** — требует использования look-up-table для поиска нужного адреса (см. пример лаб. 4).

По сути первый пункт это то, как сделано в БЭВМ.

86. Что такое архитектура NISC? Каковы область её применения, достоинства и недостатки?

NISC - No Instruction Set Computer

Появилась как доведение до абсурда микрокомандного подхода. Типа а что если отказаться от системы команд и просто генерировать эффективный микрокод? И наш алгоритм становится одной большой макроинструкцией. Очень эффективно, но теряем гибкость, универсальность.

Особенности NISC

Достоинства:

1. Упрощение аппаратуры.
2. Максимальная эффективность программного управления.
3. Нет ISA, нет проблем её проектирования.

Недостатки:

1. Невозможность бинарной совместимости.
2. Низкая плотность машинного кода.

Использование:

1. Применяется в ускорителях, в высокоуровневом синтезе (HLS), спец. вычислителях.
2. Проект **NITTA** — CGRA процессор, где вычислительные блоки управляются в стиле NISC: транслятор генерирует управляющие слова для каждого функционального блока напрямую, минуя традиционный уровень ISA.



ISA - Instruction Set Architecture. Здесь ее просто нет.

Бинарная совместимость - способность машинного кода запускаться на разных процессорах. В NISC это невозможно, потому что даже один провод, проложенный по другому влечет за собой невозможность корректно запустить весь код, который был написан для предыдущей версии.

<9 лекция>

87. Что такое стековые системы команд? Каковы основные элементы? Что такое неявная адресация аргументов?

Каковы их достоинства и недостатки?

Это система команд/архитектура, которая отстоит от других ввиду того, что она работает не с регистрами или памятью, а реально со стеком. У операций нет аргументов, потому что предполагается что некоторое их количество лежит на стеке (это и есть их *неявная адресация*). Ну и конечно из памяти мы можем загрузить в стек и выгрузить.

Такие процессоры называют "процессорами высокого уровня" за счет того, что очень нативно и удобно реализуется поддержка процедур.

Недостатки в том, что код не очень эффективен, особенно когда нам надо обращаться в глубину стека. Ограничения по размеру стека, скорость доступа к нему тоже непонятна по сравнению с регистром, с которым все ясно и понятно. Параллелизм уровня инструкций - очень больно реализуется ибо все инструкции завязаны на одно состояние - стек.

У нас может быть произвольное количество операндов у инструкций, за счет того, что неявный операнд любой инструкции - весь стек

88. Какие виды стеков есть в стековых процессорах? Их количества?

У нас может быть один и несколько стеков, что позволяет решить некоторые вопросы параллелизма (если например у нас есть два стека, то переключаясь между ними мы нативно реализуем поддержку thread'ов), или как в f32a удобнее реализовать поддержку процедур.

Виды стеков:

- стек в памяти, когда есть регистр который указывает на вершину стека и собственно память где хранятся значения
- Набор связанных регистров
- Либо еще как-то аппаратно реализованная история, которая обычно позволяет за один такт прочесть вершину стека, а также следующее после вершины стека значение

89. Каковы основные принципы описания алгоритмов для стековых процессоров? Что такое Forth и High-level language computer?

Forth - императивный (описывающий **как** что надо делать) язык программирования на основе стека. Очень хорошо компилируется под стековые архитектуры и крайне эффективен для них. Еще и процедуры поддерживает и с рекурсией и все это без жутких соглашений и сложностей на уровне процессора (все очень нативно)

Высокоуровневым процессором (High-level language computer) - как раз называют процессоры на основе стека, потому что здесь **кратно** проще работать с функциями (нам не нужны никакие соглашения о том, какие регистры могут быть изменены в ходе работы функции а какие нет, нам не надо ломать голову с передачей аргументов) мы просто кладем на стек аргументы, вызываем функцию, получаем результат в стеке.

А внутри самой функции она получает произвольное кол-во аргументов (за счет стека), ей не надо париться с тем, чтобы сохранить куда-то состояние процессора (в других процессорах куда-то в память сохраняют состояния регистров, чтобы потом их вернуть), просто считает все что ей надо, кладет результат на стек

90. Каковы источники роста производительности процессоров? (и алгоритмов, в целом эффективности решения задач)

- Рост частоты процессора - чаще тактовые импульсы -> чаще выполняем инструкции -> быстрее работаем. В два раза больше частота - в два раза больше скорость (с поправкой на скорость работы памяти)
- Специализация аппаратуры - аппаратные вычислители, например аппаратное умножение будет куда быстрее чем если мы будем через сложение и сдвиги реализовывать умножение
- Специализация системы команд - это вот история про CISC, где команды эффективнее за счет специализации
- Параллелизм уровня бит, инструкций, задач:
 - На уровне бит - увеличиваем разрядность, и теперь мы можем большие объемы памяти обрабатывать за меньшее количество инструкций
 - Инструкций - конвейеры всякие там, суперскалярность (?)

- Задач - когда у нас есть несколько ядер/потоков и мы параллелим именно на этом уровне, на уровне прикладной задачи
- Адаптация структуры вычислителя под задачу - жертвуя гибкостью вычислителя допиливаем его строго под нашу задачу, получая прирост производительности
- Динамическая адаптация - специализация аппаратуры, "перепрошивка" вычислителя (его Control Unit) на лету

91. Какие законы и ограничения влияют на производительность современных процессоров?

см вопросы 92-95, просто кратко описать законы

92. Что представляют собой закон Мура и закон Деннарда?

Закон Мура - количество транзисторов в процессорах будет в два раза больше за 1.5 года, а производительность процессора будет увеличена в два раза (эмпирический закон - наблюдение, правда перестал работать уже в 2010 году)

Закон Деннарда - размер транзистора может уменьшаться, что увеличивает производительность (меньше накладных расходов на передачу сигналов в цифровых схемах), уменьшается энергопотребление, что значит что можно поставить частоту побольше. По факту не очень работает, то есть работает но до некоторого предела. Мы уже настолько делаем транзисторы меньше, что упираемся в физические пределы и наши возможности производства.

93. Что представляет собой закон Амдала? На каком уровне параллелизма он работает?

Закон Амдала - если мы решаем задачу параллелизма, увеличивая его уровень в два раза, нам по сути надо площадь процессора увеличить в два раза (построить еще такой же вычислитель, который будет вычислять вместе с первым), что очень накладно, а еще с каждым шагом увеличения параллелизма время, которое вы выигрываете будет меньше (типа было время выполнения 100 секунд. После первого шага удвоения будет ~50 секунд, выигрыш 50 секунд. После второго шага удвоения будет 25 секунд, выигрыш уже всего 25 секунд). Ну а еще он говорит о том, что даже если мы распараллелим половину

программы и возьмем бесконечность вычислителей, то все равно ускорение будет в два раза в пределе. Для 95% распараллеливания это будет всего 20 раз, это при теоретически бесконечном числе вычислителей, не впечатляет.

Походу работает на уровне параллелизма инструкций, потому что мы рассматриваем, насколько параллелятся наш поток инструкций, который решает **одну** прикладную задачу (поэтому это не уровень параллелизма задач, а именно инструкций) (в частности вот это добавление пузырьков и есть то, что ограничивает распараллеливание в этом законе)

94. Что такое Power Wall и тёмный кремний?

Power Wall:

увеличивая частоту мы квадратично увеличиваем потребление, а значит и тепловыделение, значит есть предел повышения частоты после которого мы уже не сможем эффективно и в нужном объеме отводить тепло от чипа

Темный кремний:

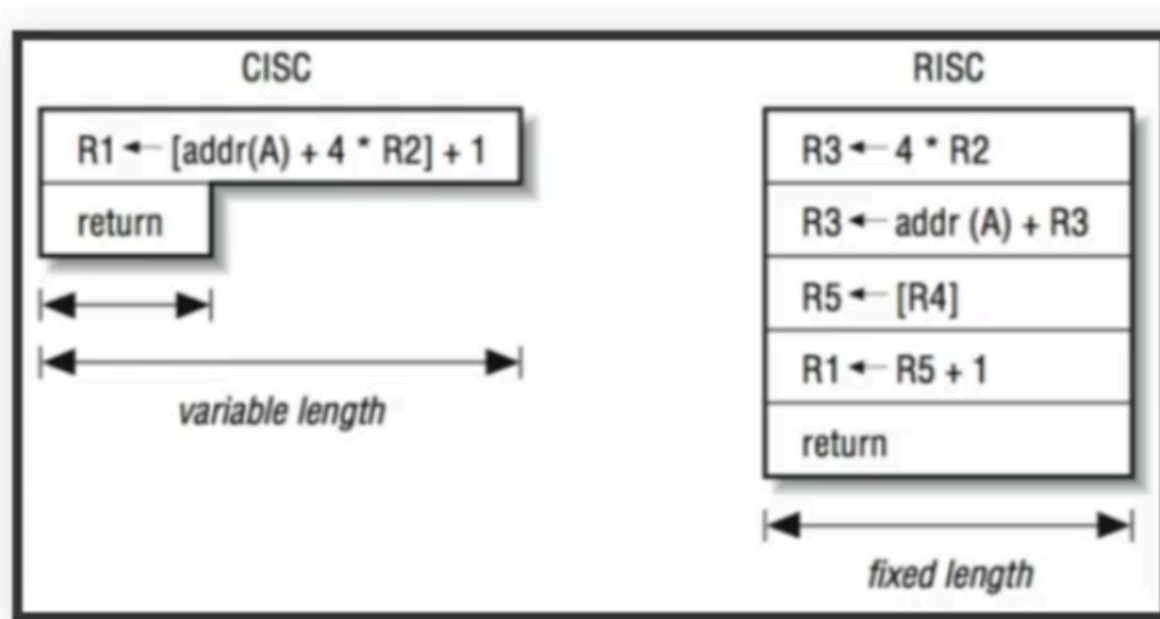
Тут короче суть в том, что если мы в некотором процессоре будем повышать плотность транзисторов, то в какой-то момент мы не сможем больше отводить от него тепло, его будет выделяться слишком много. Таким образом существует некоторое ограничение объема вычислений на единицу площади, даже если мы сможем уместить туда гораздо больше транзисторов.

95. Какие подходы существуют к решению проблемы Memory Wall в рамках процессоров семейства фон Неймана?

Memory wall - фундаментальное ограничение, связанное со скоростью памяти - ее скорость растет **значительно** медленнее, чем скорость процессоров. И скоро (да и уже сейчас) процессоры будут упираться в скорость памяти, мы просто не сможем обеспечить достаточный поток данных для адекватного функционирования вычислительного процесса.

Решать сложно, частично решается с помощью кэшей, количества каналов доступа к памяти (в Гарвардской архитектуре, например из два что несколько решает проблему)

96. Что такое RISC? Каковы особенности и предпосылки появления (ПО и аппаратура)?



Другой подход к процессоростроению и построению системы команд - отрицаем подход CISC и максимально дробим инструкции на инструкции попроще, получая простые кирпичики и их кладем в основу системы команд. Получается, что у нас теперь не куча сложных инструкций, а небольшое количество максимально простых и базовых инструкций.

Это позволяет упростить сам процессор, сократить накладные расходы, увеличить вычислительную плотность. У нас не будет лишней схемотехники, которая используется очень редко.

Предпосылки:

- Языки высокого уровня, сложно компилировать код для CISC процессоров. Раньше люди писали на ассемблере и CISC для них был просто удобнее, но сейчас ассемблер генерируется, и проще генерировать под RISC
- Простота декодера инструкции это удобно. Единый формат инструкций это удобно

Особенности:

- Можно ускорить и оптимизировать каждую инструкцию
- Появляется параллелизм уровня инструкций
- Место микрокоманд и декодера можно пустить на более полезные задачи в духе кэша или регистров

97. Как сравниваются CISC и RISC с точки зрения архитектуры?

В CISC, по сравнению с RISC:

- очень мощные, комплексные программы

- есть операции memory-to-memory и всякие такие истории, тогда как в RISC строго Load/store
- меньше регистров видимо за счет общей сложности других частей процессора
- Плавающая длина инструкций, у RISC единый формат и размер инструкций

98. Как сравниваются CISC и RISC с точки зрения микро-архитектуры?

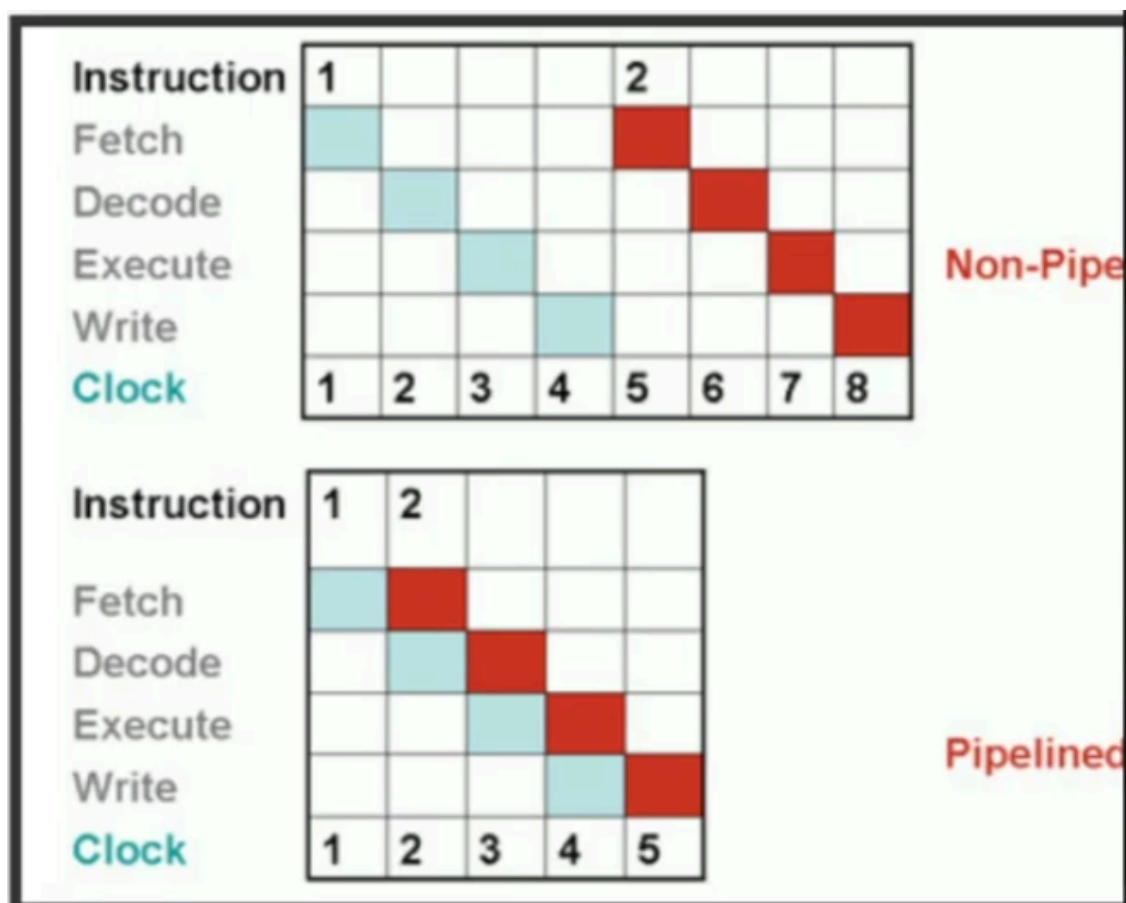
В RISC, по сравнению с CISC:

- Гораздо проще декодеры за счет простоты инструкций
- Гораздо проще Control Unit, есть место для его оптимизации (например, для реализации его как комбинационной схемы вместо микрокомандной архитектуры)
- Поддерживается конвейеризация

99. Каковы особенности кодирования инструкций при сравнении CISC и RISC?

В CISC разная длина инструкций, тогда как в RISC фиксированная. Формат инструкций в RISC более строгий и единый, не так много режимов адресации к примеру. В CISC формат может плавать, может быть огромное количество разных фишек, зашитых уже сразу в инструкции (как вот в m68k адресация с индексами и некоторым смещением)

100. Каковы принципы конвейерного исполнения инструкций? Как это влияет на загрузку и производительность?



На картинке классическая схема конвейера. Принцип работы:

- Выполнение каждой инструкции можно разбить на несколько этапов, которые задействуют непересекающиеся части процессора и выполняются за один такт
- Обеспечиваем выполнение этапов независимо друг от друга
- Как результат мы одновременно можем выполнять независимые друг от друга этапы разных функций. Т.е. пока декодируется первая инструкция, мы уже загружаем следующую инструкцию. Потом первая перейдет на выполнение, а вторая на декодирование, будет возможность взять третью инструкцию на загрузку.
- По итогу каждый такт мы забираем новую инструкцию вместо того чтобы ждать пока прошлая завершится

Для конвейеризации конечно важно, чтобы одна инструкция не зависела от результата другой иначе все ломается

101. Каковы типовые стадии RISC процессора?

1. **Instruction Fetch.** Чтение инструкции по счётчику команд.
2. **Instruction Decode.** Декодировать инструкцию и считать регистры.
3. **Instruction Execute.** Операций изменения данных.
4. **Memory Access.** Чтение/запись операндов из памяти/в память.
5. **Write Back.** Запись результата в регистры.

102. Какие проблемы и архитектурные ограничения связаны с конвейеризацией?

Инструкции должны быть независимыми друг от друга. Это достаточно сложно, потому что часто они используют например общие ресурсы процессора. Из этого вытекают:

- Структурные конфликты (например оба этапа хотят обратиться к памяти, конфликт). По сути это проблема разбиения на этапов, потому что они получаются не независимыми друг от друга (например, чтение инструкции и чтение операндов используют память с одним каналом)
- Конфликт по данным, одна инструкция еще не успела посчитать, а другой уже нужен результат
- Конфликт по управлению, условные переходы порождают проблему того, что нам вообще не ясно, какую следующую инструкцию загружать в конвейер после этого перехода, надо ждать

103. Как разрешаются структурные конфликты в конвейере?

На уровне архитектуры:

- Двухпортовая память (несколько каналов доступа к памяти, можно брать свободный)
- Гарвардская архитектура (чтение инструкции и чтение операнда теперь не конфликтуют)

На уровне выполнения:

- Разрешение конфликта пузырьком. Это холостая "инструкция", коотрая проходит по этапам, заставляя их простаивать. Это похоже на пор но НЕ пор, потому что пор тоже надо читать и декодировать. Пузырек не надо, он ничего не использует из ресурсов, они просто простаивают обрабаывая его. Результат - разносит во времени конфликтующие этапы

104. Как разрешаются конфликты по данным в конвейере?

Read after write (основной, самый простой конфликт):

- Когда за счет параллельного исполнения одна инструкция уже читает то, что другая инструкция еще не успела записать

Можно разрешить при помощи пузырька опять же.

Можно исполнять инструкции не по порядку (суть такая же как у пузырька, в том плане что нам надо разнести инструкции по времени, только вместо бесполезного пузырька мы пускаем полезную инструкцию), для этого надо как-то по умному их перетасовать.

Другие, более специфичные конфликты:

Wirite after read - могут быть проблемы с кешами, в целом не очень уж и проблема

Write after Write - когда важна последовательность записи, например, в управляющие регистры

Read after Read - тут вообще проблем нет, не важно в каком порядке куда читать

Иногда, при ложных конфликтах типа WAR и WAW можно переименовать регистры и это не отразится на логике программы. Тоже способ разрешить конфликт

Проброс операндов, см вопрос 106

105. Как происходит разрешение конфликтов при помощи пузырька в конвейере?

Разрешение конфликта пузырьком. Это холостая "инструкция", коотрая проходит по этапам, заставляя их простаивать. Это похоже на пор но НЕ пор, потому что пор тоже надо читать и декодировать.

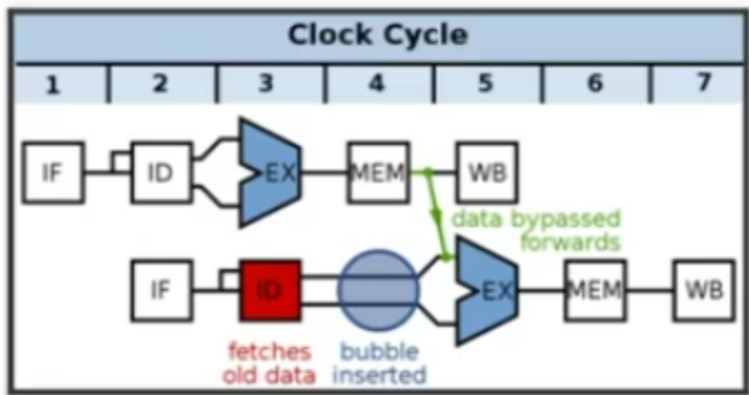
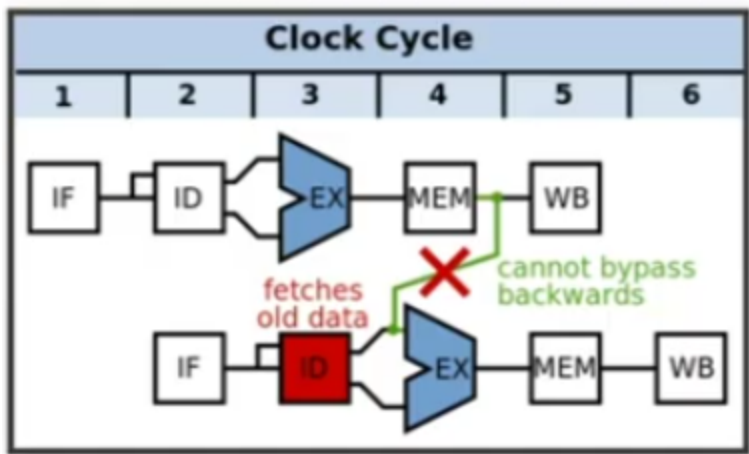
Пузырек не надо, он ничего не использует из ресурсов, они просто простаивают обрабатывая его. Результат - разносит во времени конфликтующие этапы.

Может разрешить и структурные конфликты и конфликты по данным

106. Как происходит разрешение конфликтов при помощи проброса операндов в конвейере?

Иногда бывает такое, что у нас есть конфликт по данным, когда одной инструкции нужен результат исполнения другой. Тогда мы можем не ждать пока инструкция запишет результат а сразу взять его с выхода АЛУ, без надобности ждать, пока инструкция завершит работу.

А вот таким образом решается, когда нам надо пробросить данные из будущего:



Т.е. и здесь можно применять пузырек, дабы дождаться хотя бы выпонения инструкции от которой мы зависимы (и даже так есть преимущество ведь иначе нам пришлось бы ждать еще дольше, до этапа WB инструкции, от которой мы зависимы)

<10 лекция>

107. Какие существуют способы разрешения конфликтов по управлению во время компиляции?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения

инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Решения во время компиляции (работаем с тем, что мы загрузим в процессор):

- Минимизация кол-ва переходов:
 - loop unrolling - сокращение числа переходов за счет разворачивания цикла (увеличивает кол-во инструкций, но уменьшает кол-во переходов и повышает производительность)

```
for (int x = 0; x < 100; x++) {  
    delete(x);  
}
```



```
for (int x = 0; x < 100; x += 5 ) {  
    delete(x);  
    delete(x + 1);  
    delete(x + 2);  
    delete(x + 3);  
    delete(x + 4);  
}
```

- условное перемещение данных - это когда мы используем специальные инструкции вроде `cmovle` (conditional move if less or equal), которые позволяют вообще обойтись без перехода
- Оптимизация работы предиктора переходов (см. вопрос 109-110) за счет того, что на этапе компиляции мы добавляем в инструкцию информацию о том, какой исход наиболее вероятен

108. Какие существуют способы разрешения конфликтов по управлению во время исполнения?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Решения во время исполнения (работаем с тем, как работает процессор):

- **bubble** - как только мы понимаем, что у нас идет инструкция перехода (на этапе Instruction Decode), мы останавливаем загрузку инструкций в конвейер, загружая вместо них bubble (это холостая "инструкция", которая проходит по этапам, заставляя их простаивать. Это похоже на порно HE порно, потому что порно тоже надо читать и декодировать. Пузырек не надо, он ничего не использует из ресурсов, этапы конвейера просто простаивают "обрабатывая" его), делаем так до тех пор, пока мы не поймем, как разрешится наш переход (пока инструкция перехода не будет обработана до конца).
- **Сброс конвейера** - когда мы обнаружили инструкцию перехода, мы продолжаем загружать в конвейер инструкции, но у нас есть возможность их сбросить, т.е. на каком-то этапе прервать выполнение (превратив их в "пузырек"), восстановить состояние измененных регистров и сразу начинать загружать в конвейер новые инструкции. По факту выигрыш перед прошлым пунктом в том, что если вдруг переход не состоится, у нас уже есть частично обработанные инструкции
- **Предсказание переходов** - Если мы видим, что у нас будет инструкция перехода, то мы пытаемся предсказать, произойдет переход или нет и начать выполнять инструкции согласно нашему предсказанию. Угадали = выиграли в производительности (подробно см. вопрос 110)

109. **Какие существуют способы предотвращения конфликтов по управлению через branch prediction? Что такое статические предсказатели переходов?**

Конфликт по управлению - конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо было.

Предсказание переходов (branch prediction) - если мы будем хорошо угадывать результат операций перехода (с вер. >50%), мы сможем частично решить проблему конфликта по управлению ведь сможем заранее загружать нужные инструкции и выигрывать в производительности

Предсказание может быть статическим и динамическим.

Статические предсказатели переходов содержат в себе некоторые правила, статистически на дистанции помогающие им делать хорошие предсказания.

Например, если у нас инструкция безусловного перехода, мы 100% знаем, что надо подгружать инструкции с того места, куда она переходит.

Другой пример, статистические исследования показали, что если у нас идет условный переход назад (условие с `jump` на адрес меньше текущего), то с вероятностью 90% мы все-таки прыгнем назад. Это вызвано исключительно спецификой кода, который пишут люди и связано с наличием большого количества циклов, устроенных таким образом. Тем не менее помогает предсказателю угадывать в 90% случаев в среднем.

`jump` вперед же выполняется 50/50, тут статистически ничего не выиграть, только угадать пальцем в небо.

110. Что представляют собой динамические предсказатели переходов в конвейере?

(повторим прошлый вопрос:)

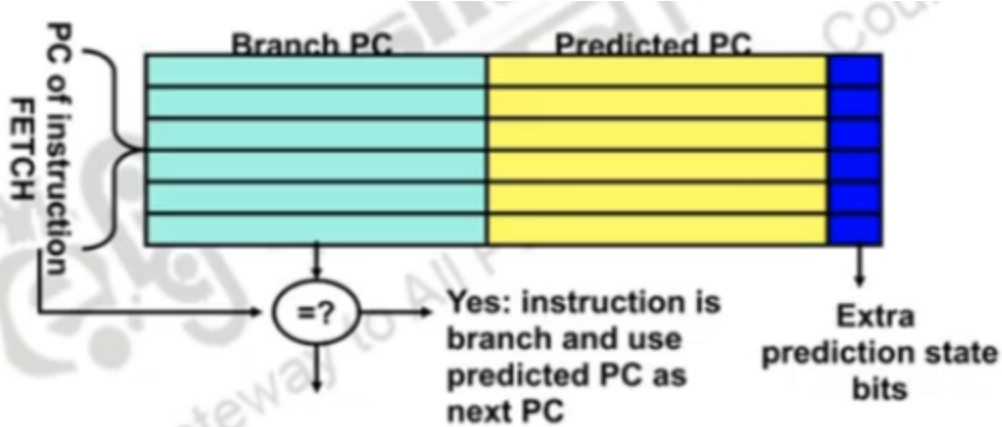
Конфликт по управлению - конфликт из-за операций условного/ безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Предсказание переходов (branch prediction) - если мы будем хорошо угадывать результат операций перехода (с вер. >50%), мы сможем частично решить проблему конфликта по управлению ведь сможем заранее загружать нужные инструкции и выигрывать в производительности

Предсказание может быть статическим и динамическим.

Динамические предсказатели переходов - когда мы стараемся угадать результат перехода непосредственно во время выполнения. Для этого у предсказателя есть таблица, в которой хранятся адреса инструкций перехода, для каждого адреса есть соответствие-

предсказание - куда мы можем перейти после этой инструкции.



Кроме этого, в этой таблице есть счетчик с накоплением (на рисунке синий) из двух бит, который увеличивается или уменьшается каждый раз, когда переход случается или не случается при исполнении инструкции условного перехода (переход случился - значит уверенность в том, что в следующий раз переход случится увеличилась и наоборот). Таким образом, на основе того, произошел ли переход на этой инструкции в прошлый раз процессор предсказывает, произойдет ли переход в следующий раз.

Есть еще многоуровневые динамические предсказатели переходов, которые умеют выявлять повторяющиеся паттерны, подробнее в лекции 10, где-то на 20 минуте

111. Как влияют различные виды конфликтов на производительность конвейера?

По сути они короче все снижают производительность/пропускную способность. Снизу нейронка достаточно хорошо описала почему именно каждый конфликт приводит к этому:

1. **Структурные конфликты:** Снижают **пропускную способность**. Если два этапа не могут поделить ресурс (например, одну шину памяти), один из них замирает. Это буквально «физическое» ограничение железа, которое не дает конвейеру работать на полной проектной скорости.
2. **Конфликты по данным:** Увеличивают **время ожидания (latency)** конкретной инструкции. Чем длиннее конвейер, тем больше тактов он будет ждать «свежий» результат от предыдущей команды. Без механизмов обхода (forwarding) такие конфликты могут замедлить выполнение программы в разы, превращая параллельную обработку в последовательную.
3. **Конфликты по управлению:** Самые «дорогие» потери. При неверном предсказании перехода приходится **сбрасывать (flush)** все инструкции, которые уже успели попасть в конвейер «по ошибке». Чем глубже конвейер, тем больше работы выбрасывается в мусор и тем больше тактов тратится на его повторное заполнение.

112. Как связаны конвейерное исполнение и спекулятивное исполнение инструкций?

Спекулятивное исполнение - это когда мы выполняем какую-то работу, еще не зная, пригодится нам ее результат или нет просто чтобы, например, не простаивать.

Соответственно в конвейерном исполнении это применяется как способ разрешения конфликта по управлению, когда мы заранее загружаем инструкции в конвейер еще даже не зная, произойдет ли у нас условный переход или нет.

113. Каковы шаги типового взаимодействия с устройством ввода-вывода? Каков интерфейс устройства для процессора?

Типовое взаимодействие:

- Конфигурация внешнего устройства
- Проверка состояния (готово ли оно?)
- Отправка данных
- Проверка статуса ответа (можно ли читать ответ?)
- Чтение ответа

Интерфейс ввода-вывода для процессора почти всегда выглядит как набор ячеек памяти или регистров, с которыми идет взаимодействие. Они подразделяются на:

- Регистры данных (полезная нагрузка)

- Регистры статуса

Над всем этим нам нужен протокол взаимодействия, иначе ничего, конечно, не будет работать.

114. Как реализуется ввод-вывод с точки зрения системы команд в Port Mapped IO?

Port Mapped IO (PMIO) - способ организации ввода-вывода, когда для регистров внешнего устройства у нас есть отдельное адресное пространство и, как правило, даже отдельные инструкции процессора, позволяющие с этими регистрами взаимодействовать. Напр. `in 15`, `out 20`

Прямо отвечая на поставленный вопрос: С точки зрения системы команд это реализуется отдельными инструкциями, с отдельной адресацией и вообще полностью изолированно от основной памяти

Достоинства PMIO

1. Минимизация логики управления (малое адресное пространство). Оптимизация IO.
2. Ввод-вывод и доступ к памяти разделены.
3. Адресное пространство памяти однородно.
4. Простота системы в целом.

Недостатки PMIO

1. Усложнение ISA и процессора.
2. Данные ввода-вывода — данные второго сорта (особенно для CISC процессоров).
3. "Лишние" копирования данных.
4. Очередное адресное пространство.

115. Как реализуется ввод-вывод с точки зрения системы команд в Memory Mapped IO?

Memory Mapped IO (MMIO) - способ организации ввода-вывода, когда наше устройство "подключается" по некоторым магическим адресам в RAM, в связи с чем читая или записывая по этим адресам на самом деле мы работаем не с памятью, а с внешним устройством. Для процессора выглядит как работа с памятью

Более формально:

1. Регистры внешних устройств отображаются в адресное пространство памяти.
2. Ввод-вывод реализуется через инструкции доступа к памяти.

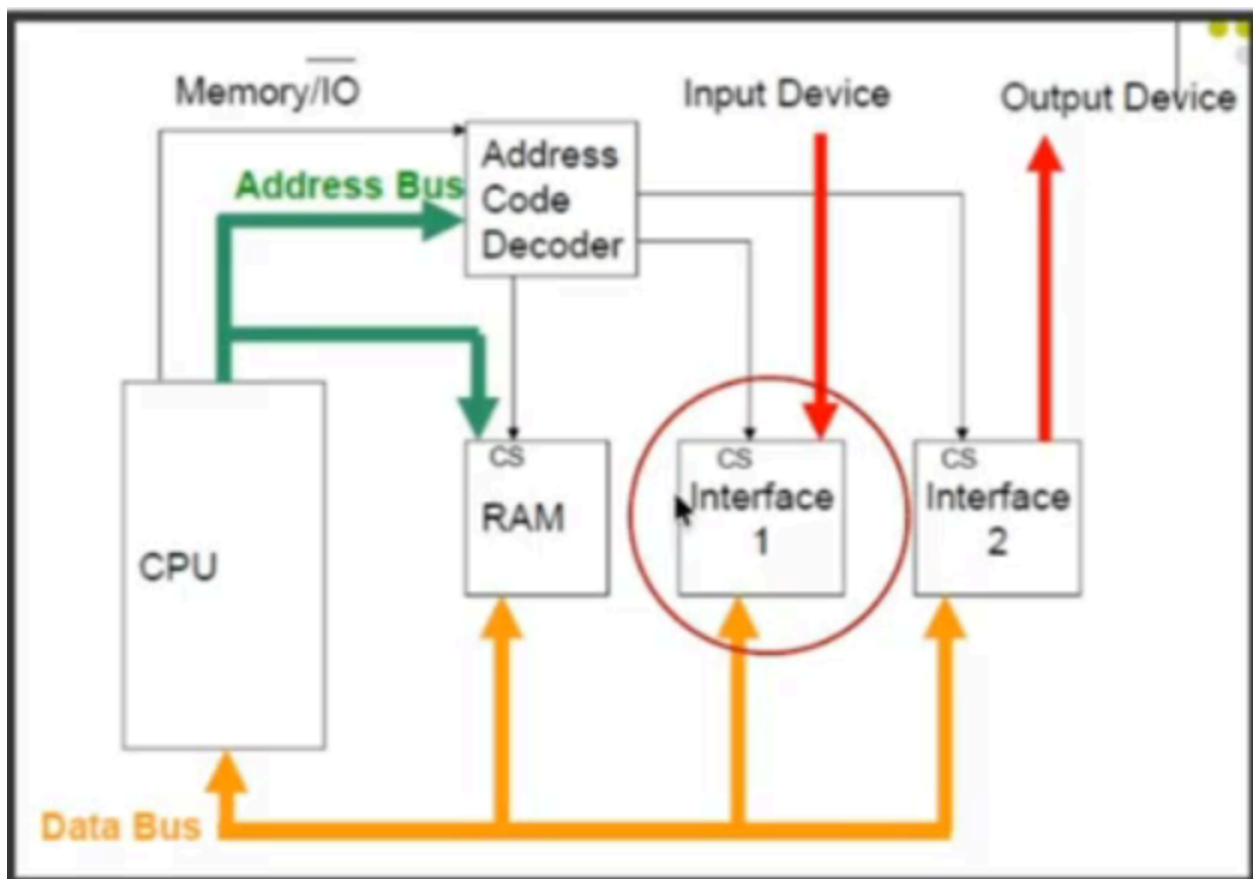
Прямо отвечая на поставленный вопрос: С точки зрения системы команд это реализуется теми же инструкциями, что и для работы с памятью, никаких дополнительных инструкций не вводится

Достоинства MMIO	Недостатки MMIO
1. Простота процессора. Без изменения микроархитектуры.	1. Одна шина для ввода-вывода и памяти. Разница скорости шин.
2. Единый набор механизмов доступа: автоинкремент, векторные операции, работа с барьерами.	2. Неоднородность памяти, сложная конфигурация системы.
3. Обработка данных без переноса в память.	3. Избыточный адрес для устройств ввода-вывода.
4. Адресное пространство памяти.	4. Проблемы с параллелизмом уровня инструкций и кешами: • flush при вводе-выводе; • порядок записи регистров.

116. Как устроен ввод-вывод в Memory Mapped IO с точки зрения устройства процессора?

Memory Mapped IO (MMIO) - способ организации ввода-вывода, когда наше устройство "подключается" по некоторым магическим адресам в RAM, в связи с чем читая или записывая по этим адресам на самом деле мы работаем не с памятью, а с внешним устройством. Для процессора выглядит как работа с памятью

"Под капотом" (=с точки зрения устройства процессора, его микроархитектуры) это устроено так:



Процессор обращается к памяти, но если это адрес IO, то Address Code Decoder выключает (с помощью cs - chip select) RAM и подключает нужный интерфейс, который выставит на шину нужное значение, а процессор его защелкнет как значение из памяти

<11 лекция>

117. Как работает программно-управляемый ввод-вывод без прерываний? Каковы его ограничения и типовой сценарий? Как связан с теоремой Котельникова?

Самый типичный подход к реализации программно-управляемого ввода-вывода без прерываний:

- Периодически опрашиваем устройство IO
- Если надо, реагируем

Проблемы в общем следующие:

- Высокое энергопотребление (процессор постоянно занят выполнением инструкций периодической проверки)
- тяжело совмещать с другими задачами
- тяжело обрабатывать сложный протокол ввода-вывода (типа SPI например)

! НО, он выигрывает у подхода с прерываниями, когда у нас очень плотный поток обрабатываемых событий. Выигрыш за счет

снижения накладных расходов на переключение контекста при прерывании !

Теорема Котельникова - это теорема про то что аналоговый сигнал можно полностью восстановить его дискретным представлением, если дискретизировать его с частотой, в два раза бОльшей максимальной частоты его спектра. И на самом деле теорема Котельникова здесь неприменима, потому что мы говорим о том, что при приеме бинарного сигнала (например, тактового) нам нужно зарегистрировать как минимум его высокое и низкое состояния (1 и 0) как минимум по одному разу, а значит считывать его как минимум с частотой в 2 раза больше чем он сам. Это условие для того, чтобы восстановить последовательность логических уровней на линии (как раз то, что встречается в задачах ввода/вывода), НЕ для того, чтобы восстановить всю временную диаграмму или аналоговый сигнал в точности.

118. Что такое дребезг контактов? В чём разница между уровнем и сигналом применительно к кнопке? Как с дребезгом бороться программно?

Дребезг контактов — это физическое явление, при котором в момент замыкания или размыкания механических контактов (кнопки, тумблера, реле) вместо одного четкого перехода уровня напряжения возникает серия быстрых, хаотичных микро-замыканий и размыканий

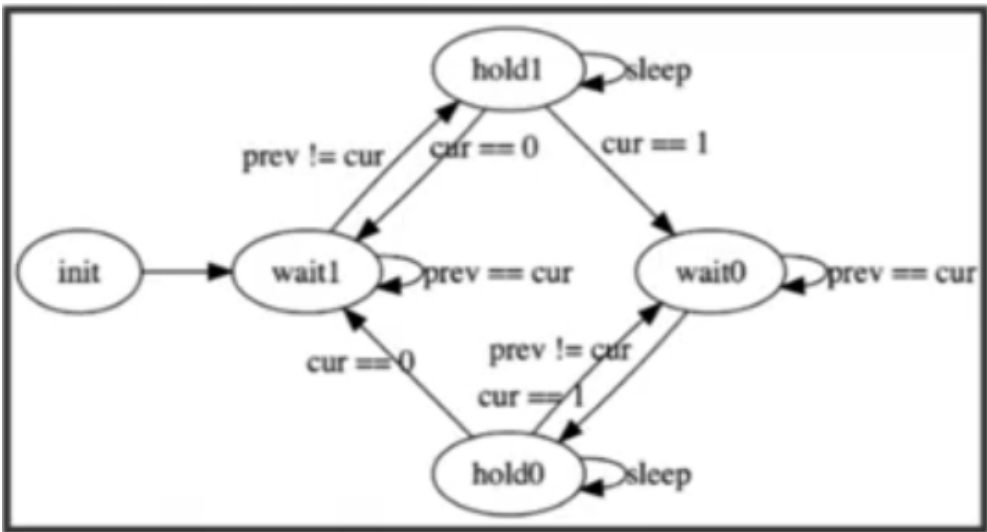
По сути **сигнал** в данном случае это изменения состояния кнопки, с не нажатого, на нажатое. **Уровень** же в момент дребезга слабо соотносится с сигналом, поэтому применяют стратегии антидребезга.

Программно можно после зафиксированного изменения состояния подождать, например сделать искусственную задержку. Можно сделать на счетчиках, доверяя уровню только если несколько раз подряд он был считан одинаково. Можно на таймере, не позволяя изменяться состоянию кнопки быстрее заданного порога. Другой способ описан в вопросе 119

119. Как реализовать псевдо-параллельный программно-управляемый ввод-вывод в одном потоке исполнения (state machine)?

Параллельный программно-управляемый ввод-вывод - это процесс ввода-вывода **без прерываний**, когда обработчику одного устройства не отдается весь поток управления (не занимает собой все время процессора), он делит этот поток с другими обработчиками.

Этот параллелизм можно сделать с помощью конечных автоматов. На примере опроса кнопок: у кнопки может быть состояние "нажата", и "не нажата". Чтобы избежатьдребезга контактов (потому что реальная кнопка не переключается четко из 0 в 1 и из 1 в 0, она быстро-быстро колеблется при переключении), переходы между этими состояниями могут осуществляться только после задержки. Например, так:



И вместо того чтобы занимать весь процессор тем, чтобы обрабатывать одну кнопку, мы делаем **неблокирующую** функцию, которая проверяет и актуализирует состояние кнопки (актуализирует состояние конечного автомата). Дальше в цикл мы можем понаставить кучу таких обработчиков разных кнопок и все они будут обрабатываться "одновременно".

Хоть и на самом деле при такой организации процессор в один момент времени работает только с одной кнопкой, за счет того что процессор очень быстро переключается между ними пользователю кажется что все кнопки обрабатываются одновременно.

это называется кооперативная многозадачность

120. Какие существуют уровни параллелизма: бит, инструкций и задач? Приведите примеры.

Виды параллелизма:

- Уровень битов

Мы обрабатываем не по одному биту, а сразу несколько.

Например, складываем 8-битные числа, проводим битовые операции сразу для 32-битного слова, пересылаем 32-битные регистры в память. Все это делается не для одного бита, а сразу для всех, получается параллельно

- Уровень команд

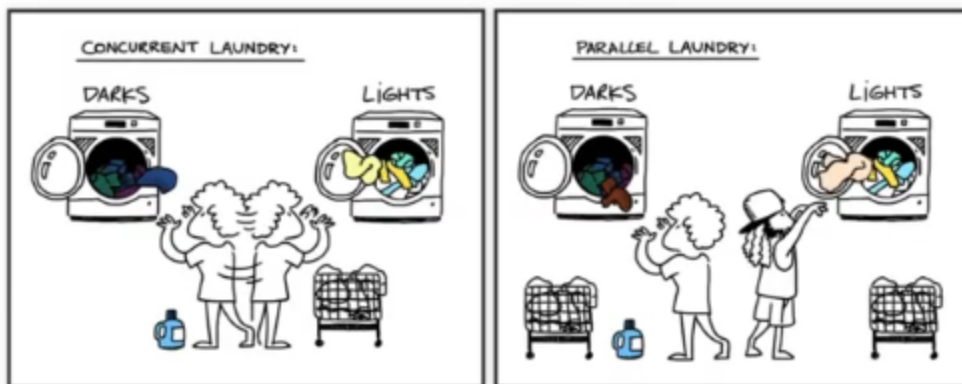
Когда мы можем выполнять сразу несколько инструкций параллельно. Например, с помощью конвейера, суперскалярности.

- Уровень задач

Когда мы выполняем несколько (в общем случае независимых) алгоритмов одновременно, параллельно

121. В чём разница между параллелизмом и конкурентностью (concurrency)? Приведите примеры.

Очень классная картинка:



Слева конкурентность (concurrency), человечек (исполнитель) один, но он по очереди выполняет две разные задачи (закидывает вещи в машинку), что при быстрой работе человечка можно считать параллельным наполнением обеих машинок.

Справа "реальный параллелизм", когда у нас два человечка исполнителя которые реально параллельно наполняют две машины (выполняют две задачи параллельно)

122. В чём заключается проблема реализации параллелизма уровня задач в архитектуре фон Неймана?

Основная проблема - архитектура фон Неймана не предназначена для такого параллелизма. У нас один поток инструкций, он не может "приостанавливаться", он работает пока не встретит `halt`. Поэтому приходится делать всякие костыли, а объединение двух

вычислителей архитектуры фон Неймана сразу порождает огромное количество проблем с синхронизацией, объединением памяти, тактовыми сигналами, конфликтами и прочим прочим прочим

123. Сравните кооперативную и вытесняющую многозадачность: достоинства, недостатки и область применения каждой.

Кооперативная многозадачность - когда у нас несколько процессов, но они все "дружат" между собой и готовы к кооперации, никто не будет делать нехорошие вещи и занимать поток надолго или блокировать его.

Достоинства:

- Атомарность задач (никто не прервет задачу, у нас сразу получается атомарность)
- Контроль за ресурсами со стороны задачи - задача сама решает, сколько ей надо взять ресурсов
- Легко реализовать (программная реализация достаточно проста и эффективна, как в примере с кнопками)

Недостатки:

- Контроль за ресурсами со стороны задачи - если задача случайно или специально зависнет, другие не получат выполнения
- Сбой одной задачи может стать глобальным сбоем
- Трудности в разработке интерактивных приложений (открываем файл блоком - и пока он не прочитается целиком с диска - интерфейс будет висеть)

Применяется кооперативная многозадачность в основном во встроенных системах, где зачастую нет проблемы доверия к выполняемым задачам в плане расходуемых ресурсов, где мы уверены что задачи будут кооперироваться и не жадничать. Ну и ограниченность ресурсов таких систем делает кооперативную многозадачность (которая проще в реализации) более выгодным вариантом.

А еще применяется как основа системы асинхронности языков программирования (например, `asyncio` в Python, где можно передать управление с помощью `await`)

Вытесняющая многозадачность супер подробно расписана в вопросе 127.

124. Какие механизмы необходимы для реализации кооперативной многозадачности?

Нужно реализовать механизмы:

- **остановки**: задача добровольно передает управление диспетчеру
- **сохранения** состояния задачи: все состояние всего ее окружение а затем как-то уметь восстановить
- **планирования**: диспетчеризация порядка выполнения задач (кто будет выполняться следующим?)
- **возобновление**: восстановить состояние, на котором задача прервалась, передать ей управление

Далее есть опциональные механизмы:

- **изоляция** задач: чтобы процессы не портили друг друга
- **взаимодействие** между задачами: очень часто стоит задача взаимодействия между процессами, они связаны

125. Как взаимодействуют кооперативная многозадачность и система прерываний?

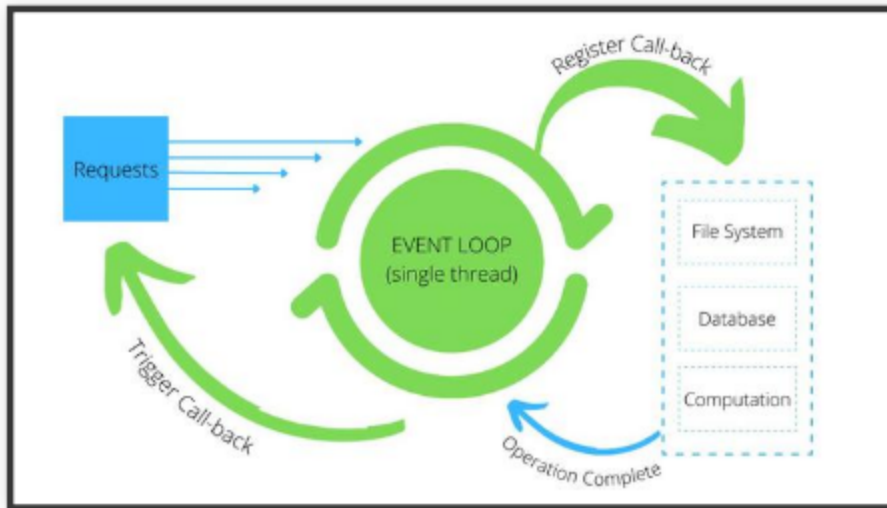
По сути в кооперативной многозадачности система прерываний не используется, если только механизм добровольной остановки задачи не реализован через вызов программного прерывания, что было бы странно, ведь проще делать это через системный вызов.

Короче пропустил я этот вопрос к сожалению, в лекции не сказано про это :(В целом просто можно поговорить про кооперативную многозадачность и систему прерываний

Кстати использование кооперативной многозадачности не отменяет аппаратные прерывания, и их все еще можно использовать для реализации протоколов / осуществления ввода-вывода и всяких таких штук, так что система прерываний и кооперативная многозадачность могут уживаться вместе спокойно

126. Как реализуется кооперативная многозадачность на уровне приложения через Event-loop?

По сути Event Loop - один из главных механизмов Node.js и один из способов реализации кооперативной многозадачности.



По центру у нас есть *цикл событий*, который проверяет нет ли новых запросов. Как только в очереди появляется запрос, он начинает его выполнять и выполняет, пока не нужно будет использовать что-то внешнее (диск, базу данных, еще что-то). Когда мы натываемся на это, мы создаем заявку в операционную систему (делаем системный вызов, который ОС выполнит впоследствии), в очереди регистрируем колбэк и переключаемся на другую задачу. Когда ОС (или какая-то внешняя система) даст нам ответ, event-loop это увидит при ближайшей проверке и сможет продолжить выполнение с того места, с которого надо собственно

127. Каковы достоинства и недостатки вытесняющей многозадачности?

Вытесняющая многозадачность - когда у нас несколько процессов, которые "думают" (могут думать) что они самые главные, а значит могут занимать весь поток управления, блокировать его и так далее. Для обеспечения многозадачности таких процессов, нужен внешний механизм, который будет распределять ресурсы между этими задачами, приостанавливать одни и запускать другие.

- Переключение процессов может происходить буквально между любыми двумя инструкциями
- Распределение процессорного времени осуществляется планировщиком

Достоинства:

- Простота разработки ПО. Нам не надо думать, когда отдавать управление диспетчеру, ОС сделает все сама

- Ресурсами рулит операционная система. Мы изолированы от других задач и защищены от недобросовестного использования ресурсов другими задачами
- Интерактивность системы (очень высокая скорость реакции)

Недостатки:

- "Лишние" переключения. Мы всегда распределяем время между задачами, это накладные расходы на переключение. Хотя иногда нам без разницы и было бы эффективнее выполнить сначала одну задачу целиком, затем другую.
- Процессы достаточно тяжелые (состояния со всеми регистрами, прерывания)
- Непредсказуемое переключение - нас могут прерывать в любой момент, повлиять мы на это не можем. Проблемы синхронизации и гонок из-за этого (нарушается атомарность операций, отсюда могут вытекать все классические проблемы многопоточного программирования)
- Непредсказуемая длительность работы из-за непредсказуемых прерываний (их обработка будет занимать какое-то, в общем случае непредсказуемое время)

Применяется в основном как основа операционных систем общего назначения

128. Какие механизмы необходимы для реализации вытесняющей многозадачности?

Необходимо реализовать механизмы (отличие от вопроса 123 только в первом пункте):

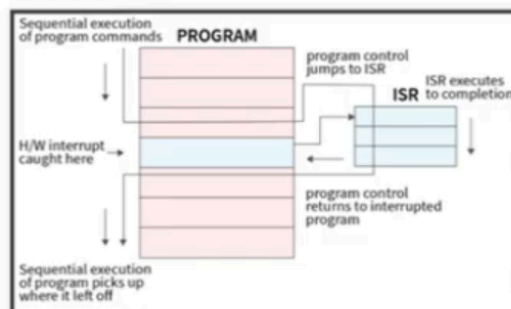
- ~~остановки~~: задача добровольно передает управление диспетчеру <- это то, как было в вопросе 123. Вот как здесь:
 - **прерывания**: способ забрать поток управления у задачи, независимо от желания задачи
 - **сохранения** состояния задачи: все состояние всего ее окружение а затем как-то уметь восстановить
 - **планирования**: диспетчеризация порядка выполнения задач (кто будет выполняться следующим?)
 - **возобновление**: восстановить состояние, на котором задача прервалась, передать ей управление
-

Система прерываний

Архитектура фон Неймана характеризуется:

1. последовательным исполнением команд и [без]условным переходом;
2. процессор будет "молотить" инструкции столько, сколько сможет;
3. оптимизация "число-дробилки" для одной задачи.

Система прерываний позволяет **переключиться** (совершить переход к заданной инструкции) **по внешнему событию** и **вернуться назад** после обработки.



Система прерываний - "хак" архитектуры, который нарушает ее линейность и позволяет внешним событиям нарушать ход выполнения инструкций (например выполнить какую-то процедуру по нажатию кнопки пользователем).

129. Что даёт переход от программно-управляемого ввода-вывода (polling) к вводу-выводу через систему прерываний с точки зрения эффективности и параллелизма уровня задач?

По сути преимущество, что нам не надо постоянно опрашивать состояние передачи и ждать готовности данных. Мы просто занимаемся своими делами, а когда данные будут готовы обрабатываем их в обработчике.

Также и при передаче: загружаем данные, дальше передача идет "сама" прерывая основной поток иногда на очень короткое время, чтобы вытеснить новые уровни на шине.

Так мы можем максимально быстро обрабатывать приходящие данные (не надо ждать следующую итерацию polling'a) и не занимать основной поток ожиданием, этим самым polling'ом. За счет прерываний можно очень четко выдерживать тайминги передач (чтобы следовать условиям протоколов и внешних устройств)

130. Как по шагам происходит обработка прерывания?

У нас есть основной поток, где у нас выполняются инструкции. У нас есть система прерываний, которая позволяет процессору

регистрировать какое-то событие (например, сработал аппаратный таймер).

Когда это происходит и событие зарегистрировано, мы сохраняем текущее значение Program Counter и переходим по вектору прерывания (это адрес процедуры-обработчика прерывания, зачастую хранящийся где-то в отдельном месте в памяти), начинаем выполнять эту процедуру.

После ее завершения, если не произошло еще одного прерывания (да, во время обработки одного прерывания может прилететь еще одно), мы возвращаемся туда, где прерывались. Прерывание обработано.

131. Почему появление системы прерываний приводит к автоматическому появлению параллелизма уровня задач? Охарактеризуйте его.

Системы прерываний без многозадачности существовать не может! Как только появляется система прерываний, сразу же появляется разделение - основная программа, и программа обработчик прерываний. И мы переключаемся между этими "задачами", обеспечивая многозадачность.

В целом это можно так не рассматривать, но в подавляющем большинстве случаев основная программа и обработчик прерываний - две абсолютно разные задачи, не синхронизированные между собой, еще и выполняющиеся параллельно независимо друг от друга. А значит многозадачность

В целом вот шаги формально:

1. Выполнение "основного" потока.
2. Запрос прерывания (HW).
3. Сохранение адреса возврата.
4. Вызов обработчика прерывания (ISR) — обычного кода в специальном месте.
5. Завершение ISR.
6. Восстановление адреса возврата.
7. Продолжение основного потока.

132. Как классифицируются прерывания по источнику (синхронные/асинхронные, аппаратные/программные)? Их отличительные особенности.

Глобально есть два источника прерываний:

- **Аппаратные** (вызывается железом)
 - **Внешние** или *асинхронные* (асинхронны относительно инструкций процессора)
Преполнение таймера, нажатие клавиши, приняли пакет по сети
 - **Внутренние** или *синхронные* (синхронные относительно инструкций процессора)
Деление на ноль, ошибка доступа к памяти, какая-то критическая ошибка
- **Программные** (вызывается инструкцией)
Взаимодействие программы и операционной системы.
Например, можно использовать для кооперативной многозадачности, вызывая диспетчер через программное прерывание

Аппаратные прерывания можно использовать для системы ввода-вывода, реализованной с прерываниями.

133. Каковы задачи контроллера прерываний? Что такое маскируемые и немаскируемые прерывания? Относительный и абсолютный приоритеты?

Контроллер прерываний (как реализация *системы прерываний*), обычно реализовывается аппаратно (можно достичь очень высокой скорости обработки прерываний). Его задача - фиксация события, триггерящего прерывание, переключение контекста на его обработчик и возврат по завершению обработки.

Прерывания можно разделить на:

- **Маскируемые.** Есть возможность запретить прерывание, установив, например, соответствующий бит в регистре разрешения прерываний
- **Немаскируемые.** Соответственно те прерывания, которые нельзя запретить. Обычно они ставятся на какие-то глобальные сбои аппаратуры, например, сбой памяти.

Из wiki, на лекции не упомянуто ->

Контроллер прерываний может также типы приоритизации:

- **Относительный.** Значит, что обработчик прерывания не может быть прерван другим прерыванием (даже более приоритетным)
- **Абсолютный.** Значит, что даже обработчик прерывания может быть прерван другим прерыванием, если оно является более приоритетным

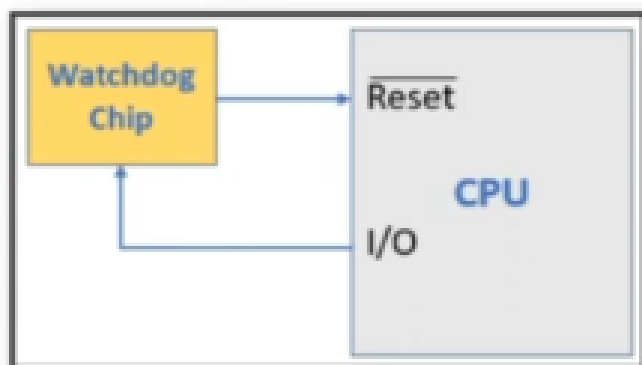
134. Какие виды событий существуют в системе прерываний (по фронту, по уровню, по сообщению, doorbell)? В чём особенности каждого?

Виды прерываний (событий, триггерящих прерывания):

- **По фронту.** Изменение уровня на проводе. Например из 0 в 1. Или из 1 в 0
- **По уровню.** Пока на линии есть какой-то уровень, будет триггериться прерывание
- **По сообщению.** У нас просиходит прерывание и его может вызвать несколько источников. А информация о том, какой именно источник вызвал это прерывание находится в определенном заранее известном месте.
- **По дверному звонку (Doorbell).** НАСКОЛЬКО Я ПОНЯЛ - Это прерывание, изолированное от процесса ввода/вывода и передачи данных. Т.е. нам например по какому-то протоколу идут данные. Прерывание по фронту будет давать только 1 бит информации, а прерывание по doorbell будет говорить "мы приняли очередное машинное слово, его можно забрать из нужного регистра" и процессор заберет его оттуда

135. Что такое сторожевой таймер и каков принцип его работы?

Сторожевой таймер - аппаратно реализованная схема контроля от зависания системы.



Как это устроено:

Есть система, которая имеет возможность перезагрузить весь

процессор (Reset) и имеет возможность получать сигналы от процессора. Система (Watchdog Chip) внутри имеет таймер, который сбрасывается по сигналу от процессора. Если таймер превышает некоторый порог, то система считает, что процессор завис и перезагружает процессор нафиг.

В результате: пока все работает штатно, все классно. Таймер периодически сбрасывается, сторожевой таймер не трогает процессор. Как только процессор завис и перестал сбрасывать таймер - сторожевой таймер жестко сбрасывает нас аппаратно, так куда больше шансов поддержать работоспособность системы в целом.

<12 лекция>

136. Какие подходы существуют к решению проблемы изоляции регистров и инструкций в памяти при многозадачности?

Параллелизм требует:

- Обеспечить **взаимодействие** между задачами (иногда это надо)
 - **Совмещение** задач по используемым ресурсам: по процессорному времени, по памяти (инструкций/данных/регистры)
 - **Изоляцию** между задачами
-

ответ по сути тот же что в 137, только изоляция регистров может быть:

- программным сохранением регистров в память (которая тоже как-то должна быть изолирована)
- лишь у нас физически два набора регистров которые переключаются

137. Какие подходы существуют к решению проблемы изоляции данных в памяти (статика, куча, стек) при многозадачности?

статика - она же *статическая память* секция .data

куча - она же *динамическая память*. можно выделить там себе

место, можно освободить. Находится под управлением выполняемой задачи

стек - он же ***автоматическая память***. выделяется и освобождается автоматически в зависимости от того, насколько глубоко мы ушли во вложенные процедуры

Многозадачность требует в том числе совмещение задач по используемым ресурсам, в частности по памяти. Но помимо задачи совмещения также стоит задача изоляции, чтобы одна задача не имела доступа к памяти другой задачи, иначе мы получаем уязвимости безопасности и ситуации, когда одна задача по глупости программиста крашит другую / крашит систему.

Есть три основных подхода к решению проблемы изоляции памяти, все они работают в runtime:

- **Банки памяти**

Сегодня уже мертвая технология. Классически это вообще не средство изоляции. Идея: давайте подключим к процессору несколько микросхем памяти. Например повесим их на одну шину и будем переключаться между ними, выбирая с кем именно мы сейчас работаем. Это можно использовать для аппаратной изоляции задач: задача физически не сможет прочитать что-то из другой микросхемы, ведь процессор позволяет ей читать только из своей.

- **Сегментация (сегментная память)**

Сегодня уже умирающая технология. Описана в вопросе 140.

- **Виртуальная память**

Самая применяемая на данный момент. Описана в вопросе 141.

138. Как банки памяти используются для расширения адресного пространства?

Идея банков памяти: давайте подключим к процессору несколько микросхем памяти. Тогда мы сможем, например, перевести процессор в определенный режим, режим записи в определенную микросхему памяти на шине и даже за счет этого получить возможность рулить бОльшим количеством памяти, чем позволяет процессор (=адресное пространство). Это и есть расширение адресного пространства (адресное пространство по сути

расширяется за счет предзаданного неявного бита/битов выбора микросхемы памяти).

139. Как банки памяти используются для расширения машинного слова?

Идея банков памяти: давайте подключим к процессору несколько микросхем памяти. Условимся, что в одном банке будут храниться младшие биты машинного слова, а во втором - старшие. Тогда при чтении каждый банк выставит на шину свою часть слова, а процессор будет иметь возможность прочитать информацию по объему БОльшую чем машинное слово каждого банка по отдельности.

Здесь есть еще приколы с трансляцией адреса для каждого банка. Представим, что наше машинное слово - 16 бит, банки памяти хранят по 8 бит.

Чтобы получить слово по адресу 0b10100 надо условно говоря прочитать байт по 0b10100 и байт по 0b10101 и склеить их. Поэтому можно считать что первый банк хранит четные адреса, другой банк - нечетные. По факту - в нашем примере каждый банк игнорирует младший бит и выдает байт по 0b1010.

Слегка запутанно описал, обсуждали это в начале 13 лекции

140. Что такое сегментная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

Сегментная память - идея о том, чтобы разделить физическую память на сегменты, каждый из которых будет характеризоваться его номером, адресом начала и размером сегмента. При доступе в память мы будем указывать: какой сегмент нам нужен, какое смещение относительно него нам нужно сделать. По таблице сегментов по номеру сегмента ищем адрес начала сегмента в памяти. Проверяем смещение - если смещение больше размера сегмента - выдаем ошибку.

Более того, можно делать сегменты только на чтение или запретить исполнение инструкций по определенным адресам, храня

метаинформацию об этом в таблице сегментов. В общем получаем много плюшек, которые помогут обеспечивать безопасность.

Таким образом можно изолировать память задач при многозадачности: каждую задачу кладем в свой сегмент, итог - никакая задача не сможет обратиться к памяти другой задачи, потому что получит ~~не-нам~~ ошибку адресации. А еще один и тот же сегмент можно подключить к разным задачам, обеспечивая совместное взаимодействие или, например, совместное использование библиотек.

Недостатки:

- Это **очень сложно**, особенно сложно написать компиляторы которые будут хорошо делить все на сегменты и поддерживать. Максимум что можно - разделить на какие-то большие сегменты, но теряется часть классной задумки где мы максимально дробим все на сегменты.
- **Внешняя фрагментация.** Когда при выделении/освобождении сегментов мы получаем маленькие кучоки памяти, которые не используются, потому что сильно меньше, чем нам нужны для новых сегментов.

141. Что такое виртуальная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

Сегментная память не получила распространения потому, что становится сложной. Получается нам надо придумать какой-то интерфейс, который бы сделал все гораздо проще.

Сделаем так, что все будет обставлено так, будто каждой задаче предоставлено все адресное пространство и она тут вообще одна и делает что хочет. Получается нам надо сделать какое-то отображение из адресов, которыми оперирует каждая задача в реальные адреса в памяти.

Делается это так - разбиваем всю память на небольшие кусочки, назовем их *страницы*. И каждая страница адресного пространства задачи будет отображена на такую же страницу куда-то в памяти. Получается чтобы обеспечить задачу памятью нужно, чтобы было понятно, куда в память будет отображаться каждая страница. Это дает возможность разбросать страницы в физической памяти как

угодно, при этом для задачи память будет выглядеть целостной и линейной.

А еще такой подход позволяет хранить некоторые страницы не в оперативной, а в любой другой памяти. Например, на жестком диске. Так работает swar

Чтобы это все работало нам нужна Page table (таблица страниц), которая по номеру страницы будет выдавать адрес начала этой страницы в памяти. К этому адресу мы прибавляем смещение и читаем, все.

Итог: задача думает, что у нее есть все адресное пространство (простота написания кода), хотя по факту она делит физическую память с другими задачами, не имеет доступ к памяти других задач, память этой задачи может быть собрана из кусочков, лежать отдельными кусками в физической памяти и т.д.

Отдельное большое преимущество, что отсутствует внешняя фрагментация памяти: мы выделяем память по страницам и нет требования на непрерывность выделяемых кусочков памяти, так что можно заполнять ее плотно, без промежутков.

Недостатки:

- Page table быстро становится просто огромной, и по ней долго искать адреса страниц. Надо городить какие-то кэши и все связанные с этим страдания
- Нет изоляции внутри задач. Задача может сломать сама себя, перезаписав свои инструкции, не очень хорошо, в сегментах мы могли это настроить.
- Нет предсказуемости времени доступа к памяти (попадем ли мы в кэш? будет ли страница в оперативной памяти или на жестком диске? Или вообще на магнитной ленте?)

142. Что такое внутренняя и внешняя фрагментация в контексте сегментной памяти?

см. 143

143. Что такое внутренняя и внешняя фрагментация в контексте виртуальной памяти?

Внутренняя фрагментация - когда у нас появляется неиспользуемое пространство внутри сегмента/страницы, потому что выделили больше чем реально надо.

Внешняя фрагментация - когда у нас появляется неиспользуемое пространство между выделенными сегментами/страницами. За счет требования к непрерывности выделяемого куска памяти маленькие кусочки, которые уже свободны могут оказаться невостребованными и простаивать.

144. Как появление сегментной памяти улучшило пользовательский опыт?

см. 140 для контекста

В части *пользовательского опыта* (опыта человека, который получает возможность запускать программы и получать от этого бенефиты):

- Теперь стало возможным запускать программы, которые не оглядываются на другие и используют все адресное пространство. Ранее такие программы ломали бы другие программы, сейчас они спокойно уживаются
- п.1 за счет изоляции памяти программ, ошибки одной программы в части менеджмента памяти перестали быть проблемой других запускаемых программ
- Появилась возможность создавать shared сегменты - подключить один и тот же сегмент (например, код библиотеки) к нескольким программам
- Если программа поддерживает несколько сегментов, появится безопасность в части настройки прав на каждый сегмент и доступа к ним

145. Как появление виртуальной памяти улучшило пользовательский опыт?

см. 141 для контекста

В части *пользовательского опыта* (опыта человека, который получает возможность запускать программы и получать от этого бенефиты):

- Теперь стало возможным запускать программы, которые не оглядываются на другие и используют все адресное пространство. Ранее такие программы ломали бы другие программы, сейчас они спокойно уживаются
- п.1 за счет изоляции памяти программ, ошибки одной программы в части менеджмента памяти перестали быть проблемой других запускаемых программ
- Появилась возможность расширить память программ за счет того, что страницы можно хранить где угодно, например на диске
- Более эффективное управление памятью со сниженной фрагментацией

146. Какие уровни (виды) задач существуют: основной поток, прерывание, процессы, потоки, зелёные потоки?

Типовые виды задач (условно):

1. **Main/Kernel/Основной поток** — исходный поток инструкций.
2. **Прерывания**. Особый исходный код обработчиков прерываний.
3. **Процессы**. Изолированные адресные пространства. Нет прямого доступа.
4. **Потоки**. Работают в адресном пространстве процесса. Прямой доступ ко всем его данным.
5. **Зелёные потоки**. В рамках Run-Time или виртуальной машины.

Зеленые потоки - это реализация кооперативной многозадачности в каком-то отдельном потоке. Например для обеспечения работы асинхронности для какого-либо языка программирования.

147. Как могут взаимодействовать процессы через основной поток? Какие бывают разделяемые ресурсы?

Иногда процессам надо взаимодействовать (они же все таки на одной машине выполняются, а значит потенциально могут решать схожие задачи, требующие обмена информацией между потоками). Но тут проблема - каждый процесс изолирован по памяти и имеет свое адресное пространство.

Пути решения:

- Shared Memory - когда некоторые страницы доступны сразу для всех процессов
- Signals - один процесс может вызвать "прерывание" в другом процессе. Например один процесс может попросить ОС вызвать функцию exit в другом процессе
- IO - можно вместе писать в файлы, через сеть, через трубы (pipes)

Организацию всех этих путей обеспечивает и менеджерит непосредственно ОС/ядро/основной поток

148. Что такое процессоры ввода-вывода? Каковы их назначение, интерфейс, достоинства и недостатки?

Процессор ввода вывода - отдельный процессор, стоящий рядом с основным. Ему во время работы основного процессора отправляется программа заточенная под задачи ввода/вывода, которую он начинает выполнять, а основной процессор занимается своими задачами. Это основана задумка и преимущество - разгружает процессор от рутинных задач ввода/вывода

Сейчас почти не применяется ввиду того что это оч сложная и специфичная конструкция. Заменена DMA с похожей концепцией но сильно проще.

149. Что такое контроллер прямого доступа к памяти (DMA)? Каковы его назначение, интерфейс, достоинства и недостатки?

DMA - Direct Memory Access - контроллер прямого доступа к памяти - некоторое устройство, заточенное под то, чтобы работать с внешними устройствами, но главная его задача - получить блок данных от внешнего устройства, разместив его где скажут в памяти или выдать блок и попросить отправить его внешнему устройству. По сути упрощенный Channel IO (процессор ввода/вывода) с тем же назначением - освободить процессор от рутинных задач ввода/вывода

Достоинства:

- DMA может быть заточен под память - по частоте, по системе команд, по организации работы с памятью и устройствами. Это

позволяет ему быть более эффективным

- Интерактивность, мы разгружаем процессор от задач ввода/вывода, они идут полностью автономно независимо от процессора
- Параллелизм, DMA может работать сразу с несколькими внешними устройствами параллельно

Недостатки:

- Сложности при непоследовательном доступе к памяти - DMA заточен под линейные непрерывные блоки
- Процессор перестает в полной мере контролировать системную шину в части доступа к памяти, надо ломать голову, как их синхронизировать

150. Какие существуют способы интеграции и взаимодействия контроллера прямого доступа к памяти (DMA) с процессором?

DMA имеет доступ к памяти наравне с процессором. Отсюда проблема конфликта, когда оба хотят получить доступ к памяти.

Как решить:

- **Пакетный режим.** Даем приоритет для DMA: пока обрабатывается очередной пакет, процессор ждет.
- **Циклический режим.** Работаем по циклам, в каждом из которых даем волю отдельно DMA, отдельно процессору
- **Прозрачный режим.** Процессор имеет приоритет по доступу к памяти над DMA. DMA работает по остаточному принципу.

151. Что такое гонка состояний (race condition)? Почему она возникает при вытесняющей многозадачности?

Гонка состояний - когда результат выполнения программы зависит от порядка выполнения задач. Этот порядок в случае вытесняющей многозадачности непредсказуем.

Возникает преимущественно именно при вытесняющей многозадачности, потому что именно этот подход предполагает, что задача может быть приостановлена вообще между двумя любыми инструкциями.

152. Как синхронизируются потоки? Что такое атомарная операция, мьютекс и семафор?

С целью избежать гонок состояний были придуманы разные ухищрения, в частности:

Атомарная операция - операция, выполнение которой не может быть прервано. Если она начала выполняться, она выполняется полностью

Мьютекс - некая программная сущность, которую одновременно может захватить только один поток. Реализована может быть разными способами, например с использованием разделенной памяти

Семафор - общий случай мьютекса. Некая программная сущность, которую одновременно может захватить не более N потоков. Реализована может быть разными способами, например с использованием разделенной памяти

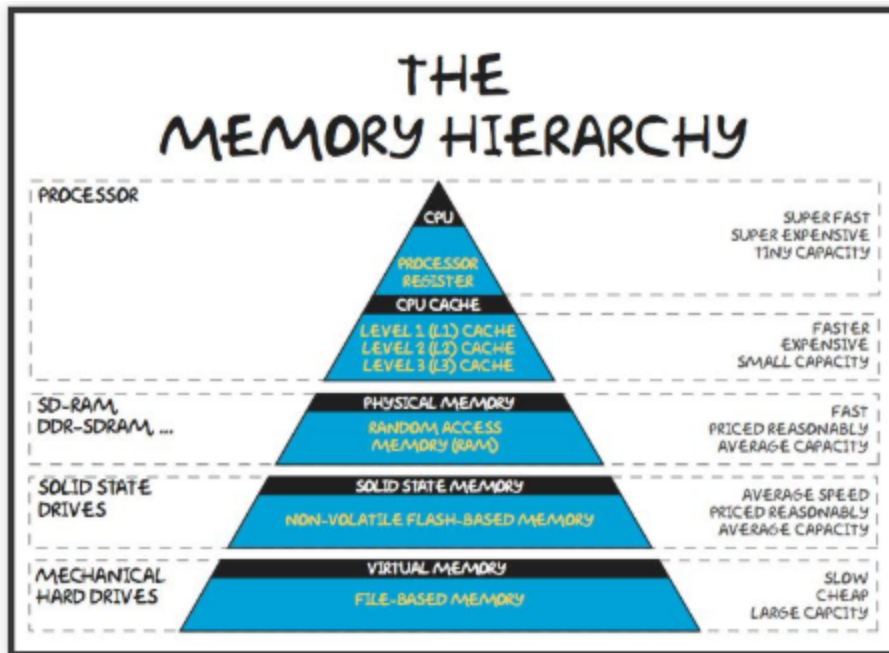
<13 лекция>

153. Почему необходима иерархия памяти в современных компьютерных системах?

Иерархия памяти возникает естественным образом из такого свойства памяти - чем память быстрее, тем она дороже. Из-за этого получается так, что у нас есть очень ограниченное количество быстрой памяти и очень много медленной. Возникает иерархия, "пирамида" когда мы комбинируем разные виды памяти, чтобы эффективнее решать свои задачи

154. Какие основные виды памяти входят в иерархию памяти и каковы их характеристики?

См. пирамиду памяти из курсов Клименкова. Ну или вот:



По скорости идут:

Процессор, его регистры -> его кэши -> физическая память -> ssd диски -> механические жесткие диски -> всякие сетевые диски, магнитные ленты и т.д.

155. **В чём различие между явной иерархией памяти и скрытой? Приведите примеры.**

Есть два типа иерархии памяти с точки зрения того, как это все выглядит для разработчика:

Явная иерархия - мы четко можем определить, какой тип памяти в какой момент времени для хранения чего конкретно будет использован. С расчетом на то, что мы как программисты лучше знаем, что лучше положить в SRAM что в DRAM что на HDD.

Скрытая иерархия - мы как программисты имеем доступ ко всей памяти, а наша система пытается на лету все подстроить и хранить так, чтобы было максимально быстро и эффективно. Плюс - не перегружает программистов и код, реализация "в лоб" с точки зрения программиста. Минус - усложняет систему, ну и не всегда это будет максимально эффективно

В современном мире на уровне работы с памятью чаще всего применяется скрытая иерархия, потому что виртуальная память объединяет в себе и кэши, и основную память, и память жестких дисков (скидывает в своп страницы, если требуется). Это и есть скрытая иерархия. При этом частичные признаки явной иерархии

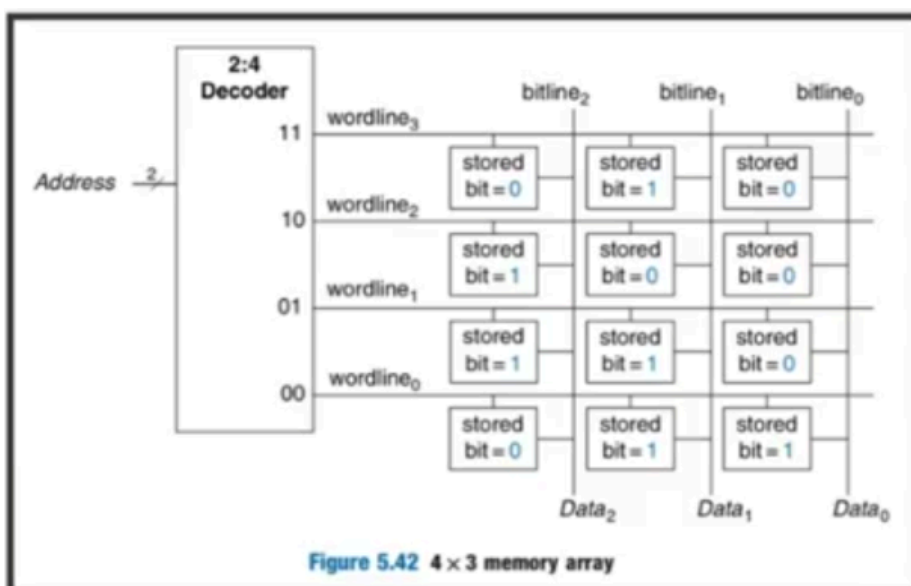
так или иначе тоже есть, например при работе с вводом-выводом, регистрами там и так далее

156. **Какие типы доступа к памяти существуют (произвольный, последовательный, гибридный)? Приведите примеры.**

1. С **произвольным доступом**, Random-Access Memory (RAM).
Задержка доступа не зависит от истории запросов.
2. С **последовательным доступом** (жёсткие диски, магнитные ленты, **AWS Tape Gateway**).
 - Длительная задержка при смене адреса.
 - Часто, высокая скорость последовательной записи/чтения.
3. **Гибридные**: библиотека магнитных лент, векторные операции.

157. **Каково устройство памяти с произвольным доступом (decoder, wordline, bitline)? Как устроена ROM-ячейка и как происходит чтение?**

Чаще всего устроено так - мы берем простейшие ячейки, которые умеют хранить один бит и собираем их в матрицу. Число столбцов соответствует числу бит в машинном слове, число строк - тому, сколько машинных слов мы сможем хранить. Сбоку подключается декодер, на который приходит адрес. Как только адрес приходит, декодер включает определенную строчку, заставляя ячейки, расположенные по этому адресу выставить на шину соответствующий бит.



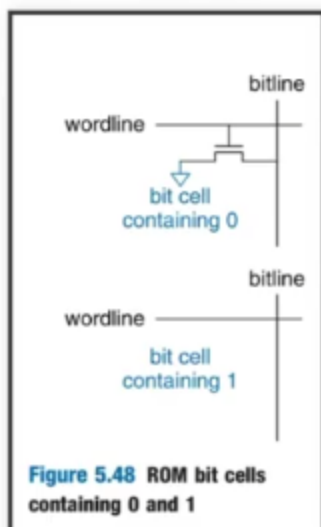
В целом это работает только для небольших матриц, банально потому, что сделать большой декодер достаточно сложно, что линии становятся очень большими для больших матриц и все это начинает

работать медленно и плохо. Поэтому в больших чипах все устроено чуть сложнее но базово именно так.

ячейка **ROM (Read Only Memory)** - ячейка памяти для постоянного хранения информации, базово позволяет только читать из нее.

Есть два типа таких ячеек - ячейка которая хранит "1" и которая хранит "0"

Которая хранит "1" не делает ничего, потому что и так по умолчанию на шину выставляется "1". Которая хранит "0" содержит транзистор - если данная ячейка выбрана декодером, то ячейка начинает замыкать линию в "0".



Часто производят такие чипы так - на всех местах по умолчанию есть транзистор, они все хранят "0". Но при первичной прошивке памяти мы выжигаем ненужные транзисторы и эти ячейки становятся единицами.

Чтение такой памяти происходит так - выставляем на битлайны единицу, декодер выбирает нужный wordline. Сигнал на линии wordline открывает ("замыкает") транзисторы в тех ячейках, где они есть. Эти ячейки начнут сливать ток в bitline и таким образом выставят на bitline 0. Там где транзисторов нет, так и останется единица. Слово прочитано!

158. Каково устройство многопортовой памяти? Как связана площадь с количеством портов?

В целом память с произвольным доступом можно сделать многопортовой. Для этого нам надо поставить второй декодер, проложить вторые линии wordline и bitline и дублировать обвязку

"сердца" ячейки памяти (в случае SRAM "сердце" - это два инвертора, а обвязка - два access FET'а)

SRAM Multiport

- SRAM so far can do either one read or one write/cycle
- We can do multiple reads and writes with multiple ports by adding one set of wordlines and bitlines per port
- Cost/bit? For N ports...
 - Wordlines: $\frac{N}{2}$
 - Bitlines: $\frac{2 \cdot N}{2}$
 - Access FETs: $\frac{2 \cdot N}{2}$
- Wires often dominate area $\rightarrow O(N^2)$ area!

Ну и чем больше будет портов тем кратно дороже будет память, тем больше площади на чипе все это будет занимать. Применяется крайне редко, потому что количество памяти останется таким же, а ее стоимость умножается кратно

159. Сравните технологии реализации битовых ячеек (Flip-flop, ROM, SRAM, DRAM) по числу транзисторов, скорости, плотности и стоимости. В какой области иерархии памяти каждая из них применяется?

Для реализации памяти нам полезно уметь делать битовые ячейки - некоторые конструкции, которые могут быть в двух устойчивых различных состояниях. Именно они и хранят нужную нам информацию

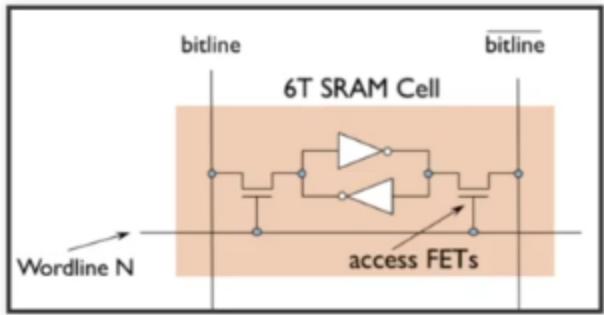
Тип ячейки	Число транзисторов	Задержка	Типовой объем	Цена за Гб	Г, В, р
Flip-Flop (это вот как раз триггеры всякие)	~20	ооочень маленькая, 20ps	1-5кБ	Оч много	В р
ROM	0-1	Чтение быстро, запись часто очень затруднена	-	В целом недорого, но depends	В п х н п

Тип ячейки	Число транзисторов	Задержка	Типовой объем	Цена за Гб	Г
		или невозможна			М
SRAM	6	наносекунды	10кБ-10МБ	~1000\$	К
DRAM	1	Десятки наносекунд	16ГБ-64ГБ	~10\$	С
					О
					П

160. Каковы технологии реализации SRAM ячеек? Как происходит чтение и запись?

Реализуется с помощью 6 транзисторов. Из них по два уходит на каждый из двух инверторов; и два для обеспечения доступа к ячейке.

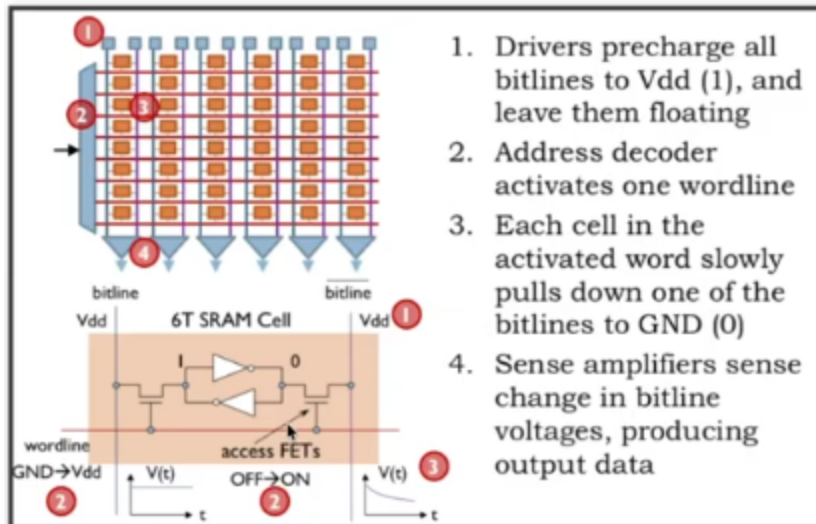
На картинке треугольники - инверторы, они же логическое "НЕ". Такая комбинация двух инверторов заставляет их поддерживать одно из двух устойчивых состояний, которые и интерпретируются как сохраненный бит



Чтение происходит следующим образом: когда декодер выставляет "1" на wordline, боковые транзисторы (access FETs) подключают инверторы к bitline которые по умолчанию оба подключены к питанию. И соответственно в зависимости от того, в каком состоянии сейчас инверторы у нас на линиях будет либо "1,0" либо "0,1" и специальное устройство amplifier на выходе интерпретирует эти

состояния как 1 или 0 и выдает его на шину. Более формально:

SRAM Read

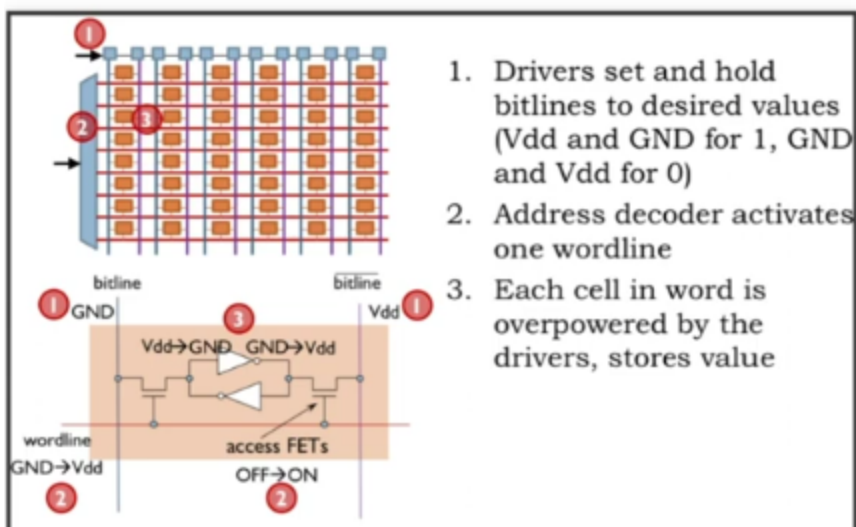


Запись происходит следующим образом:

Вместо того, чтобы оба bitline подключать к питанию. Мы одну линию подключаем к питанию, а другую к нулю. По итогу инверторы выбранной ячейки перестроятся под заданное состояние и примут новое устойчивое состояние.

Более формально:

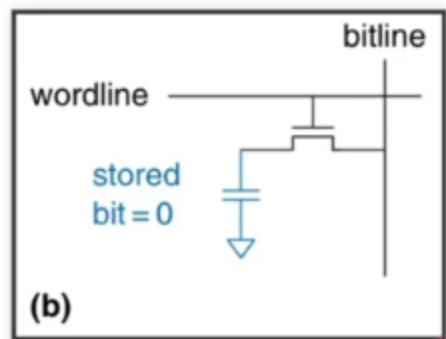
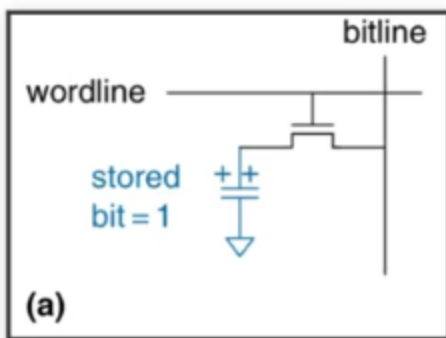
SRAM Write



161. **Каковы технологии реализации DRAM ячеек? Как происходит чтение и запись? Что такое регенерация и зачем она нужна?**

SRAM позволяет сделать память с 6 транзисторами на ячейку. Это очень дорого и расточительно, никуда не годится. Поэтому придумали DRAM (dynamic random access memory).

Состояние в ячейках DRAM хранится в конденсаторе, и 1 или 0 определяется тем, заряжен конденсатор или нет.



Из-за такого упрощения возникают проблемы:

- Заряд конденсатора утекает, его надо перезаряжать постоянно
- Считать значение можно только один раз, так как чтение бита разряжает конденсатор
- Нужен специальный контроллер, который постоянно будет перезаряжать нужные конденсаторы

Запись происходит так:

- Декодер выбирает ячейку через wordline.
- На bitline выставляем или 1 или 0.
 - Если 1 - конденсатор зарядится, если 0 - разрядится.
- Ячейка записана

Чтение происходит так:

- Выставляем на битовую линию половину напряжения, которое считается единицей
- Активируем wordline
- Если там была 1, то на линии мы увидим всплеск, пока конденсатор не разрядится до 0.5. Если там был 0, то линия просядет пока конденсатор не зарядится до 0.5
- Процесс чтения ломает заряд, который был в ячейке, поэтому после чтения надо восстановить его "записью"

Регенерация - процесс, когда контроллер памяти периодически производит процесс "чтения" памяти, обновляя заряды конденсаторов, чтобы они не утекали (точнее утекать они все равно будут, просто обновление не дает этому повлиять на нас).

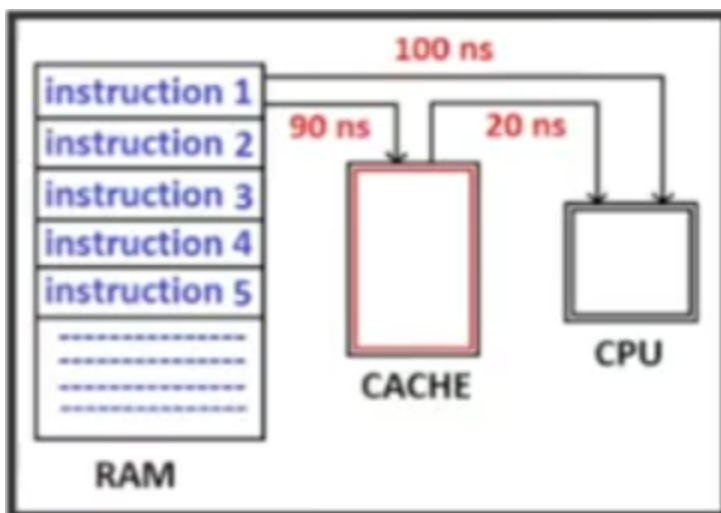
При этом по факту, конечно, когда мы говорим что мы выставляем что-то на битовую линию, подключаем ее к питанию или половине питания на самом деле подразумевается, что подключение идет вероятнее всего через сопроствление. Причина этого - ячейка должна суметь пересилить эту линию и выставить на ней нужный уровень, при этом не создавая больших токов. Но это небольшое отступление в сторону физики и электротехники

162. Что такое кеш? Каково его назначение, место в иерархии памяти и основные метрики?

Кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью

Назначение - ускорить доступ к памяти, пытаьсь обойти проблему "чем больше памяти, тем она медленнее (либо дороже)". Место в иерархии памяти между процессором и основной памятью.

Выглядит в архитектуре это так:



При этом для процессора по сути ничего не меняется

Метриками кэша являются:

- Скорость доступа
- Объем кеша
- Размеры кеш-линий
- Уровень ассоциативности
- Уровень попадания (hit rate) - как часто в среднем происходят cache hit по отношению к общему числу запросов

163. Что такое принцип локальности и каковы его виды (временная, пространственная) применительно к механизму кеширования?

Принцип локальности - идея о том, что по факту в прикладных задачах чаще всего доступ к памяти происходит неравномерно, т.е. в некоторой степени предсказуемо из-за локальности данных. Например, процессор последовательно читает инструкции по последовательным адресам

Есть временная и пространственная локальность данных (в лекции жестко не разделялось):

- **Временная локальность** - если мы обратились к какому-то адресу памяти, вероятно что процессор обратится к нему снова в ближайшем будущем
- **Пространственная локальность** - если процессор обратился к какой-то ячейке памяти, то велика вероятность что скоро он обратится к соседним ячейкам

164. Как пошагово происходит процесс чтения данных через кеш-память процессора?

Ситуация: мы хотим получить значение по некоторому адресу в памяти.

- Сначала наш запрос попадает в кеш, кеш начинает поиск на совпадение тега какой-либо строки с запрошенным адресом
- Если **кеш находит адрес** это называется **cache hit**, данные из внутренней таблицы кеша попадают сразу в процессор
- Если **кеш не находит адрес** это называется **cache miss**
 - Данные запрашиваются извне (из кеша другого уровня, либо непосредственно из основной памяти). Время доступа становится непредсказуемым - память может быть заблокирована кем-либо, кеш может иметь произвольное количество уровней
 - Кеш выделяет строку в своей внутренней таблице (свободную, если есть; либо перезаписывает какую-то из имеющихся)
 - Записывает в эту строку данные, помечает их тегом (адресом, с которого они были взяты)
 - Отдает данные процессору

165. Как пошагово происходит процесс записи данных через кеш-память процессора? Каковы политики записи при попадании (write-through, write-back) и при промахе (write-allocate, no-write-allocate)?

Ситуация: мы хотим записать данные по некоторому адресу в памяти.

Наш запрос попадает в кеш и дальше есть два варианта:

В кэше уже лежит эта ячейка памяти, **cache-hit**:

Тут есть несколько вариантов, их называют *политиками записи*, применяется одна из перечисленных:

- **Немедленная запись (write-through)** - обновляем ячейку в кеше, передаем изменение дальше. Таким образом значение будет обновлено на всех уровнях кэширования и в памяти. Происходит это все долго
- **Отложенная запись (write-back)** - мы обновили ячейку в кэше и все. В случае, если когда-то эту ячейку потребуется удалить из кеша, обновление пойдет на следующий уровень кэширования/в основную память
- **Гибридные варианты** - объединение предыдущих двух, не разбиралось подробно

В кэше нет этой ячейки памяти, **cache-miss**:

Тут тоже несколько вариантов, а именно два:

- **Запись с размещением (write-allocate)** - ячейка размещается в кеш, дальше все по сценарию **cache-hit**
- **Запись без размещения (no-write-allocate)** - запись производится напрямую в память, кеш не трогаем. Минус в том, что возможно было бы эффективно разместить ячейку в кеше, что позволило бы следующие операции записи выполнять куда быстрее

166. Что такое кеш-промах? Как промах по-разному влияет на чтение данных, запись данных и чтение инструкций?

Кеш-промах (cache-miss) - когда мы обращаемся к кешу с запросом определенного адреса (на чтение/запись), а этой ячейки в кеше не оказалось. Плохая ситуация, если случается часто и систематически, то может полностью убить все преимущества кеша

"Типы" кеш промахов:

- **Чтение данных** - Средняя задержка. Может компенсироваться параллелизмом уровня инструкций (в случае суперскалярности, например)
- **Запись данных** - Минимальная задержка. Запись может быть отложена, например в буфер
- **Чтение инструкций** - Большая задержка. Процессору все равно нужно будет дождаться, пока инструкция будет загружена

<14 лекция>

В лекции рассматривается как вообще работает кэш:

Кеш - это память (практически всегда это SRAM, быстрая удобная, не надо регенерировать) и логика (вся обвязка которая делает просто кучу ячеек именно кэшем)

Логика должна уметь:

1. Искать кэш-линию (машинное слово например) по тегу
2. Понимать какая кэш-линия сейчас самая бесполезная, уметь замещать ее новой информацией
3. Предзагружать данные, инструкции
4. Менеджерить работу с памятью (группировать операции и все такое прочее)
5. Взаимодействовать с кешами других уровней

Масштабировать базово кеш (=логику кеша) можно по времени или по площади:

Ставим один компаратор (1 единица площади), но тогда он должен будет проверить в соло все кэш-линии (N единиц времени)

Либо ставим N компараторов (N единиц площади) и они все проверяют свою кэш-линию (1 единица времени)

167. Что такое ассоциативность кеша? Какова "структура" адреса с точки зрения кеша?

Ассоциативность кэша - способ выбора кэш-линии под конкретную ячейку памяти.

Т.е. нам приходит информация, у нее есть тег (идентификатор области внешней памяти). В какую ячейку кеша ее сохранить? На

этот вопрос отвечает ассоциативность. (это прямо базовое-базовое определение, его надо знать 100%)

Выбор ассоциативности кеша может сильно повлиять на скорость поиска в кеше. Пример:

- Если мы кладем в первую попавшуюся ячейку, то чтобы найти какую-то ячейку в кеше придется проверить их все в худшем случае. $O(N)$
- Можно выстроить что-то поинтереснее, хранить по определенным правилам, тогда и поиск будет быстрее (как в программировании: бинарное дерево, поиск $O(\log(N))$ / хешмап вообще за $O(1)$)

Структура адреса с точки зрения кеша (а надо ли нам сохранять весь адрес как тег хэшлинии?):

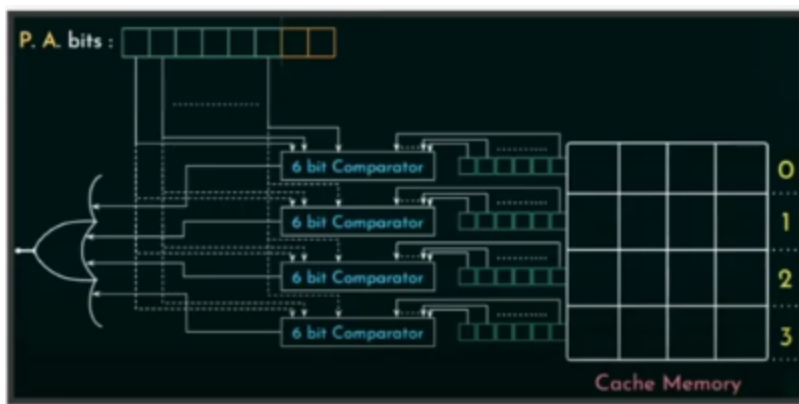
- **Младшие биты.** Можно отбросить, потому что хеш оперирует блоками а не байтами. Например, если блок хеша 64 байта то можно спокойно отбросить 6 бит адреса
- **Средние биты.** Циклически повторяются, когда мы читаем память последовательно.
- **Старшие биты.** +- уникальны для большого куска памяти
- **Очень старшие биты.** Можно тоже отбросить, потому что они часто вообще в памяти не используются (64 бита адреса могут адресовать колоссальный объем памяти, из-за этого старшие биты часто не используются)

168. Каковы особенности реализации полностью ассоциативного кеша?

(ассоциативность кеша - характеристика кеша, определяющая в какую из кеш-линий может попасть ячейка/блок информации в зависимости от его адреса)

Решение "в лоб" - любой блок памяти может попасть в любую кеш-линию. Для каждой кеш-линии стоит компаратор, который сравнивает пришедший адрес с тегами кеш-линий. Приходит адрес - компараторы сравнивают, если кто-то нашел, данные выставились, пошел доступ

Из адреса отбрасываются только младшие биты (на картинке их два отброшено, потому что в каждой кеш-линии хранится 4 байта)



Особенности:

- Очень много логики, все эти компараторы это достаточно дорого и сложно. Логика вытеснения может быть произвольной сложности
- Но достаточно эффективен

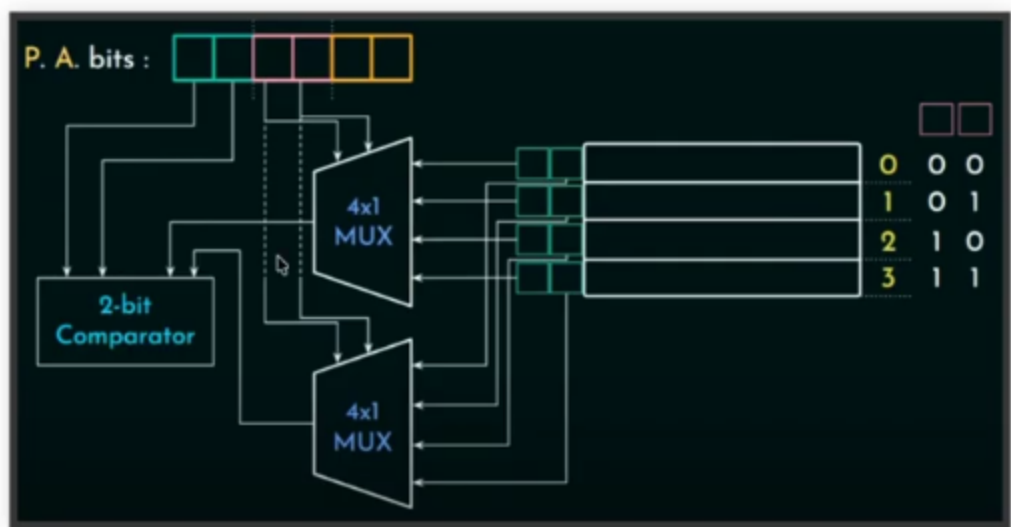
169. Каковы особенности реализации кеша с прямым отображением?

Возьмем адрес, отрежем у него младшие биты потому что мы храним блоки. Возьмем столько средних битов, сколько кеш-линий у нас есть (это будет номер нашей кеш-линии. Эквивалентно остатку от деления адреса на размер кеша). А в тег запишем только оставшиеся старшие биты.

Тогда при поиске мы точно будем знать, в какой ячейке кеша искать запрошенный адрес (просто берем средние биты, это и будет номер кеш-линии). Берем оттуда тег, если он совпадает с запрошенным адресом, значит мы попали в кеш (cache-hit), пошел доступ к данным

На картинке два младших бита обрезаны потому что в каждой кеш-линии хранится 4 байта. Два средних бита используются как индекс ячейки кеша, потому что у нас $2^2=4$ кеш-линии, ну и для однозначной идентификации два зеленых старших бита

записываются в тег.



Особенности:

- В теге хранится куда меньше информации
- Упрощение за счет того, что компаратор теперь один
- Нет вариативности логики вытеснения - такая ассоциативность четко задает куда мы должны записать очередной блок

170. Что такое множественно-ассоциативный кеш и что определяет уровень множественности?

Множественно-ассоциативный кеш - гибрид прошлых двух. Мы берем кеш с прямым отображением и повторяем его несколько раз. Таким образом мы теперь для одного индекса можем хранить несколько блоков, что неплохо.

Как ищем - берем адрес и ищем его в каждом подкеше с прямым отображением как обычно, получаем несколько блоков и смотрим с кем у нас совпадает тег.

171. Как связана ассоциативность кеша с механизмом вытеснения и замещения?

Механизм вытеснения и замещения нужен нам тогда и только тогда, когда для одного блока памяти по определенному адресу может быть несколько возможных мест в нашем кеше. Например для кеша с прямым отображением не нужна логика вытеснения, замещения, потому что там одному адресу строго соответствует одно место в кеше

172. Как работает механизм вытеснения и замещения LRU в кеше?

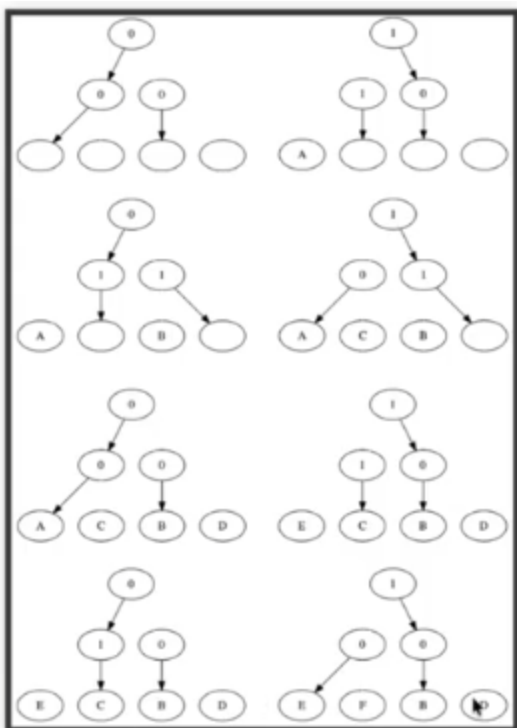
Когда нам надо понять, какую кеш-линию можно наименее болезненно удалить из кеша возникает проблема как понять, какая именно кеш-линия самая ненужная сейчас.

Есть тип такого мезанизма как LRU (last-recently-used) когда мы считаем что наверное самая ненужная линия это та, к оторой уже давно никто не обращался. Как понять какая? Для этого мы заводим глобальный счетчик доступов к кешу (это будет наш "timestamp") и при чтении/записи в кеш-линию будем записывать текущее значение счетчика. Если требуется вытеснить какую-то линию, то вытесняем ту, у которой записанное значение минимально.

Звучит неплохо, на деле очень сложно реализовать сортировку этих значений на лету, плюс у счетчика должна быть как-то органичена разрядность, а это проблемы с переполнением и короче сейчас такая схема используется очень редко из-за сложностей в реализации.

173. Как работает механизм вытеснения и замещения PLRU в кеше?

Гениальный способ определения самой ненужной ячейки - псевдо LRU. Как он работает, нарисовано на картинке:



Мы создаем бинарное дерево, где его листья - наши кеш-линии, а в любой другой вершине либо 0 либо 1. "0" означает стрелку влево, "1" означает стрелку вправо. При доступе к какой либо ячейке мы меняем все 1 на 0, а 0 на 1 по пути к этой ячейке через дерево. Таким образом получается, что дерево всегда будет указывать на "самую ненужную" ячейку, которую можно перезаписать

Так как это аля бинарное дерево, число листьев (кеш-линий) должно быть степенью двойки, правда в кешах это так и есть зачастую. Число ячеек для реализации такого дерева - $N-1$ **бит**

174. Почему необходимо использование многоуровневой иерархии кеш-памяти в современных процессорах?

Мы вынуждены использовать многоуровневую систему, потому что мы одновременно хотим большой и быстрый кеш, но при этом не хотим отдавать много денег/места/энергии.

Поэтому придумали сделать многоуровневую структуру, которая является компромиссом по объему/скорости/стоимости.

Классифицировать многоуровневые кеши можно по 3 признакам:

- Разделенный/унифицированный
 - Включающий/исключающий
 - Частный/общий
-

175. В чём разница между разделённым и унифицированным кешем и где они применяются?

Разделенный многоуровневый кеш: Мы создаем два физически разных кеша, один будет отвечать за данные, другой за инструкции. И получается что мы на уровне кешей имитируем гарвардскую архитектуру, хотя память и канал доступа к памяти один! Получаем возможности конвейеризации без конфликтов по доступу к памяти. Лучше работают каждый из кешей за счет бОльшей локальности обрабатываемых данных

Разделённый кеш (Split)

- 👍 **Достоинства:**
 - **Параллельный доступ (Нет структурных конфликтов):** Конвейер процессора может одновременно выбирать следующую инструкцию кода (из L1i) и читать/писать переменную для текущей операции (из L1d). Они не стоят в очереди друг за другом.
 - **Оптимизация под тип данных:** Кеш инструкций (L1i) можно сделать только для чтения, что упрощает его схемотехнику (не нужны цепи записи).
- 📉 **Недостатки:**
 - **Жесткое разделение объема:** Если программе нужно очень много данных, но сам её код крошечный, половина кеша (L1i) будет простаивать пустой, а во второй половине (L1d) будет жесткая нехватка места.

Унифицированный многоуровневый кеш: у нас просто один кеш :/

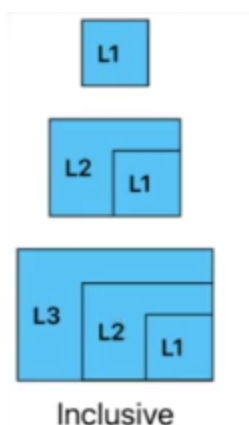
Унифицированный кеш (Unified)

- 👍 **Достоинства:**
 - **Гибкость использования пространства:** Память выделяется динамически. Если коду нужно много данных, они займут 90% кеша. Если выполняется сложный код с минимумом данных, кеш забьется инструкциями. Пространство не пропадает зря.
- 📉 **Недостатки:**
 - **Конфликты за доступ (Structural Hazards):** Если процессору нужно одновременно считать инструкцию и прочесть переменную из памяти, им приходится вставать в очередь к одной и той же матрице кеша, что создает задержку в один такт.

Обычно по уровням может отличаться разделенность/унифицированность.

176. В чём разница между включающим и исключаящим кешем и где они применяются?

Включающий многоуровневый кеш:



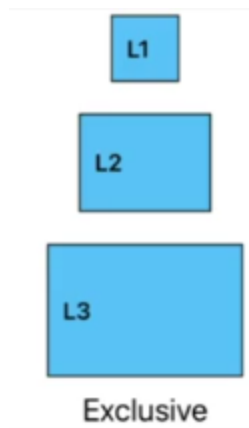
Все, что хранится в L1 точно хранится в L2 и L3. Все что есть в L2

точно есть в L3.

Включающий кеш (Inclusive)

- 👍 **Достоинства:**
 - **Простое поддержание когерентности (Актуальности):** Чтобы проверить, нет ли у какого-то ядра измененной переменной, процессору достаточно заглянуть в общий L3. Если во включающем L3 этой строки нет — значит, её гарантированно нет ни в одном приватном L1/L2 во всем процессоре. Искать по всем ядрам не нужно.
- 🗡️ **Недостатки:**
 - **Неэффективное использование памяти (Дублирование):** Маленькие и очень дорогие кеши L1 и L2 полностью копируются внутрь L3. Полезный суммарный объем всей системы кешей уменьшается.

Исключающий многоуровневый кеш:



Каждый кеш сам за себя.

Исключающий кеш (Exclusive)

- 👍 **Достоинства:**
 - **Максимальная полезная емкость:** Строка данных находится строго на одном уровне (или в L1, или в L2, или в L3). Суммарный объем кеш-памяти процессора равен $L1 + L2 + L3$, ничего не дублируется. Это критично при малых объемах кеша.
- 🗡️ **Недостатки:**
 - **Сложный поиск и когерентность:** При промахе в L1 процессору приходится делать операцию перекалывания (Eviction): выталкивать старую строку из L1 в L3, а новую забирать из L3 в L1. Это увеличивает трафик данных внутри процессора. Для проверки актуальности данных приходится опрашивать все приватные кешы всех ядер.

177. В чём разница между частным и общим кешем и как они применяются в многоядерных процессорах?

Разделение актуально для многоядерных систем

Частный кеш: только одно ядро имеет доступ к этому кешу

1. Частный кеш (Private Cache)

- 👍 **Достоинства:**
 - **Высокая скорость доступа ядра:** Кеш находится физически близко к конкретному ядру, имеет минимальные задержки (latency) и работает на максимальной частоте. Нет очередей из других ядер.
- 🗋️ **Недостатки:**
 - **Дублирование данных:** Разные ядра могут одновременно загрузить в свои личные кеши одну и ту же строку из ОЗУ. Память расходуется неэффективно.
 - **Конфликты (проблема когерентности):** Если Ядро №1 изменит переменную в своем частном кеше, а Ядру №2 понадобятся эти данные, возникнет конфликт — данные у Ядра №2 устарели. Требуются сложные протоколы синхронизации (например, MESI), чтобы избежать ошибок в вычислениях.

Общий кеш: несколько ядер имеет доступ к этому кешу

2. Общий кеш (Shared Cache)

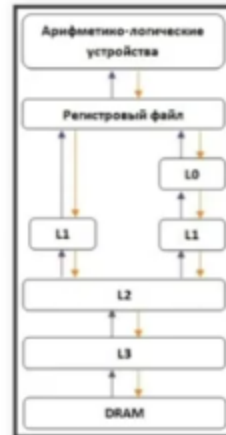
- 👍 **Достоинства:**
 - **Меньше кеш-промахов:** Весь объем кеша доступен любому ядру динамически. Если одному ядру нужно много памяти, оно может занять хоть весь общий кеш целиком.
 - **Нет дублирования:** Каждая строка данных из ОЗУ хранится в таком кеше в единственном экземпляре. Все ядра читают из одного источника.
 - 🗋️ **Недостатки:**
 - **Постоянная синхронизация доступа:** Поскольку несколько ядер одновременно стучатся в один пул памяти, на шине возникают «пробки». Нужна сложная арбитражная логика, которая выстраивает ядра в очередь. Это увеличивает задержки доступа по сравнению с частным кешем.
-

По уровням частность/общность кеша может отличаться

178. **Что представляет собой типовой многоуровневый кеш: L1, L2, L3, L4? Каковы их назначение, тип ячеек, классификация?**

Уровни кеширования в процессорах

- **L0** (опция) — специальный кеш для: стека, int/float чисел. Обычно доступен за такт.
- **L1** — быстрый, неотъемлемая часть процессора. Обычно разделён (Banked).
- **L2** — обычно часть процессора. От 128 Кбайт до 1-12 Мбайт. Обычно общий (Shared).
- **L3** до 24 Мбайт и более. Синхронизация данных в многоядерных процессорах.
- **L4** (экзотика для серверов и мейнфреймов). Оптимизация интерфейса доступа к памяти.



L0 - очень специфичный кеш очень близко к процессору который используется его внутренними структурами, сопроцессорами и очень сильно с ними завязан и фактически неотделим. Чаще всего даже не выносится как отдельный кеш

L1 - тоже достаточно неотделимый, очень оптимизирован под конкретный процессор, под структуры конкретного процессора. Чаще всего частный, разделенный под имитацию гарвардской архитектуры.

L2 - уже такие, хорошие может объемы в себе хранить, до десятка мегабайт. Обычно разделяется между процессорными ядрами

L3 - самый большой. За счет простоты доступа к L3 (к нему один канал доступа как видно по картинке, в отличие от L2) реализуется малым кол-вом логики что хорошо сказывается на его объеме и оптимизации.

L4 - появляется на практике редко и обычно только на большом серверном оборудовании, когда специфика задач тесно связана с памятью и мало связаны между собой

179. Что такое когерентность кеш-памяти и каково её значение для многоядерных систем? Какие существуют возможные состояния кеш-линий?

Когерентность - целостность данных в кешах процессора между собой и в памяти. Т.е. ситуация, когда все ядра при чтении будут получать одинаковые и более того актуальные данные, согласованные с RAM.

В многоядерных системах с этим бывают проблемки потому что ядра могут осуществлять параллельный доступ к данным на чтение/запись + DMA еще может работать и все такое.

Для решения этой проблемы придумали хранить состояния кеш линий, ну и целый протокол на его основе - MOESI:

- **Modified** - текущее ядро изменило эту кеш линию, здесь свежие данные и они есть только в этом ядре. Память об этом еще не знает
 - **Owned** - текущее ядро является владельцем данных и готово выдать другим ядрам корректные данные, причем другие могут хранить копию только в состоянии Shared
 - **Exclusive** - данные в линии есть только у текущего ядра и они актуальны, совпадают с тем что в оперативной памяти. Ядро может спокойно туда писать.
 - **Shared** - данные актуальны, но свободно их можно только читать. Если хочется изменить, придется сначала инвалидировать их в других ядрах, а свое перевести из Shared в Modified или Exclusive
 - **Invalid** - Данные невалидны или пустая строка
-

Честно, у меня ломает мозг от всех этих состояний, так что вот очень наглядные примеры, как эти состояния меняются и ядра взаимодействуют друг с другом:

Сценарий 1: Рождение и Чтение данных (Прوماхи и Обмен)

Представим, что изначально данных в кэшах нет (**Invalid**).

- Ядро 1 читает данные из памяти:
 - Было: **Invalid**
 - Стало: **Exclusive (E)**. Ядро 1 понимает: *«Я первый и единственный, кто взял эти данные, они совпадают с памятью».*
- Теперь Ядро 2 хочет тоже ПРОЧИТАТЬ эти же данные:
 - Ядро 2 делает запрос. Ядро 1 «слышит» это.
 - Переход: У Ядра 1 состояние меняется из **Exclusive** → **Shared**.
 - У Ядра 2 состояние становится **Shared**.
 - *Итог:* Оба сидят в **Shared (S)**. Данные у обоих одинаковые и совпадают с памятью.

Сценарий 2: Кто-то хочет ЗАПИСАТЬ (Борьба за уникальность)

Снова сидим в ситуации, когда у Ядра 1 и Ядра 2 данные в состоянии **Shared**.

- Ядро 1 решает ЗАПИСАТЬ (изменить) данные:
 - Прямо в *Shared* писать нельзя. Ядро 1 кричит по шине: *«Инвалидировать этот адрес!»*.
 - Переход у Ядра 2: **Shared** → **Invalid** (его копия мгновенно «умирает»).
 - Переход у Ядра 1: **Shared** → **Modified** (M). Теперь у него уникальные, измененные данные. Память устарела.

*(А если бы Ядро 1 изначально было в состоянии **Exclusive**, оно бы перешло в **Modified** молча, никого не инвалидируя — ведь оно и так знало, что копия есть только у него!)*

Сценарий 3: Появление магии Owned (Главная фишка MOESI)

Продолжаем сценарий 2. Сейчас Ядро 1 находится в состоянии **Modified** (у него свежий черновик), а Ядро 2 — в **Invalid** (у него ничего нет). Память — старая.

- Ядро 2 вдруг снова хочет ПРОЧИТАТЬ эти данные:
 - Ядро 2 делает запрос в систему.
 - Ядро 1 «слышит» этот запрос и понимает: *«Опа, у меня в кэше лежат свежие данные (Modified), а в оперативной памяти — старое барахло. Если Ядро 2 пойдет в оперативку, оно прочитает чушь. Надо его спасти»*.
 - Ядро 1 перехватывает запрос и **само лично отправляет** свежие данные из своего кэша в кэш Ядра 2. В медленную оперативную память ничего не пишется!
 - Переходы:
 - У Ядра 1: **Modified** → **Owned** (O). Оно стало «Владельцем» и взяло на себя обязательство когда-нибудь потом обновить оперативку.
 - У Ядра 2: **Invalid** → **Shared** (S). Оно получило свежие данные и просто читает их.

И описание состояний на примере (не суупер классное объяснение, но мне помогло):

1. 🟢 State E (Exclusive) — «Я читаю это один»

- **Суть:** Вы скачали документ с сервера. Вы точно знаете, что больше никто его себе не скачивал.
- **В чате:** Коля скачал файл. В памяти и у Коли всё одинаковое. Маша файл не открывала.
- **Плюс:** Коля может редактировать этот текст прямо сейчас, никого не спрашивая.

2. 🔴 State M (Modified) — «Я изменил, сервер не знает»

- **Суть:** Вы изменили текст у себя на компьютере, но кнопку «Сохранить на сервер» еще не нажали.
- **В чате:** Коля переписал абзац. У него — самая свежая версия. На сервере — старая. У Маши этого файла вообще нет.
- **Плюс:** Самая быстрая работа. Коля продолжает писать.

3. 🔵 State S (Shared) — «Читаем вместе»

- **Суть:** Документ скачали сразу несколько человек. Все просто читают.
- **В чате:** И у Коли, и у Маши открыт один и тот же текст. Он совпадает с сервером.
- **Минус:** Никто не имеет права менять текст без предупреждения. Если Коля захочет нажать «Редактировать», он обязан крикнуть в чат: «*Маша, удали свой черновик, я переписываю!*».

4. 🟡 State O (Owned) — «Я главный, но делюсь» (Фишка MOESI)

- **Суть:** Вы изменили документ, не сохранили на сервер, но переслали копию коллеге. Теперь вы за нее отвечаете.
- **В чате:** Коля изменил текст (он был в *Modified*). Тут прибегает Маша: «*Дай почитать!*». Коля не тратит время на отправку файла на медленный Сервер. Он скидывает Маше измененный текст напрямую из своего кэша.
- **Итог:** У Коли статус меняется на **Owned** (он теперь владелец актуального текста). У Маши статус становится **Shared** (она просто читает). На Сервере всё еще старая версия, но Коля «помнит», что перед закрытием программы именно он должен обновить Сервер.

5. ⬛ State I (Invalid) — «Текст удален / устарел»

- **Суть:** Ваша копия больше не имеет силы. Выкиньте её.
- **В чате:** Маша читала файл (была в *Shared*). Коля решил этот абзац изменить. Он кричит: «*Я меняю!*». У Маши на экране этот абзац мгновенно перечеркивается и превращается в **Invalid**. Если Маше снова понадобится этот абзац, ей придется просить его заново (у Коли или у Сервера).

контекст для вопросов 180-182

Из прошлых вопросов мы поняли, что между кешами нужно наладить обмен информацией, это в том числе нужно для поддержания когерентности и реализации MOESI. Существует три основных подхода к реализации этого обмена:

- Справочник
 - Отслеживание
 - Перехват
-

180. Как происходит обмен информацией о кеш-линиях через справочник? Как этот подход масштабируется?

Один из способов реализации обмена информацией это через справочник. Т.е. появляется некоторый центральный блок, который подключен к кешу каждого ядра, и внутри себя хранит таблицу-справочник. Для каждой кеш-линии в системе он хранит:

- **Modified bit.** Изменена ли эта линия по сравнению с тем, что есть в основной памяти (RAM)
- **Presence bits.** Для каждого ядра здесь есть отдельный бит, который показывает, есть ли копия этой кеш-линии в кеше конкретного ядра.

(вот эти биты, это то что было показано на картинке в лекции, по факту справочник может реализовывать +- любой протокол, например MOESI так что заостряться на именно такой организации смысла нет. Вопрос в целом про концепцию централизованного справочника)

Одно из главных достоинств - относительно нелесообразная масштабируемость. Недостаток в том, что появляется некоторый централизованный объект, который легко может стать бутылочным горлышком по производительности

В любом случае таблица справочника позволяет узнавать, изменена ли определенная линия в кешах относительно RAM и если изменена, то в каких кешах можно найти ее актуальную копию.

Как это работает на пальцах:

- **Ядро 1 хочет ПРОЧИТАТЬ данные:**

Оно тихонько отправляет запрос Справочнику: «Можно мне строку №5?».

Справочник смотрит в свою таблицу: «Так, строка №5 сейчас никем не изменена.

Но она уже есть у Ядра 3 и Ядра 4». Справочник отмечает у себя: «Теперь она есть и у Ядра 1» (ставит галочку в Presence bits) и выдает данные. Никакого общего шума.

- **Ядро 1 хочет ЗАПИСАТЬ (изменить) данные:**

Оно пишет Справочнику: «Я хочу изменить строку №5». Справочник смотрит в

таблицу и видит, у кого именно сейчас открыта эта строка (например, у Ядра 3 и Ядра 4). Справочник **целенаправленно** отправляет команду «Удалить!» *только*

Ядру 3 и Ядру 4. Остальные 1020 ядер процессора вообще об этом не узнают и продолжают спокойно работать.

181. Как происходит обмен информацией о кеш-линиях через отслеживание? Как этот подход масштабируется?

Идея обмена информацией через отслеживание в следующем - давайте сделаем так, что общение кешей с памятью будет проходить по одной общей шине. Получится так, что каждый кеш сможет слушать, что остальные делают с памятью и в соответствии с этим обновлять там свои статусы, пометать данные и так далее.

Плохо, что нам приходится тянуть одну общую шину через все кеши, затем нам приходится как-то диспетчизировать эту шину, выстраивать кеши в очередь. Либо, если эта шина многоканальная, то это все становится очень сильно дороже и сложнее, каждому кешу надо следить за всеми каналами и все такое.

Проблема масштабирования тут стоит очень остро, потому что кол-во логики возрастает очень сильно от количества ядер

ВАЖНО, что здесь мы говорим о том, что прокладываем общую **шину адреса**, и кеши опираются только на это ("о, другой кеш записал в память адрес, который есть и у меня. Инвалидирую свой адрес")

182. Как происходит обмен информацией о кеш-линиях через перехват? Как этот подход масштабируется?

Если в случае подхода через отслеживание мы прокладывали только шины адреса, то здесь мы прокладываем еще и шину данных. Теперь кеши могут видеть, кто что отправляет в память и не

только обновлять состояния своих ячеек, но и сразу обновлять их значения! Это сильно оптимизирует всю систему, ну и помогает в полной мере реализовать MOESI (потому что честно говоря в логике "через отслеживание" у кешей нет вариантов обмениваться между собой данными, и как тогда работает логика Owned-Shared непонятно. в случае перехвата все супер)

183. **Какие существуют типы кеш-промахов (3C + 1: cold, capacity, conflict, coherence)? Какие способы смягчения характерны для каждого?**

Вот здесь реальные типы кеш-промахов (в отличие от вопроса 166, где их вроде как нельзя назвать типами кеш-промахов). Все хорошо описано на картинке:

Кеш-промахи. Классификация (3C + 1)

1. **Cold / Compulsory** (холодный)

 - Первое обращение к блоку — его ещё нет в кеше.
 - Неизбежен; смягчается **предвыборкой** (prefetch).
2. **Capacity** (по ёмкости)

 - Рабочий набор больше кеша — блок был вытеснен ради других.
 - Смягчается увеличением размера кеша.
3. **Conflict** (по конфликту)

 - Несколько блоков отображаются в одну линию — вытеснили, хотя место в кеше было.
 - Актуален для direct-mapping и set-associative.
 - Смягчается ростом ассоциативности.
4. **Coherence** (когерентный)

 - Блок инвалидирован из-за модификации в другом кеше / DMA.
 - Возникает только в **многоядерных/многомастерных** системах.

184. **Что такое CAP-теорема? Какова область её применения и как она применяется к когерентности кешей?**

CAP-теорема - то, что мы рассматривали для разных уровней кешей и их синхронизации, только обобщенное в целом на любую распределенную систему. Например, бд которая хранится распределенно на несколько серверов. Так вот, сама теорема говорит о том, что у каждой системы есть три параметра:

1. **Согласованность (Consistency)**
— каждое чтение получает последнюю запись или ошибку.
2. **Доступность (Availability)** — каждый запрос получает ответ (без ошибок), но и без гарантии, что он содержит последнюю запись.
3. **Устойчивость к разбиению (Partition Tolerance/Brainsplit)** — система продолжает работать, несмотря на произвольное количество потерь в сети между узлами.

И что все три параметра одновременно обеспечить невозможно, можно только два

Ну и в области когерентности кешей у нас нет требования к "Устойчивости к разбиению" так что теорема применяется в формате "возможно обеспечить Согласованность и Доступность"

185. Что означают термины "Consistency", "Availability" и "Partition tolerance" в CAP-теореме? Опишите их.

CAP-теорема - то, что мы рассматривали для разных уровней кешей и их синхронизации, только обобщенное в целом на любую распределенную систему. Например, бд которая хранится распределенно на несколько серверов. Так вот, сама теорема говорит о том, что у каждой системы есть три параметра:

1. **Согласованность (Consistency)**
— каждое чтение получает последнюю запись или ошибку.
2. **Доступность (Availability)** — каждый запрос получает ответ (без ошибок), но и без гарантии, что он содержит последнюю запись.
3. **Устойчивость к разбиению (Partition Tolerance/Brainsplit)** — система продолжает работать, несмотря на произвольное количество потерь в сети между узлами.

И что все три параметра одновременно обеспечить невозможно, можно только два

186. Как кеш влияет на производительность памяти? Какие существуют способы оптимизации доступа, включая AoS и SoA?

- В теории кеш-память по производительности будет обгонять обычную память, соответственно кеш позитивно влияет на производительность памяти, по крайней мере он для этого был придуман

В лекции 14 на 1:15:14 приведен хороший пример об оптимизации перемножения матриц с учетом особенностей работы с кешем.

Глобально (помимо вот примера с матрицами) есть два больших способа оптимизации доступа:

Array of Structures, Structures of Arrays

```
struct point3D {
    float x;
    float y;
    float z;
};

struct point3D points[N];

float get_point_x(int i) {
    return points[i].x;
}
```

```
struct pointlist3D {
    float x[N];
    float y[N];
    float z[N];
};

struct pointlist3D points;

float get_point_x(int i) {
    return points.x[i];
}
```

Таким образом у нас будет в первом случае оптимизированы операции на чтение одной точки (так работает в реляционных базах данных, так как классический usecase - получение отдельных записей).

А во втором - оптимизировано под запрос колонок, например для подсчета средней координаты x (так работают колоночные базы данных)

187. Как кеш работает с виртуальными и физическими адресами?

Есть момент с тем, что в реальных системах на процессорах запускается несколько задач-процессов. Чаще всего у каждого процесса своя виртуальная память (разобрано ранее в вопросах), и соответственно каждый процесс запрашивает данные в виртуальной адресации, которая затем уже транслируется в физическую (что занимает время).

Так вот вопрос - кешу следует опираться на виртуальный адрес (это будет быстрее, но между процессами будет коллизия адресов, а значит при переключении между процессами надо будет его сбрасывать, ведь у каждого своя память) или все-таки на физический (но нам надо будет тратить время на трансляцию виртуального адреса в физический, а кеш хочется быстрый)?

Решается компромиссом - очень быстрый кеш делают на виртуальных адресах, а уже другие уровни, которым не так критична скорость опираются на настоящий физический адрес

Кеш-память с виртуальными адресами

Преимущества (ВА)

- **Быстрота доступа.** Доступ к памяти минуя трансляцию адреса.
- **Простота реализации.** Простота кеш памяти. Не требуется трансляция адресов перед проверкой кеш попадания.

Недостатки (ВА)

- **Проблемы совместимости процессов.** Требуется различать контексты из-за коллизий виртуальных адресов.
- **Неэффективность при смене контекста.** Смена контекста может потребовать очистку кеша для предотвращения конфликтов данных.

Кеш-память с физическими адресами

Преимущества (ФА)

- **Универсальность.** Физические адреса уникальны в рамках системы.
- **Консистентность данных:** Физические адреса упрощают поддержание консистентности кешей и памяти.

Недостатки (ФА)

- **Задержки из-за трансляции адресов:** Доступ к данным в кеше по физическому адресу требует предварительной трансляции виртуального адреса.
- **Сложность реализации:** Необходимость поддержки трансляции адресов увеличивает сложность управления кешем.

Возможны смешанные варианты:

- L1 — виртуальные адреса для минимизации задержек,
- L2, L3 — физические для универсальности и консистентности.

188. **Что такое HyperThreading? Как два логических ядра делят физический кеш и как это влияет на производительность?**

HyperThreading - подход который не дает настоящим нескольким процессам, но за счет двух наборов основных регистров позволяет одному физическому ядру обрабатывать два потока вычислений одновременно, переключаясь между ними за счет отдельных комплектов регистров. Переключение например может происходить в момент cache-miss на одном ядре и пока ждем ответа переключаемся на другое ядро.

Кеш такие логические ядра делят непосредственно одни и те же кеши, поэтому если оба потока словят cache-miss на производительности это скажется плохо - физическое ядро будет простаивать. Но в среднем HyperThreading по сравнению с одним ядром позволяет выполнять несколько больше инструкций и "имитировать" два ядра гораздо дешевле, чем действительно построить два ядра

189. В чём суть уязвимостей Meltdown и Spectre? Как кеш и спекулятивное исполнение создают канал утечки данных?

Обе уязвимости основываются на том, что мы начинаем выполнять инструкции, которые не должны быть выполнены. Например потому что у нас длинный pipeline, или потому что branch-predictor сказал что надо идти туда.

Meltdown - позволяет прочитать данные, которые запрещены для чтения данному процессу. В общем случае работает так:

C

```
// 1. Попытка прочитать секретный адрес из пространства ядра ОС
char secret = *kernel_address;

// 2. Использование секрета как индекса для обращения к разрешенному массиву
char data = user_array[secret * 4096];
```

За счет спекулятивного исполнения получалось так, что еще до проверки доступа, элемент user_array по индексу secret*4096 оказывался в кеше. Затем можно легко проверить какой из элементов user_array (это наш массив) находится в кеше. И тот, кто в нем находится и есть секрет, который мы хотим узнать

Spectre - основан на злоупотреблении branch predictor, заставляя его ошибиться в предсказании перехода и выполнить то, что выполнять было нельзя. Например:

C

```
if (index < array_size) {
    char element = array[index];
    char data = user_array[element * 4096];
}
```

Здесь предиктор ошибается и начинает читать по адресу, который выходит за пределы массива. Затем, конечно, все откатывается,

когда процессор понимает, что индекс выходит за пределы. Вот только это значение уже лежит в кеше.

<15 лекция>

190. Что представляет собой концепция уровневой организации компьютерных систем и каковы её преимущества?

Уровневая организация - естественный способ организации вычислительных систем, где система разделена на группы компонентов (называемых уровнями), которые имеют некоторую иерархию и относительную независимость друг от друга.

Каждый уровень решает конкретную задачу, предоставляя конкретный интерфейс для других уровней, скрывая внутренние детали реализации.

Такие уровни абстракции позволяют разделять команды разработчиков, снижая общую сложность разработки, делают системы **модульными** и **вариативными**, позволяя заменять реализацию некоторых уровней целиком

191. Какие примеры уровневой организации вычислительных систем вы знаете? Чем они отличаются друг от друга?

Lava Flow - антипаттерн проектирования (как делать не надо), когда проект можно представить как извергающийся вулкан. Т.е. во время разработки проекта вулкан периодически извергается (приходит новый разработчик, появляется новое требование, меняется подход), далее из него изливается лава (пишется новый код там и так далее) вот только вся эта лава (и вся работа над проектом) накладывается поверх предыдущих, уже окаменевших слоев. Никто не подвергает сомнению старый код, никто его не рефакторит, просто фигачат сверху больше и больше.

OSI (Open Systems Interconnection) Model -

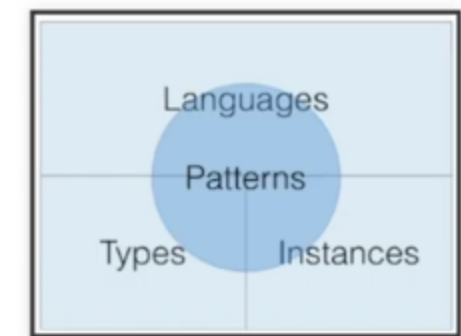
1. Application Layer
2. Presentation Layer
3. Session Layer
4. Transport Layer
5. Network Layer
6. Data Link Layer
7. Physical Layer

здесь интересно, что между уровнями нет их "горизонтальной" связанности. Т.е. когда у нас идет пересылка HTTP пакета, он всегда идет на всех уровнях сразу. У нас пакет формируется, передается на следующий уровень, там может быть порезан, передан разными путями и технологиями. И при этом для каждого самого низкоуровневого куска этого пакета будет прослеживаться все наследование вплоть до Application level.

Уровни организации для компьютеров в целом



Уровни организации вычислительных процессов (не останавливались подробно на лекции)

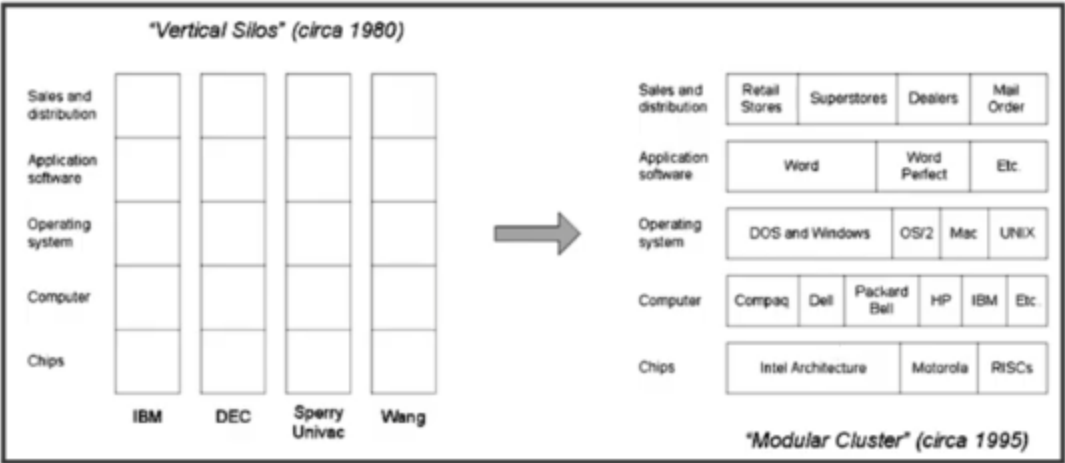


Уровни организации прикладных проектов (не останавливались подробно на лекции)

- 1. Layered project organization.
- 2. Frameworks, библиотеки, API.
- 3. Языки программирования.
- 4. Операционные системы.
- 5. Системы команд (ISA), ПЛИС.
- 6. Виртуализированные ресурсы.

И в целом это все про разработку компьютерных систем в виде уровней с четкой иерархией и строгим разделением

192. Что такое явление разделения на уровни (disaggregation) и каково его современное положение? Что было до него?



Изначально каждая коммерческая компания стремилась покрыть все уровни:

Application Software	программы
Операционные системы	драйверы устройств
Архитектура	регистры инструкций
Микро - архитектура	контроллеры каналы передачи данных
Логика	сумматоры память
Цифровые схемы	AND вентили NOT вентили
Аналоговые схемы	усилители фильтры
Устройства	транзисторы диоды
Физика	Электроны

Они разрабатывали свои процессоры, свои чипы, собирали свои устройства, писали под них свои ОС, под эти ОС писали свое ПО.

Затем произошла реорганизация всего этого (*disaggregation*) и сейчас уже очень мало компаний, которые покрывают настолько много уровней. По большей части сейчас у нас есть компании, которые производят конкретно процессоры и специализируются на этом. Есть компании, которые пилят прикладной софт. Есть компании, которые пишут и распространяют свои ОС.

Это огромный шаг навстречу прогрессу потому, что позволяет каждому заниматься своим делом и делать его хорошо. При этом на процессорах разных производителей мы можем запустить одну и ту же ОС, на разных ОС можем запустить одно и то же ПО.

Все это позитивные последствия разделения на уровни.

Единственное конечно в идеальном мире если компания покрывает все уровни сразу, то она может писать сверх-эффективное ПО, поскольку есть возможность заточить все уровни конкретно под нашу аппаратуру, не оглядываясь на то (и не адаптируя под то), что еще есть в индустрии

193. Почему необходимо нарушение уровневой иерархии в современных системах? Приведите примеры.

Разделение на уровни это классно все супер ровно до того момента как только у нас появляются строгие требования реального времени, или мы становимся очень чувствительны к сокращению времени выполнения или энергопотребление стремимся оптимизировать или еще что. В таких задачах мы не можем игнорировать внутренние уровни

Чтобы обеспечить выполнение этих наших задач мы вынуждены нарушать уровневую иерархию разными способами. Начиная от того, что на верхнем уровне языка мы задумываемся о том как работают нижние уровни, например кэши. Заканчивая тем, что мы пишем большие ассемблерные вставки, реорганизовываем структуры данных, вмешиваемся в нижние уровни и все такое.

194. Какие проблемы возникают при обеспечении реального времени в современных процессорах?

Со всеми "наворотами" и оптимизациями процессоров которые мы рассматриваем в курсе, при обеспечении задач реального времени в современных процессорах возникает куча проблем: спекулятивное исполнение, кеш-память, виртуальная память все это призвано улучшить производительность но на практике порождает недетерминированность времени выполнения, когда мы не можем понять, сколько именно будет выполняться конкретная инструкция, сколько именно будет реакция системы на воздействие. Поэтому нам приходится как-то обходить эти все штуки на программном уровне

195. Что такое низкоуровневый параллелизм? Приведите примеры.

Низкоуровневый параллелизм - параллелизм, который достигается внутри процессора на уровне его микроархитектуры. Это тот параллелизм, который не виден для программиста и в целом не влияет на прикладное программирование.

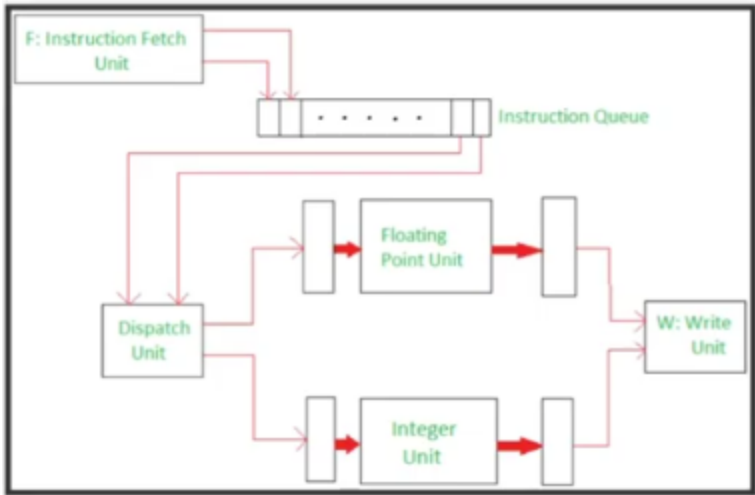
Примеры организации такого параллелизма:

- **Конвейеризация** то, что рассматривалось подробно ранее. Когда инструкция разбивается на несколько этапов, а разные этапы могут выполняться параллельно
- **Множественные функциональные узлы** когда у нас есть независимые функциональные блоки для разных операций и мы можем выполнять их одновременно
 - Суперскалярный процессор
 - Very Long Instruction Word

196. Что такое суперскалярные процессоры и каковы принципы их работы, достоинства и недостатки?

Идея суперскалярного процессора родилась из конвейерной организации, когда стало понятно, что этап исполнения инструкций достаточно тяжелый, под него заточено много логики, он может быть долгим и в целом может тормозить весь конвейер. Люди подумали, а давайте у нас процессор будет парсить инструкции так быстро как может, затем будет складывать их в очередь, а отдельный юнит будет диспетчеризировать параллельное исполнение этих инструкций за счет того, что у нас есть несколько независимых функциональных узлов, умеющих выполнять инструкции. А по

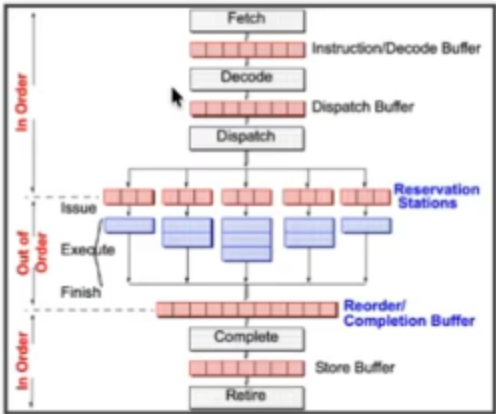
готовности будет собирать результаты и писать их в память



197. Как пошагово выполняются инструкции в суперскалярном процессоре?

Суперскалярный процессор. Структура

- 1. Инструкции читаются в очередь команд по порядку.
- 2. Декодируются и перемещаются на станции резервирования.
- 3. Станции резервирования выполняют переупорядочивание:
 - по мере доступности данных;
 - по мере доступности вычислительных ресурсов.



1. Вход (По порядку — In-Order)

- **Fetch & Decode:** Процессор берет инструкции из памяти по порядку и разбивает их на мелкие микрокоманды.
- **Dispatch:** Команды распределяются по станциям резервирования.

2. Исполнение (Вразнобой — Out-of-Order)

- **Issue & Execute:** Команда ждет на станции, пока для нее готовы данные. Как только данные приехали, она мгновенно выполняется, обгоняя другие «зависшие» команды.

3. Выход (Снова по порядку — In-Order)

- **Retire:** Результаты временно складываются в буфер переупорядочивания. Процессор записывает их в реальную память строго по порядку (1, 2, 3...), маскируя внутренний хаос.

Суперскалярный процессор. Достоинства

1. Рост производительности. Сглаживание длительности выполнения инструкций.
2. Повышение уровня загрузки ресурсов.
3. Совместимость с существующим машинным кодом.
4. Компилятор может устранить значительное количество конфликтов за счёт сортировки инструкций.

Суперскалярный процессор. Недостатки

1. Конфликты по данным оказывают значительное влияние на производительность и сложность процессора.
2. Высокое энергопотребление.
3. Проблемы детерминированности работы многоядерных процессоров.

Если говорить подробно:

● Суперскалярный процессор. Достоинства

1. Рост производительности и сглаживание длительности выполнения инструкций

За счет параллельных исполнительных блоков процессор может выполнять несколько команд за один такт. Если одна инструкция «зависла» из-за промаха мимо кеша (*cache-miss*), конвейер не останавливается — станции резервирования пускают вперед другие команды, сглаживая задержки.

2. Повышение уровня загрузки ресурсов

Исполнительные блоки (математические ALU, блоки плавающей точки FPU) не простаивают. Железо постоянно загружено работой из разных участков кода, которые удалось запустить одновременно.

3. Совместимость с существующим машинным кодом

Это важнейший плюс. Программисту или компилятору не нужно знать, сколько внутри чипа ядер или ALU. Вы пишете обычный последовательный код (например, x86). Процессор сам на лету аппаратно распараллелит его, переупорядочит в ROB и выполнит быстрее. Старые программы без перекомпиляции работают быстрее на новом чипе.

4. Помощь компилятора (сортировка инструкций)

Хотя процессор сам умеет переупорядочивать инструкции на станциях резервирования, умный компилятор может заранее расставить команды в коде так, чтобы минимизировать зависимости по данным. Это сильно упрощает работу планировщика внутри CPU.

● Суперскалярный процессор. Недостатки

1. Влияние конфликтов по данным на сложность процессора

Если Инструкция №2 жестко зависит от результата Инструкции №1, их нельзя выполнить параллельно или поменять местами. Чтобы решать такие конфликты на лету, инженерам приходится строить сложнейшие аппаратные структуры: схемы переименования регистров, буферы ROB, станции резервирования и общие шины данных (CDB). Это раздувает размер кристалла процессора.

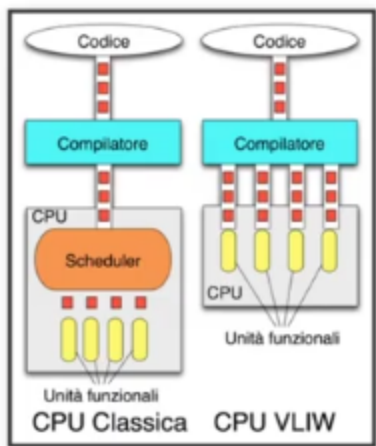
2. Высокое энергопотребление

Вся эта логика «умного» внеочередного анализа (проверка зависимостей, предсказание переходов) жрет колоссальное количество энергии и выделяет тепло. Процессор тратит электричество не только на сами вычисления, но и на обслуживание своего сложного конвейера.

3. Проблемы детерминированности работы многоядерных процессоров

Прямая связь с темой Real-Time. Поскольку каждое ядро независимо и хаотично переупорядочивает инструкции, предсказывает переходы и ловит *cache-miss*, абсолютно невозможно предугадать точное время выполнения задачи. Когда несколько таких «хаотичных» ядер начинают одновременно делить общую память или кеш L3, возникает полный недетерминизм. Для систем жесткого реального времени такие процессоры не подходят.

VLIW-процессор - поскольку у нас теперь несколько исполнителей, несколько функциональных блоков и мы заранее знаем, какие операции они могут исполнять, то зачем нам вообще во время работы диспетчеризировать маленькие инструкции на эти исполнители если можно просто сделать большие инструкции, которые объединяют сразу несколько операций и идеально заполняют все наши вычислительные блоки:

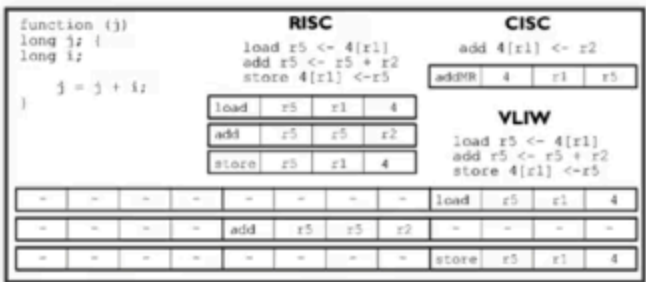


VLIW. Достоинства

- 1. Упрощение процессора, снижение энергопотребления.
- 2. Упрощение декодера. Рост частоты.
- 3. Компилятор имеет больше информации о коде, он лучше знает, что параллельно!

VLIW. Недостатки

- 1. Сложность компилятора.
- 2. Высокая нагрузка на каналы данных и регистровые файлы.
- 3. Конфликты конвейера приводят к простоям всех узлов.
- 4. Проблемы условных переходов.
- 5. Низкая плотность кода.
- 6. Ширина команды — ограничение микроархитектуры.



● VLIW. Достоинства

1. Упрощение процессора и снижение энергопотребления

Поскольку в процессоре больше нет сложнейшей логики внеочередного исполнения, предсказателей и буферов переупорядочивания, кристалл чипа становится меньше, проще и потребляет в разы меньше энергии.

2. Упрощение декодера и рост частоты

Декодеру не нужно на лету анализировать зависимости. Он берет готовое длинное слово и сразу раскидывает его микрокоманды по исполнительным блокам. Меньше стадий конвейера — выше тактовая частота.

3. Компилятор знает больше

Компилятор видит всю программу целиком на этапе сборки. У него много времени, чтобы проанализировать код, найти независимые инструкции и идеально распараллелить их.

● VLIW. Недостатки

1. Сложность компилятора

Написать компилятор под VLIW — это адская задача. На него перекладывается вся ответственность за производительность.

2. Высокая нагрузка на регистры

Так как параллельно выполняется много команд, им всем одновременно нужны порты доступа к регистровому файлу, что создает бутылочное горлышко.

3. Конфликты конвейера приводят к простою ВСЕХ узлов

Это критический минус. Если в суперскалярном OoO-процессоре одна команда ловит cache-miss, другие ее обходят. Во VLIW, если одна инструкция внутри «длинного слова» зависла (ждет память), **замирает весь процессор целиком**. Все параллельные блоки простаивают.

4. Низкая плотность кода и «пустые» операции (NOP)

Посмотри на таблицу внизу слайда. В первом такте выполняется только `load`, а остальные исполнительные блоки пусты (прочерки -). Процессор вынужден кодировать эти пустоты как холостые команды `NOP` (No Operation). В итоге бинарный файл программы раздувается в размерах, забивая кеш команд мусором.

5. Ограничение микроархитектуры (Ширина команды)

Код жестко привязан к железу. Если компилятор собрал программу под процессор с 4 исполнительными блоками (длина слова рассчитана на 4 команды), этот код физически **не запустится** на новом процессоре VLIW с 8 блоками. Программы нужно полностью перекомпилировать под каждое новое поколение процессоров.

+к недостаткам - из-за очень широких инструкций нам надо делать все широкое - широкие каналы доступа к памяти, широкие регистры и все такое

199. Как соотносятся суперскалярные и VLIW процессоры применительно к разным классам задач?

- **Суперскаляр** — general-purpose CPU (x86, ARM): неоднородные нагрузки, важна совместимость с уже скомпилированным кодом.
- **VLIW** — DSP, GPU shader cores, специализированные ускорители (TI C6000, Эльбрус, Intel Itanium): регулярные нагрузки, компилятор знает структуру кода.

Итого:

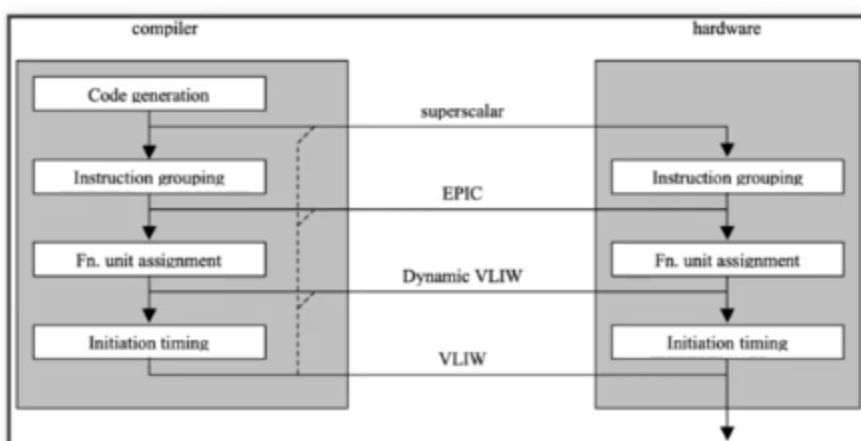
Суперскаляр лучше для каких-то универсальных штук, и особенно для задач где много переходов. Потому что суперскалярный процессор гораздо лучше с этим справляется и может подмешивать разные инструкции разные потоки и все такое. +он обратно совместим, ассемблерный код может запускаться на любом процессоре который поддерживает ISA на котором написан этот код

VLIW лучше всего работает в узкоспециализированных задачах, например где надо делать кучу однотипных вычислений (в числодробилках). Эти процессоры более энергоэффективны, более предсказуемы и плотнее загружают вычислительные возможности процессора

200. Какие существуют промежуточные варианты между суперскалярным и VLIW процессором? Какие проблемы они решают?

Если смотреть чуть более общим взглядом, то суперскаляр и VLIW это всего лишь крайности, где в одном случае мы поток инструкций в рантайме группируем и пытаемся пережевать, а в другом у нас уже на уровне инструкций все объединено и в рантайме нам надо только выполнять инструкции вообще не задумываясь.

На самом деле можно разбить код на 4 этапа обработки. Любой код должен их пройти, вопрос только в том, что что-то можно делать на этапе компиляции, а что-то делать в самом процессоре во время работы. Вот именно по тому, где какие этапы выполняются и получаются промежуточные архитектуры:



⚙️ Четыре шага обработки кода

Любой параллельный запуск инструкций требует выполнения четырех шагов:

1. **Code generation:** Генерация базового машинного кода.
2. **Instruction grouping:** Поиск независимых команд и объединение их в группы для параллельного запуска.
3. **Fn. unit assignment:** Привязка конкретной команды к конкретному исполнителю (например, эту — в ALU №1, эту — в FPU №2).
4. **Initiation timing:** Определение точного такта (времени), когда эту команду нужно физически выстрелить в конвейер.

Из этого получаем еще две промежуточные архитектуры:

1. Superscalar (Обычный суперскаляр — x86, ARM)

- **Как работает:** Компилятор делает только первый шаг (`Code generation`). Он просто выдает плоский последовательный поток команд.
- **Железо делает ВСЁ остальное:** Сами кремниевые схемы на лету группируют инструкции, распределяют их по исполнительным блокам (порты ALU/FPU) и решают, в какой такт их запустить (OutOfOrder).
- *Итог:* Железо сложное и горячее, софт простой.

2. EPIC (Explicitly Parallel Instruction Computing — Intel Itanium)

- **Как работает:** Компилятор делает первые два шага. Он генерирует код и **явно группирует** независимые команды в блоки (bundles), подсказывая процессору: *«эти три штуки можно запустить одновременно»*.
- **Железо делает остальное:** Процессор берет эти группы и сам решает, на какие конкретно внутренние ALU их раскидать и в какой микротакт запустить.
- *Итог:* Попытка найти компромисс между VLIW и Суперскаляром.

3. Dynamic VLIW

- **Как работает:** Компилятор берет на себя уже три шага. Он не только группирует команды, но и жестко прописывает, в какой функциональный блок (`Fn. unit assignment`) должна пойти каждая инструкция.
- **Железо делает минимум:** Железо отвечает только за тайминг (`Initiation timing`) — проверяет, не случилось ли задержки памяти (cache-miss), и вовремя дает отмашку на старт.

4. VLIW (Чистый VLIW — Эльбрус, DSP)

- **Как работает:** Компилятор делает **абсолютно всё** (все 4 шага). В кодовой строке (длинном слове) уже жестко зашито: в каком такте, на каком конкретно ALU и какие три команды будут выполнены одновременно.
- **Железо делает НИЧЕГО:** Процессор превращается в «глупый исполнитель». Он просто берет готовые строчки-команды и без всякого анализа раскидывает их по кремниевым блокам. Если компилятор ошибся с таймингом — процессор зависнет.

Важно отметить, что из-за сложности компиляторов и не очень большого роста в производительности EPIC процессоры не получили большого распространения и развития

Еще момент - отличие VLIW и Dynamic VLIW в том, что во VLIW в коде уже четко задано в какой момент он должен какие инструкции на каких АЛУ запустить, а в Dynamic VLIW так - если на каком-то АЛУ инструкции исполняются быстрее чем других, то он не простаивает, а идет дальше по *своему* потоку инструкций, которые назначены на этот АЛУ.

Пример: Если АЛУ2 закончило свою задачу быстрее, а АЛУ1 все еще занято тяжелой операцией (например, ждет данные из памяти из-за *cache-miss*), то **АЛУ2 не стоит на месте** - железо разрешает АЛУ2 взять *следующую* назначенную ему команду из конвейера и запустить её.

201. Что такое барьеры памяти и в каких ситуациях они применяются?

Барьер памяти - специальная инструкция, которая предписывает компилятору (при генерации инструкций) и процессору (при исполнении инструкций) не смешивать обращения к памяти. Чтобы обращения после барьера никогда не выполнялись раньше обращений до барьера. Применяется когда нам важна последовательность обращений к памяти (например, сначала записали новые данные, и *только потом* установили флаг готовности новых данных)

Актуально для процессоров, которые могут перемешивать выполнение инструкций (например, для суперскалярных процессоров) и для компиляторов, которые тоже иногда для оптимизации могут переставлять инструкции (чтобы избежать пузырьков конвейера, к примеру)

202. Как суперскалярное исполнение и store buffers влияют на порядок обращений к памяти в многоядерных системах? Что такое барьер памяти (fence) и зачем он нужен?

Суперскалярное исполнение подразумевает несколько исполнительных блоков, умеющих выполнять инструкции. Поскольку поток один, а исполнителей несколько, этот поток распределяется на них, а как именно - только самому процессору известно и

выполняются соответственно в хаотичном порядке. Поэтому инструкции чтения могут быть выполнены в произвольном порядке.

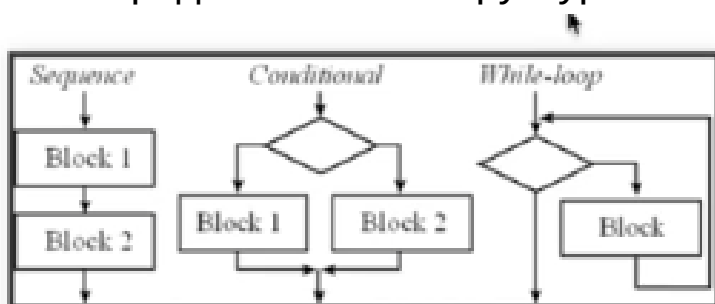
Store buffers (буфер между АЛУ's и памятью/кешем) выравнивают эту историю, записывая результаты вычислений в память ровно в том порядке, в котором они шли в исходном потоке. Так что инструкции записи выполняются в том порядке в котором они шли изначально.

И здесь могут возникать проблемы, потому что порядок этих всех обращений/чтений/записей относительно друг друга может быть произвольным вообще

Барьер памяти - специальная инструкция, которая предписывает компилятору (при генерации инструкций) и процессору (при исполнении инструкций) не смешивать обращения к памяти. Чтобы обращения после барьера никогда не выполнялись раньше обращений до барьера. Применяется когда нам важна последовательность обращений к памяти (например, сначала записали новые данные, и *только потом* установили флаг готовности новых данных)

203. Какие ключевые элементы составляют структурное программирование? В чём отличие от кода на ассемблере?

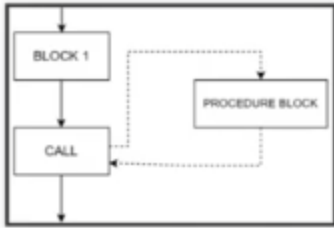
Язык высокого уровня - это когда мы переходим от инструкций и джампов, ну т.е. вообще от абстракций процессора к структурным блокам и их последовательностям. В особенности так реализован язык Си, в основе которого заложена идея, что любую программу можно представить как структурные блоки.



Из три типа:

- **Последовательный** - просто некоторая совокупность операций
- **Условный** - в зависимости от условия выполняем или одно или другое
- **Циклический** - выполняем некоторый блок пока есть некоторое условие

Ну и можно называть отдельные блоки и вызывать их:



204. Как реализуются процедуры на уровне ассемблера? Что такое call/return и зачем нужен стек вызовов?

При написании кода нам так или иначе нужны процедуры. Зачем:

- с целью переиспользования машинного кода
- для оптимизации кеша (инструкции не дублируются по разным адресам в разных местах программы, а лежат все компактно в одном месте)

Есть как минимум три подхода к реализации процедур:

- `inline` - мы просто вместо вызова процедуры вставляем все инструкции этой процедуры и все. Да у нас дублируется код, но формально "вызов" процедуры осуществлен
- `goto` - путем инструкций перехода осуществляем вызов и возврат
- `call/return` - аппаратная поддержка процедур с сохранением адреса возврата в стек вызовов (он нужен для реализации вложенности и реентерабельности)

205. Что такое реентерабельность и почему она важна?

Приведите примеры.

Реентерабельность - это свойство процедуры, позволяющее ей быть вызванной повторно до того как завершился предыдущий ее вызов. Т.е. если мы можем безопасно прервать ее выполнение ее же вызовом.

Рекурсия например активно использует это свойство, когда функция вызывает сама себя. Либо прерывание, которое может прерывать собственный обработчик.

206. Как реализуется механизм рекурсии?

Есть два варианта, как можно реализовать рекурсию:

1. Реализация через автоматическую память (Аппаратный стек)

Это классический способ в современных системах (`push` , `pop`), показанный в нижнем блоке кода и на схеме справа:

- **Создание фрейма (Stack Frame):** При каждом рекурсивном вызове инструкции `call` процессор выделяет на вершине аппаратного стека памяти новый изолированный фрейм.
- **Изоляция данных:** В этот фрейм автоматически сохраняются:
 - Новое значение аргумента (например, `n - 1` для факториала).
 - Своя собственная, уникальная локальная переменная (`int tmp`), которая резервирует память (`reserve mem`) строго под этот конкретный шаг.
 - **Адрес возврата** — точка в коде, куда нужно вернуться, когда этот шаг завершится.
- **Схлопывание стека:** Когда функция доходит до базового случая (`if (n == 0) return 1;`), выполнение начинает двигаться назад. Команда `return (RET)` забирает адрес возврата со стека, уничтожает текущий фрейм (`pop`) и возвращает управление на предыдущий уровень рекурсии (как показано стрелками на схеме справа).

2. Реализация через статическое выделение памяти для каждого входа

Это альтернативный, более редкий и жесткий подход, показанный в верхнем блоке кода:

- **Суть:** Вместо динамического стека программист или компилятор заранее создает массив структур (контекстов) в памяти: `enter1, enter2...` под каждый возможный уровень вложенности.
- **Как работает:** Функция принимает указатель на конкретную структуру `struct swap* base` . Каждому вызову принудительно отдается свой фиксированный кусочек памяти `base->tmp` .
- **Ограничение:** Количество рекурсивных шагов жестко ограничено числом заранее созданных структур.

У второго подхода плюс, что нам не нужен аппаратный стек, не нужна никакая аппаратная поддержка там и все такое.

207. В чём заключается проблема распределения регистров и как работает алгоритм раскраски?

Эта проблема возникает на этапе компиляции, когда язык высокого уровня нам надо перевести в инструкции. И нам бы очень хотелось, чтобы код выполнялся быстро, в частности с минимальным числом обращений в память. Т.е. было бы классно, чтобы у нас все крутилось и вычислялось в регистрах. Но регистров у нас не всегда

так много, как хотелось бы и более того не всегда они одинаковы, значит надо как-то распределять этот орагниченный ресурс.

Один из вариантов распределения - на этапе компиляции будем считать, что у нас бесконесное количество регистров, не ограничиваем себя в этом. Далее каждый такой регистр станет вершиной графа, а связаны вершины будут в том случае, если во время работы программы есть момент, когда нам нужны значения обеих переменных. Далее решается задача раскраски графа, где разные цвета означают разные физические регистры.

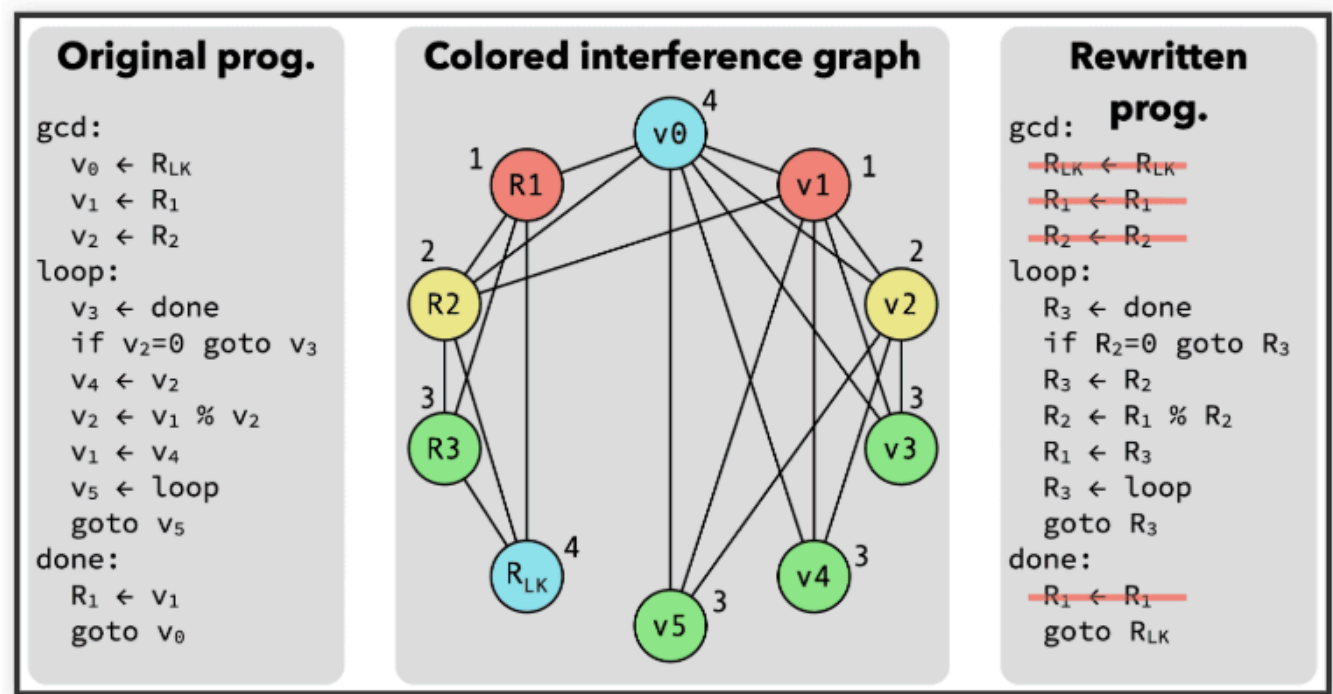
Формальное правило: Ребро между двумя виртуальными переменными v_i и v_j проводится тогда и только тогда, когда существует такая точка (инструкция) в программе, где **обе** эти переменные **одновременно** являются «живыми» (live).

Что значит «живая переменная» (Formal Definition)

Переменная v называется **живой** в точке программы P , если выполняются два условия одновременно:

1. Значение переменной v было записано **до** точки P .
2. В коде **после** точки P есть хотя бы один путь, на котором значение этой переменной будет **считано (использовано)** до того, как в неё запишут новое значение.

Интервал времени (или кусок кода) между первой записью в переменную и её самым последним использованием называется **интервалом активности (Live Range)**.



полиморфизм и замыкания? Пример на Smalltalk.

В объектно-ориентированной философии (например, в Smalltalk) условный оператор `if-else` полностью удален из синтаксиса языка. Вместо него логика ветвления реализуется за счет динамической диспетчеризации (вызова методов) и упаковки кода в анонимные функции.

- **Классы вместо булевых типов:** Истина (`True`) и Ложь (`False`) - это два самостоятельных класса-наследника, унаследованных от общего абстрактного класса `Bool`.
- **Полиморфные методы:** В обоих классах определен метод `ifTrue:ifFalse:`, но их внутренняя реализация отличается:
 - В классе `True` метод жестко запрограммирован выполнить **только первый** переданный блок кода (замыкание) и проигнорировать второй.
 - В классе `False` метод наоборот — выполняет **только второй** блок кода.
- **Роль замыканий:** Код каждой ветки («тогда» и «иначе») оборачивается в анонимные функции. Это необходимо, чтобы изолировать инструкции и не выполнять их раньше времени.

КАКУЮ РОЛЬ ТУТ ИГРАЮТ ЗАМЫКАНИЯ:

Представь, что у нас есть вымышленная функция `my_if`, и мы пытаемся вызвать её в обычном языке (например, в Python или Си):

python

```
my_if(17 * 13 > 220, print("Выиграл!"), print("Проиграл!"))
```

Используйте код с осторожностью.



Как рассуждает процессор, когда видит эту строчку? В обычных языках действует жесткое правило: **нельзя зайти внутрь функции, пока не вычислены все её аргументы**.

Процессор идет слева направо по строке:

1. Вычисляет условие: `17 * 13 > 220`. Получает `True`.
2. Видит второй аргумент: `print("Выиграл!")`. Процессор **тут же выполняет его**, и на экран выводится надпись «*Выиграл!*».
3. Видит третий аргумент: `print("Проиграл!")`. Процессор **тоже выполняет его**, и на экран выводится «*Проиграл!*».
4. И только теперь, когда оба текста уже напечатаны на экране, процессор заходит внутрь функции `my_if(True, ..., ...)`.

Получился бред: у нас сработали **обе ветки сразу**. Смысл условного оператора полностью уничтожен.

Как квадратные скобки (замыкания) в Smalltalk спасают ситуацию?

В Smalltalk этот же код пишется в квадратных скобках. Скобки — это и есть команда: «Запакуй этот код в замыкание и не выполняй его прямо сейчас!».

smalltalk

```
(17 * 13 > 220) ifTrue: [ print("Выиграл!") ] ifFalse: [ print("Проиграл!") ]
```

Используйте код с осторожностью.



Теперь процессор идет по строке совершенно иначе:

1. Вычисляет условие и получает объект `True`.
2. Видит код `[ print("Выиграл!") ]`. Благодаря скобкам он **не выполняет принт**. Он просто создает в памяти «коробочку» (замыкание), внутри которой лежит эта инструкция.
3. Видит код `[ print("Проиграл!") ]`. Он точно так же создает вторую «коробочку» в памяти.
4. Передает обе закрытые коробочки объекту `True`.

Объект `True` принимает эти две коробки. Поскольку он по своей природе является «Истиной», он знает, что вторую коробку нужно выкинуть в мусорку (она никогда не откроется), а вот у первой коробки нужно сорвать пломбу (вызвать метод `.value`).

Процессор открывает первую коробку, и только в этот самый момент на экран лениво выводится надпись «*Выиграл!*».

209. Что такое ленивые вычисления и как они связаны с реализацией условного оператора?

Ленивые вычисления (Lazy Evaluation) - это стратегия вычисления, при которой выражение вычисляется не в момент его создания, а только тогда, когда его результат **реально потребовался** для работы программы. Например не вычислять аргумент функции, пока к нему не обратились внутри функции

Условный оператор по своей сути всегда должен быть ленивым, потому что выполняется или одна ветка или вторая. Нельзя пройти по обеим веткам, а оставить только результат одной из них. Поэтому без ленивых вычислений условный оператор теряет смысл

210. Как `async / await` связан с кооперативной многозадачностью на уровне языка программирования?

Механизм `async / await` — это синтаксический инструмент языка программирования, созданный специально для реализации **кооперативной многозадачности**.

Он позволяет нескольким задачам выполняться условно параллельно в рамках одного и того же физического потока.

Главный принцип кооперативной многозадачности — **добровольная передача управления**. В отличие от вытесняющей многозадачности ОС (где планировщик жестко прерывает поток в любой такт времени), в кооперативной модели задача работает до тех пор, пока сама не скажет: *«Я упираюсь в долгое ожидание (например, чтение файла или сетевой запрос), я временно отдаю процессорное время другим»*.

211. Что такое FPGA и каковы его преимущества и недостатки?

FPGA - Field-Programmable Gate Array (он же ПЛИС - программируемая логическая интегральная схема) это вычислитель, являющийся промежуточным решением между **ASIC - Application-Specific Integrated Circuit** и **CPU - Central Processor Unit**.

В чем концепция - сри это супер универсальный вычислитель, реализующий временные вычисления. Он относительно медленный, содержит много логики для интерпретации и выполнения команд и вычислительный процесс в нем представлен как алгоритм, последовательность команд. Это одна крайность. Другая крайность это asic - супер специализированная под задачу огромная комбинационная схема, за счет того, что ей не надо поддерживать универсальность она может быть реализована супер эффективно.

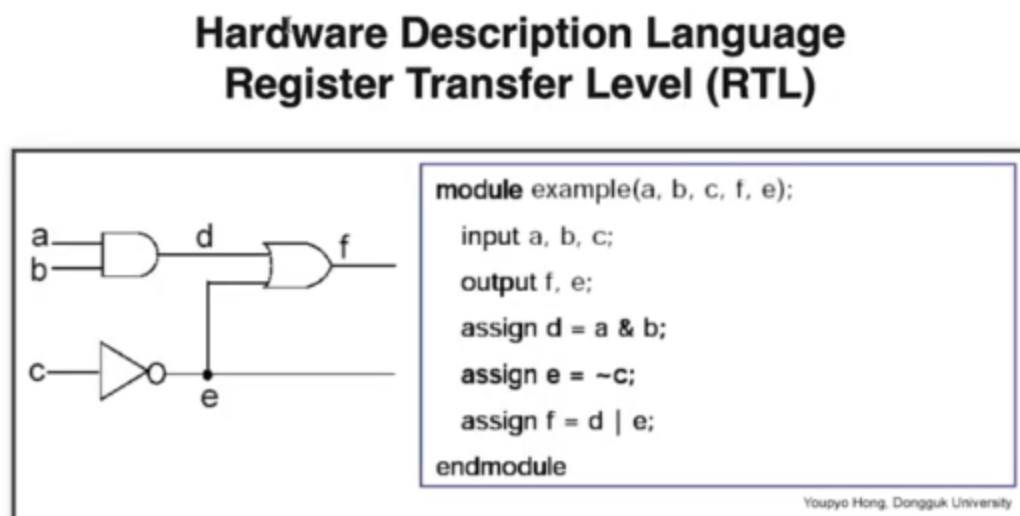
Но подход asic это очень сложно, схему надо спроектировать, произвести и это все нетипичное и это все под заказ. Поэтому есть компромисс - fpga. Это такая схема, которая может быть сконфигурирована под задачу, затем реконфигурирована и поэтому нам не надо производить ничего с нуля. Покупаем готовый типовой чип, настраиваем его под работу с конкретной задачей и радуемся как быстро все выполняется (чуть медленнее все же чем asic но СИЛЬНО быстрее сри)

Преимущества - быстрее сри на порядки, на порядки же дешевле asic (для небольших партий)

Недостатки - если запускать огромное производство на 100500 миллионов штук, в пересчете на штуку дешевле все-таки выйдет asic чем fpga потому что ну asic это более специализированная штука, а затраты на его разработку распределятся за счет большой партии

212. Какие этапы включает в себя синтез для FPGA?

Вместо инструкций здесь у нас используется языки описания архитектуры (способ представления схемы в текстовом формате)



Дальше это преобразовывается, раскрывается в **Generic Boolean** т.е. в простую логическую схему, состоящие из комбинационных схем.

Дальше исходя из того, что у нас есть в чипе формируется физическое воплощение этой схемы

213. Как FPGA устроены внутри?

ПЛИС состоит из множества маленьких логических блоков, собранных в матрицы. Каждый блок умеет реализовывать небольшие таблицы истинности. Блоки мы можем произвольно соединять между собой.

Также есть блоки памяти (RAM) и подобие регистров + специальные блоки под частые задачи (например сумматоры, делители), чтобы не тратить на них кучу блоков.

Каждый логический блок мы конечно можем настроить, загрузив туда нужную таблицу истинности

214. Что такое HDL и RTL? Чем описание аппаратуры на HDL отличается от программирования на языках высокого уровня?

1. Что такое HDL и RTL?

- **HDL (Hardware Description Language — Язык описания аппаратуры):** Это специализированный язык программирования (самые популярные — Verilog и VHDL), который используется для моделирования, проектирования и верификации цифровых электронных схем (например, для FPGA или ASIC).
- **RTL (Register-Transfer Level — Уровень регистровых передач):** Это стиль (или уровень абстракции) написания кода на языке HDL. На уровне RTL разработчик явно описывает схему как **совокупность регистров** (триггеров для хранения состояния) и **комбинационной логики** (логических вентилей И, ИЛИ, НЕ), которая преобразует данные при их пересылке между этими регистрами от такта к такту.

Главные отличия описания аппаратуры на HDL от программирования на языках высокого уровня:

А. Модель времени и параллелизм (Главное отличие)

- **Языки высокого уровня:** Основаны на последовательном (временном) исполнении. Инструкции выполняются процессором строго по очереди, одна за другой (**линия 1** , затем **линия 2**). Параллелизм здесь искусственный (многопоточность на уровне ОС).
- **Языки HDL:** Описывают **пространственную структуру**. Все строки кода, описывающие независимые логические блоки, физически работают **одновременно и параллельно**. Ток течет по всем виртуальным проводам на чипе одновременно.

Б. Что является результатом компиляции

- **Языки высокого уровня:** Компилятор превращает код в **последовательность машинных инструкций** (бинарный код для CPU), которые будут поочередно загружаться в АЛУ процессора.
- **Языки HDL:** Процесс «компиляции» называется **синтезом (Synthesis)**. Код превращается в **Netlist (список связей)** — реальную физическую схему из транзисторов, логических вентилей, триггеров и проводов, которые будут вытравлены в кремнии (ASIC) или сконфигурированы в LUT-таблицах (FPGA).

В. Понятие переменной

- **Языки высокого уровня:** Переменная — это именованная ячейка в оперативной памяти, куда можно записать значение, а потом перезаписать его.
- **Языки HDL:** Переменная — это **физический объект**. Это либо **физический провод (wire)**, который просто передает электрический сигнал из точки А в точку Б, либо **физический триггер/регистр (reg)**, который запоминает уровень напряжения по фронту тактового сигнала.

215. Как соотносятся CPU и FPGA с точки зрения модели программирования, модели вычислений, модели исполнения и элементов микроархитектуры?

Ну тут я лучше чем нейронка не опишу, все по фактам раскидывает:

1. Модель программирования (Programming model)

Уровень того, как разработчик описывает задачу.

- **CPU — Императивное программирование (Imperative programming):** Код пишется на классических языках (C/C++). Программа представляет собой **последовательность инструкций (команд)**, пошагово меняющих состояние памяти. Мы явно описываем *как* и в каком порядке процессор должен выполнять шаги.
- **FPGA — Декларативное программирование (Declarative programming):** Код пишется на языках описания аппаратуры (VHDL/Verilog). Разработчик не пишет алгоритм, а **декларирует (описывает) саму цифровую схему**: как физически соединены между собой провода, регистры и логические элементы.

2. Модель вычислений (Computation model)

Уровень организации вычислений во времени и пространстве.

- **CPU — Временные вычисления (Temporal computation):** Одно фиксированное железное ядро (АЛУ) выполняет разные задачи **последовательно во времени**. Сначала АЛУ считает одну операцию, в следующем такте на этом же АЛУ считается другая операция.
- **FPGA — Пространственные вычисления (Spatial computation):** Каждая операция из вашего алгоритма получает **свой собственный физический кусок железа** в пространстве чипа. Все эти вычислительные узлы работают **одновременно и параллельно**. Данные просто «текут» по схеме (Dataflow).

3. Модель исполнения (Execution model)

Уровень того, как железо управляет потоком задач на ходу.

- **CPU — Динамическое планирование и последовательное исполнение (Dynamic-scheduling Sequential-execution):** На вход подается один последовательный поток команд. Процессор внутри себя на ходу (**динамически**) решает, как распределить команды по конвейеру, пытается угадать переходы (Branch Prediction) и переупорядочить инструкции (Out-of-Order).
- **FPGA — Статическое планирование и потоковое исполнение (Static-scheduling Dataflow-execution):** Никакой динамики на ходу нет. Вся схема жестко спланирована заранее, еще на этапе компиляции (синтеза проекта). Железо работает как конвейер на заводе — данные поступают на входы и волной идут через логические элементы (**Dataflow**).

4. Микроархитектура (Micro-architecture)

Физические кирпичики, из которых состоит чип.

- **CPU:** Состоит из универсальных фиксированных блоков:
 - **ALU** (арифметико-логические устройства)
 - **Register File** (регистровые файлы для хранения промежуточных данных)
 - **Branch Predictors** (предсказатели переходов)
 - **Cache** (сложная иерархия кэш-памяти)
- **FPGA:** Состоит из огромного регулярного массива мелких настраиваемых блоков:
 - **CLB** (Configurable Logic Blocks) — те самые блоки, внутри которых сидят таблицы истинности (LUT), триггеры и мультиплексоры.
 - **Switch box** — программируемые матрицы коммутации (ключи), которые соединяют CLB между собой нужными дорожками.
 - **Block memory** (BRAM) — встроенные мелкие блоки памяти, разбросанные по кристаллу.

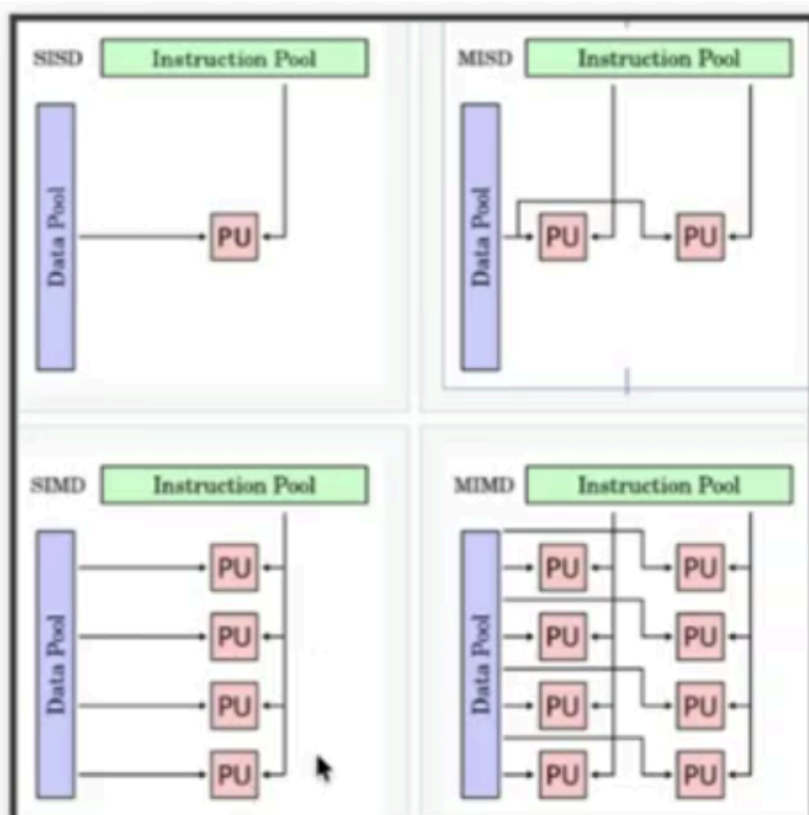
Микроархитектуру вообще достаточно глупо сравнивать, она очень разная и очень специфичная для каждого из двух подходов.

216. Что представляет собой классификация Флинна и какие классы архитектур она выделяет? Примеры.

Классификация Флинна - попытка классифицировать параллельные процессоры.

Каждому процессору сопоставляется два параметра:

- Количество потоков инструкций
- Количество потоков данных



Тут вот PU это processor unit, некоторые блоки которые могут принимать инструкцию, принимать данные и делать результат. Без параллелизма.

Выделяются 4 группы:

Single/Multiple Instruction Simple/Multiple Data

(соответственно SISD, SIMD, MISD, MIMD)

SIMD - когда одна операция запускается параллельно сразу для нескольких наборов данных. Подходит для однотипной обработки: для нейросетей, для графики, для матриц. Пример: GPU

MISD - не получил широкого распространения, потому что очень мало есть задач когда для одних данных нужно посчитать несколько разных инструкций. Мооожет применяться в системах с супер требованием к надежности управления, когда одни данные прогоняются через несколько

MIMD - практически все процессора попадают сюда, они могут обрабатывать много потоков инструкций и у каждой свои входы по данным. Такие, сильные и независимые отдельные блоки процессорные

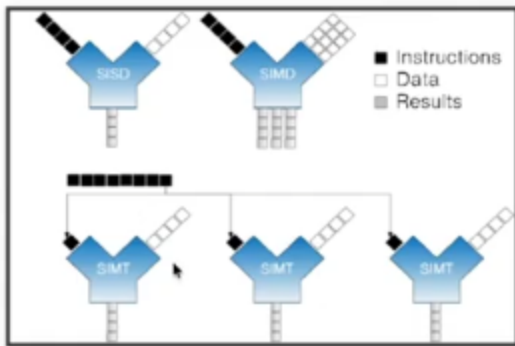
Супер простая классификация, но не очень классная, потому что ну 80% процессоров относятся к MIMD это не очень интересно так классифицировать

217. Что такое SIMD и SIMT архитектуры? В чём их различия?

Что такое lockstep execution?

SIMD - когда одна операция запускается параллельно сразу для нескольких наборов данных. Подходит для однотипной обработки: для нейросетей, для графики, для матриц.

Но вместе с тем есть расширение этой архитектуры - **SIMT** (Single Instruction, Multiple Threads)



Было как сверху стало как снизу. Зачем это сделано? Чтобы обойти основное ограничение которое есть в SIMD - это плохая работа с ветвлениями.

Тут мы можем решать какие инструкции каким вычислителям отправлять, а каким не отправлять. Потому что теперь это не один вычислитель с кучей входов и кучей выходов, это теперь куча вычислителей.

218. Что представляет собой классификация Дункана и чем она отличается от классификации Флинна? Какие проблемы она решает? Что включает верхний уровень классификации?

Дункан решил что низкоуровневый параллелизм никому не интересен, потому что внешне оно выглядит будто инструкции выполняются линейно. При этом базировался он все равно на таксономии Флинна, но расширил и включил новшества в отрасли. Разбил MIMD на составные части.

Решает проблему излишней простоты классификации Флинна, которая несколько синтетическая и не погружается глубоко в детали реализации процессоров.

Он делит архитектуры на три большие группы (это верхний уровень классификации):

Синхронные архитектуры

Класс процессорных архитектур, обладающих единым/централизованным/синхронным управлением, определяющим поведение всего процессора в целом.

- Векторные архитектуры.
- SIMD архитектуры:
 - Процессорные массивы.
 - Ассоциативные массивы.
- Систолические массивы.
- Совмещение памяти и вычислений*.

MIMD архитектура

Ключевой вопрос — организация памяти:

- разделяемая,
- распределённая.

Аналогия из современных веб приложений:

- синхронизация через единую базу данных;
- микросервисная архитектура.

219. Какие основные типы синхронных архитектур выделяются по классификации Дункана?

Синхронные архитектуры

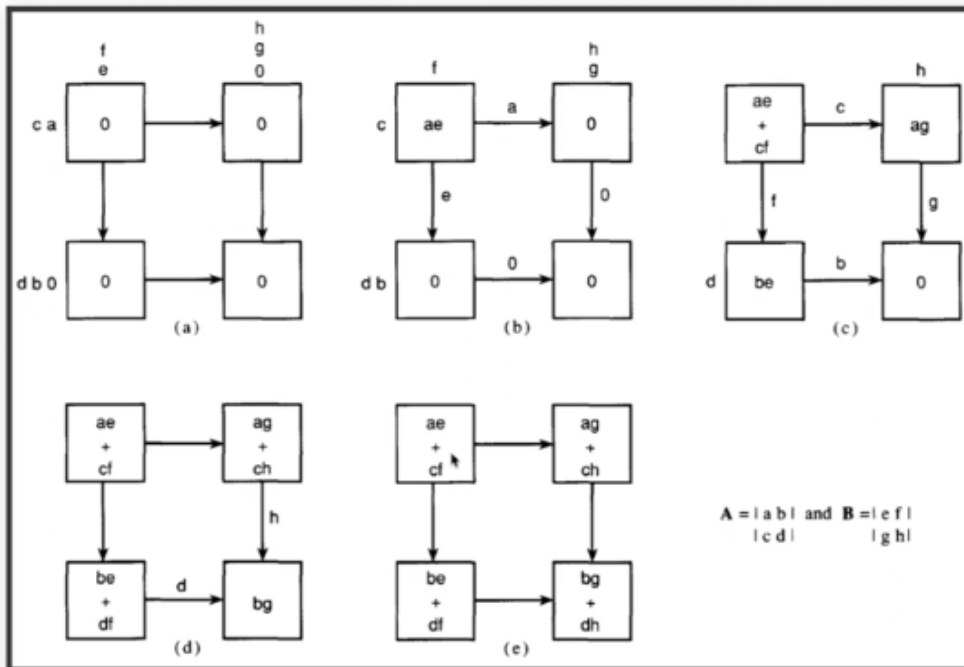
Класс процессорных архитектур, обладающих единым/централизованным/синхронным управлением, определяющим поведение всего процессора в целом.

- Векторные архитектуры.
- SIMD архитектуры:
 - Процессорные массивы.
 - Ассоциативные массивы.
- Систолические массивы.
- Совмещение памяти и вычислений*.

Векторные архитектуры поддерживают операции с веторами, т.е. сразу с группой скаляров

SIMD - рассмотрено ранее. Ассоциативные процессоры работают на ассоциативной памяти, которая может быстро искать данные по определенным параметрам

Систолические архитектуры:



Короче вот так можно перемножать матрицы, у нас есть вычислительные блоки которые **по внешнему общему такту** друг с другом общаются, передают по цепочке значения, и так у нас распространяются вычисления. Минус в том что это не для каждой задачи так просто сделать. Зато для умножения матриц это сууупер эффективно.

Совмещение памяти и вычислений - когда мы решаем, что упираться в скорость доступа к памяти это грустно и мы прямо в память вшиваем вычислители, которые прямо в памяти по команде процессора будут делать какие-то простые вычисления. Например, при перемножении матриц нам не надо будет тыщу раз ходить туда обратно в память, мы делаем вычисления "на месте", где "место" это чип памяти.

220. Какие принципы лежат в основе работы векторных архитектур и где они применяются?

см. вопрос 220

Применяют в системах, где очень актуальны векторные операции, для ускорения графики, обработка сигналов, спектральный анализ

221. Как работают ассоциативные массивы и где они применяются?

см. вопрос 220

Применяют в сетевом оборудовании (задача поиска в таблицах маршрутизации там очень актуальна)

222. Как работают систолические архитектуры и где они применяются?

см. вопрос 220

Применяют для получения аппаратного очень быстрого перемножения матриц (задачи машинного обучения, цифровая обработка изображений и видео)

223. Какие основные типы MIMD-архитектур выделяются по классификации Дункана?

MIMD архитектура

Ключевой вопрос — организация памяти:

- разделяемая,
- распределённая.

Аналогия из современных веб приложений:

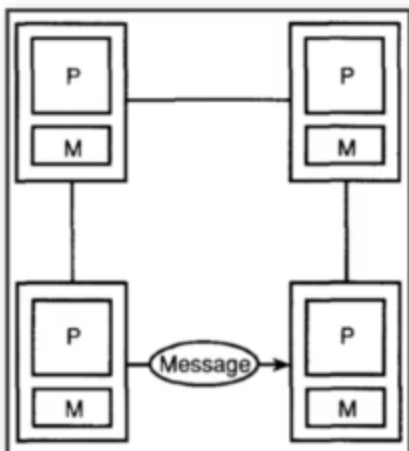
- синхронизация через единую базу данных;
- микросервисная архитектура.

224. Что из себя представляют архитектуры с разделяемой памятью? Каковы их особенности?

Память общая, у каждого процессора может быть свой кеш, но все вместе они делят одно общее пространство памяти. Соответственно возникает куча проблем того, что процессоры начинают конкурировать за память, кеши надо как-то согласовывать. Однако в такой системе проще координировать работу процессоров потому что они могут общаться через память

А еще это удобно для программистов

225. Что из себя представляют архитектуры с распределённой памятью? Каковы их особенности?



У каждого процессора своя отдельная память, а общаются они явно через сообщения между процессорами. Это кратно все усложняет, потому что надо выстраивать всю эту коммуникацию между ними, заморачиваться с протоколами и сообщениями. А еще есть огромное количество топологий подобной сети процессоров, у каждой свои особенности

Зато здесь можно обеспечить масштабирование, процессор никогда не конкурирует за память

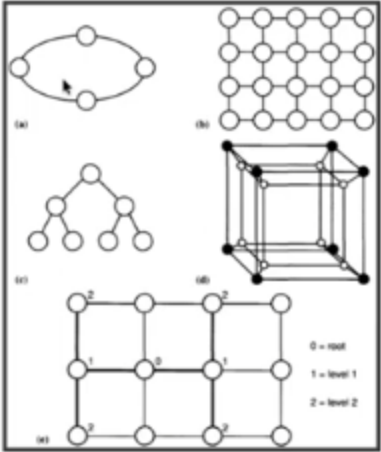
1. **Кольцевая** топология + [хорды]. Мало процессоров, передача данных не доминирует.

2. **Ячеистая** / сетка. Соединены соседи + [диагональные] + [кольцевые]. Топологическое соответствие матричным алг.

3. **Дерево**. Алгоритмы поиска и сортировки, обработка изображений, потоковые и редуccionные машины.

4. **Гиперкуб**. Коммуникационный диаметр $\log_2 N$.

5. **Реконфигурируемые**. Фиксируем сеть конфигурацией, а не маршрутом



226. Какие основные типы MIMD парадигм выделяются по классификации Дункана?

MIMD парадигма (paradigm architectures)

Имеется в виду парадигма программирования или точнее — модель вычислений (MoC).

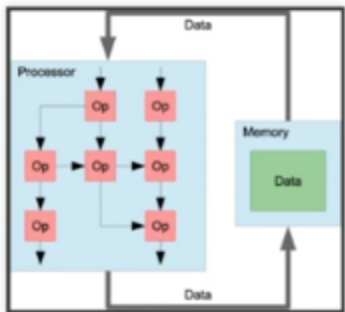
- MIMD/SIMD
- Поточковая архитектура (Dataflow)
- Редуccionная архитектура
- Wavefront

227. Что представляет собой MIMD/SIMD парадигма и где она применяется?

Короче тут чтобы обеспечить MIMD мы берем несолько SIMD процессоров которые умеют обрабатывать один поток инструкций. Но за счет того, что их просто несколько получается MIMD.

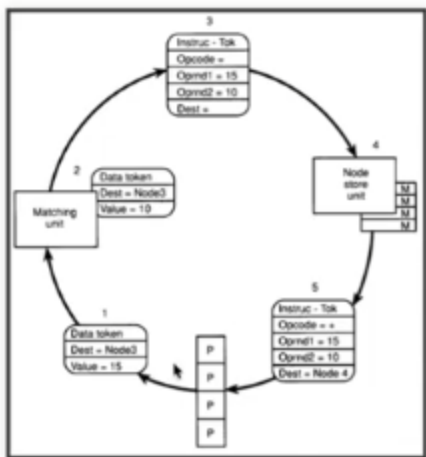
Тут это сразу влечет за собой иерархию, процессоры выстраивают в виде дерева, где корень как главный спускает подчиненным листьям задачи

228. Как работает архитектура потоков данных (dataflow)?
Подходы к реализации.



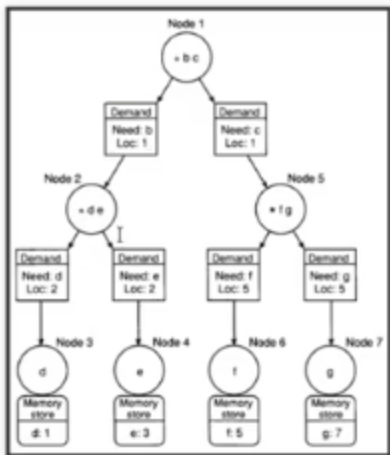
Напоминает систолическую архитектуру, только в качестве вычислительных блоков тут полноценные процессоры, которые работают асинхронно (у них свое тактирование) и могут передавать данные друг другу.

Можно реализовать как показано на верхней картинке, т.е. память одного процессора это выход другого процессора. А можно еще сделать вот так



Когда процессорные ядра соединены по кольцу и по мере продвижения данных вычисляют что надо и прокидывают дальше

229. Как работает редукционная архитектура? Каким образом она обеспечивает параллелизм?



Если в архитектуре потоков данных мы идем по графу от входных данных к выходным, то здесь все наоборот. Сначала у нас возникает потребность в результате (верхняя нода), мы понимаем что для этого надо вычислить результаты поддеревьев справа и слева, они в свою очередь спускаются еще ниже, пока на уровне листьев мы не получим исходные данные.

Получается некоторая концепция ленивых вычислений, потому что в процессе вычисления нам не нужна какая-то ветка, она и не будет ничего вычислять. Можно сэкономить на энергопотреблении вот

230. Как работает wavefront архитектура? В чём её отличия от систолической архитектуры?

Очень похоже на систолическую архитектуру и на dataflow. Отличие в том, что здесь у нас нет четкой общей синхронизации (нет общего тактового сигнала), передача данных идет по готовности. Отличие от dataflow в более строгой топологии и более топорном исполнении в том плане что в dataflow пакет может пройти через несколько узлов, пока не достигнет адресата

231. Что такое CGRA-процессоры и какие возможности они предоставляют?

CGRA (Coarse-Grained Reconfigurable Architecture)-процессоры - семейство процессорных архитектур, находящееся посередине FPGA и DSP процессорами. Идея в том, чтобы создать процессор, который будет иметь много специализированных вычислительных блоков, которые будем коммутировать между собой.

Отличие от ПЛИС в том, что ПЛИС заточен под реализацию применю любой цифровой схемы, а здесь более крупноячеистая структура с большими блоками, заточенными под какие-то прикладные задачи. Более того, можно комбинировать подход ПЛИС, который занимается пространственными вычислениями и подход CPU который занимается временными вычислениями. Получается часть задачи посчитать как "комбинационную схему" а часть посчитать через парадигму инструкций.

Тут немного страдает универсальность из-за того, что вычислительные блоки достаточно большие и достаточно сильно заточены под узкие задачи

232. В чём различие между пространственными (spatial) и временными (temporal) вычислениями на примере CGRA?

Смотри вопрос 232 второй абзац

233. Какие подходы существуют к решению проблемы Memory Wall в рамках ASIC и CGRA?

Memory Wall - дисбаланс между высокой скоростью работы вычислительных ядер и ограниченной пропускной способностью памяти. Вычислители простаивают, ожидая данные.

Когда мы уходим от классической фон-неймановской архитектуры у нас появляются разные способы как эту проблему можно смягчить (фундаментально решить эту проблему так просто нельзя, так что слово именно смягчить). Одни из таких подходов:

- **Совмещение памяти и вычислений** - (подробнее см 219 вопрос) когда мы без загрузки операндов можем выполнять операции прямо с ячейками памяти за счет установки вычислительных узлов непосредственно в чип памяти
- **Распределенная память** - когда каждый вычислительный элемент имеет свой блок памяти, или в целом вычислительные ядра имеют каждый полностью свою память
- **Ассоциативная память** - позволяет очень быстро искать данные по данным (а не по адресу, как обычная память) что колоссально снижает количество обращений к памяти